

CLD400

Developing with SAP BTP ABAP Environment

PARTICIPANT HANDBOOK
INSTRUCTOR-LED TRAINING

Course Version: 14
Course Duration: 3 Day(s)
e-book Duration: 6 Hours 30 Minutes
Material Number: 50156699

SAP Copyrights, Trademarks and Disclaimers

© 2021 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

These materials may have been machine translated and may contain grammatical errors or inaccuracies.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.

Typographic Conventions

American English is the standard used in this handbook.

The following typographic conventions are also used.

This information is displayed in the instructor's presentation	
Demonstration	
Procedure	
Warning or Caution	
Hint	
Related or Additional Information	
Facilitated Discussion	
User interface control	Example text
Window title	Example text

Contents

vii	Course Overview
1	Unit 1: About this Handbook
2	Lesson: Explaining, how the Recent Rebranding Displays in this Material
8	Unit 2: Introduction to SAP BTP, ABAP Environment
9	Lesson: Looking at the Big Picture
18	Lesson: Introducing a Modern ABAP Language
21	Lesson: Obtaining an ABAP Environment
27	Lesson: Explaining ABAP Development Tools in Eclipse (ADT)
41	Unit 3: ABAP Language Versions
42	Lesson: Explaining ABAP Language Versions
46	Lesson: Explaining ABAP Features not Supported in SAP BTP, ABAP Environment
48	Lesson: Explaining Released APIs in ABAP Environment
52	Unit 4: ABAP RESTful Programming Model
53	Lesson: Explaining ABAP RESTful Programming Model
56	Lesson: Explaining ABAP Core Data Services (CDS)
61	Lesson: Creating Data Models
65	Lesson: Explaining Business Services
68	Lesson: Defining Business Object Behavior
75	Lesson: Enabling Draft
83	Unit 5: Side-by-Side Extensions between SAP BTP, ABAP and Remote APIs
84	Lesson: Explaining Connectivity Services
87	Lesson: Accessing Internet-Facing API by URL
89	Lesson: Consuming an SAP API from the Business Hub API
94	Lesson: Consuming an On-Premise OData Service
96	Lesson: Consuming an On-Premise RFC
98	Lesson: Using a Remote Client Proxy
105	Unit 6: Custom Code Transformation
106	Lesson: Explaining Custom Code on-Premise
109	Lesson: Performing Custom Code Transformation to SAP BTP, ABAP Environment
115	Lesson: Using abapGit

125	Unit 7:	Transport of Software Components
126		Lesson: Transporting Software Components
131	Unit 8:	Roadmap of SAP BTP, ABAP Environment
132		Lesson: Exploring the Roadmap of SAP BTP, ABAP Environment

Course Overview

TARGET AUDIENCE

This course is intended for the following audiences:

Development Consultant

Business Process Owner/Team Lead/Power User

Developer

Solution Architect

UNIT 1

About this Handbook

Lesson 1

Explaining, how the Recent Rebranding Displays in this Material

2

UNIT OBJECTIVES

Explaining, how the recent rebranding displays in this material

Explaining, how the Recent Rebranding Displays in this Material



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explaining, how the recent rebranding displays in this material

Rebranding

Rebranding Information

Throughout the text and the figures, rebranding references shall be understood as follows:

All existing SAP Cloud Platform services are/are being renamed with the 'SAP' branding. On the product level, the SAP Cloud Platform services are organized into two suites, renamed to: SAP Integration Suite and SAP Extension Suite.

Background

SAP Cloud Platform evolves

Our cloud platform is one of the core pillars powering SAP Business Technology Platform (SAP BTP) and has become a key element within our broader platform offering, rather than a stand-alone offering. Its strong integration and extension capabilities are now available as services that run on SAP BTP. These capabilities are now called **SAP Integration Suite and SAP Extension Suite**

Application development and integration

Application development and integration capabilities, previously available in SAP Cloud Platform, are now cloud services that run on SAP BTP – providing users with a cloud environment to develop, manage, extend, and deliver applications.

Enterprise application integration

SAP Integration Suite, an integration platform as a service (iPaaS), enables users to implement data, application, API, and process integration projects involving any combination of cloud-resident and on-premise end points.

Application development and extensions

SAP Extension Suite services can help you build and enhance your solutions, optimize your business processes, and create an engaging digital experience. With more than 80 services available, SAP can help you boost development productivity and efficiency.

SAP Integration Suite and SAP Extension Suite are powering SAP Business Technology Platform

As a pivotal part of SAP's Business Technology Platform portfolio, **SAP Integration Suite and SAP Extension Suite** are the integration, extension, and innovative building of business applications with agility, flexibility, and choice, formerly known as SAP Cloud Platform

Integration and SAP Cloud Platform . It lets customers integrate, extend, and innovate their data, processes, and apps, and enables partners to bring unlimited innovation to the ecosystem:

SAP Integration Suite

Provides ready-built integration technology and content for SAP and non-SAP, for cloud and hybrid architectures, which help our customers to integrate everything, everywhere, by supporting a wide variety of synergic integration approaches.

SAP Extension Suite

Provides an cohesive set of services and tools to create and run processes and applications that integrate easily and securely with any SAP system, helping developers and process experts to be efficient, and to create the best possible digital experiences across all channels.

Multicloud Readiness

While the SAP Integration & Extension Suites focus on solving dedicated business needs, the underlying foundation offers a secure, multicloud, future-proof underpinning that embraces all major hyper-scaler infrastructures and cloud-based, native technologies.

Frequently asked Questions

What is happening with the SAP Cloud Platform brand?

We are removing SAP Cloud Platform as a brand name. However, the technical capabilities of SAP Cloud Platform will remain as important as ever. Concretely, this means the services under the former SAP Cloud Platform brand continue to exist, just without "Cloud Platform" in the name. For example, "SAP Workflow Management" and "SAP Enterprise Messaging."

How does cloud platform align with SAP Business Technology Platform?

As SAP Business Technology Platform has progressed, we have found it better for our customers to align our integration and extension capabilities to one platform. Our integration and extension capabilities are key to powering SAP Business Technology Platform (SAP BTP). These capabilities, along with database and data management, analytics, and intelligent technologies, are critically important components of our holistic platform that complement SAP and third-party applications to deliver agile and comprehensive digital transformation in the cloud.

What happens to our existing customers that are using SAP Cloud Platform?

We are sunsetting SAP Cloud Platform as a brand name; however, the integration and extension capabilities of SAP Cloud Platform will continue to be available as products and services under SAP Integration Suite and SAP Extension Suite. The functionality for our customers will remain unchanged. What customers will notice will be the changes in the service wherever the brand name "SAP Cloud Platform" is currently being referred to.

What is the impact on SAP Cloud Platform Trial and SAP Cloud Platform Cockpit?

Going forward, the trial will be renamed SAP BTP trial and the cockpit will be renamed SAP BTP cockpit. There is no change in capabilities or user experience in the trial and the cockpit.

For more information:

BTP:

<https://www.sap.com/products/business-technology-platform.html>



Application development:

<https://www.sap.com/products/application-development-integration.html>



SAP Integration Suite:



SAP Extension Suite:



SAP Cloud Platform:



Note:

This handbook is re-branded, but it might happen, that not all figures and screenshots display the recent branding.



LESSON SUMMARY

You should now be able to:

Explaining, how the recent rebranding displays in this material

Learning Assessment

1. BTP stands for:

Choose the correct answer.

- ☐ **A** Best Technology Platform
- ☐ **B** Business Technology Performance
- ☐ **C** Business Technology Platform

Learning Assessment - Answers

1. BTP stands for:

Choose the correct answer.

- ☐ **A** Best Technology Platform
- ☐ **B** Business Technology Performance
- ☒ **C** Business Technology Platform

This is correct.

UNIT 2

Introduction to SAP BTP, ABAP Environment

Lesson 1

Looking at the Big Picture	9
----------------------------	---

Lesson 2

Introducing a Modern ABAP Language	18
------------------------------------	----

Lesson 3

Obtaining an ABAP Environment	21
-------------------------------	----

Lesson 4

Explaining ABAP Development Tools in Eclipse (ADT)	27
--	----

UNIT OBJECTIVES

- Explain the SAP BTP and ABAP Environment
- Introduce a modern ABAP language
- Obtain an ABAP environment
- Explain ABAP Development Tools in Eclipse (ADT)

Unit 2

Lesson 1

Looking at the Big Picture



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explain the SAP BTP and ABAP Environment

Digital Platform for Intelligent Enterprises

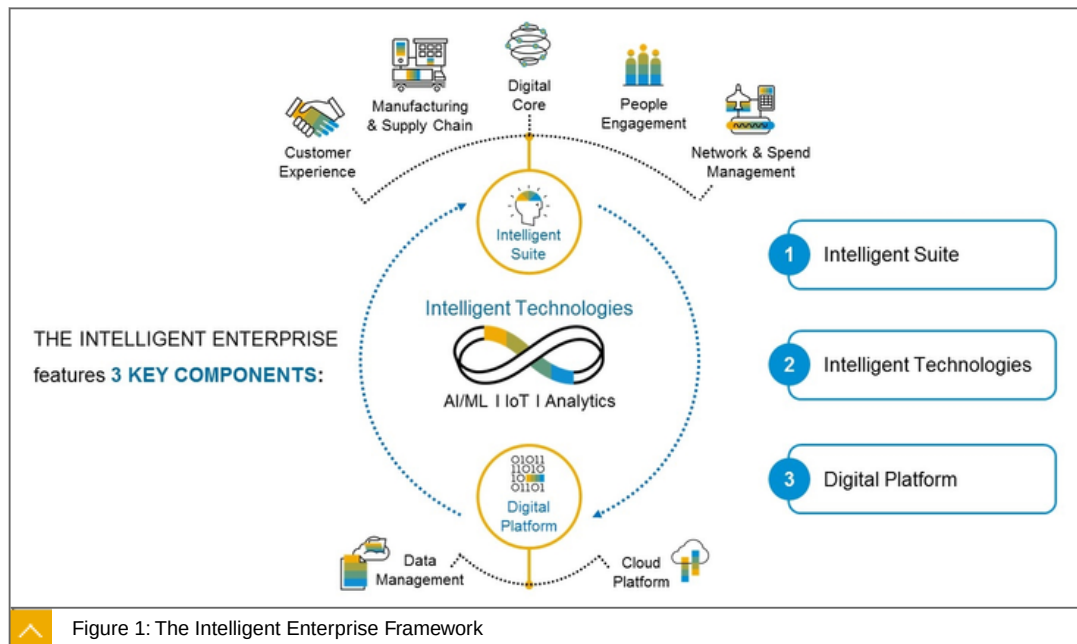


Figure 1: The Intelligent Enterprise Framework

The Intelligent Enterprise is a strategy that allows you to rapidly transform data into insight – feeding process automation, innovation, and optimal experiences. The Intelligent Enterprise has three key components:

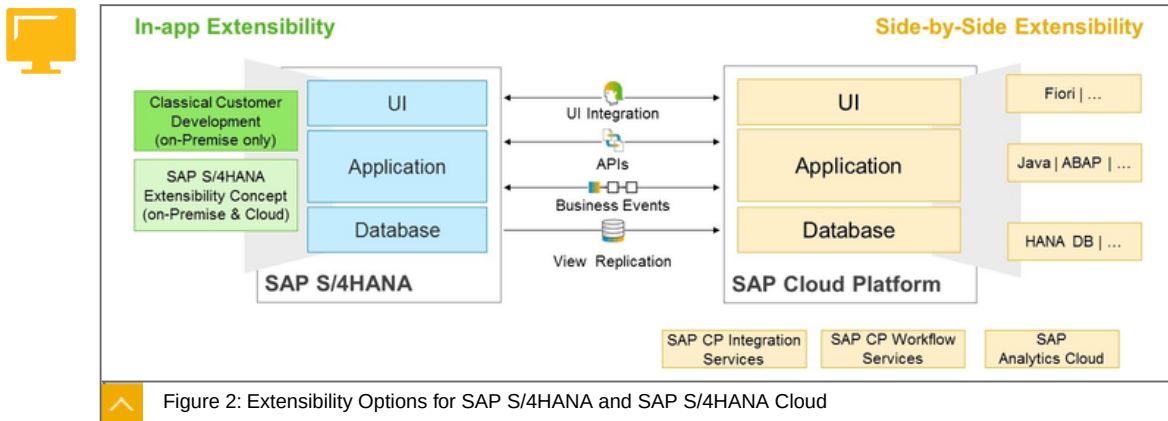
1. **Intelligent Suite** which consists of Digital Core (read SAP S/4HANA), CRM, SRM and suite of SAP products.
2. **Intelligent Technologies** consisting of Artificial Intelligence/Machine Learning (AI/ML), Internet of Things (IoT) and Analytics.
3. **Digital Platform** consisting of **Data Management** and **Cloud Platform**

The SAP Digital Enterprise Platform is comprised of a state of the art enterprise data storage and compute platform, critical data integration and application development tools, as well as best in class front office analytics tools. All of these capabilities are integrated via cloud services that enable organizations to increase value delivery to customers at significantly lower costs, through digital transformation and simplification.

Extensibility for Customers and Partners

Experience gained from various customer and partner projects has made clear that most SAP S/4HANA and SAP S/4HANA Cloud systems have been extended to fulfill customers' needs. This is due to the nature of enterprise business software, where best practices need to be aligned with custom practices, and standardized systems need to be integrated into an existing software landscape.

Therefore, extensibility must cover a broad spectrum of adaptation. This includes changes to software behavior that go beyond the capabilities of business configuration, data model extensions, data exposure and integration, layout changes to user interfaces (UIs) or forms and reports, and creation of new UIs and the customer's own applications.



In times of quickly changing business processes, extensibility projects need to be fast and efficient. For this reason, SAP S/4HANA Cloud supports either of the following approaches to extensibility:

In-app Extensibility– an approach taken directly within the software stack.

In-app extensibility focuses on adaptations created by key users, with its main features geared toward adding new fields or custom logic and adapting the standard UIs. Custom forms, reports and UIs, and custom business objects are also supported. Discovery of in-app extensibility capabilities is possible through the extensibility cockpit – a key-user app that allows developers to browse through data and process models and visualize extensibility options.

Side-by-Side Extensibility a broader approach utilizing all the capabilities of a platform-as-a-service (PaaS) platform.

Side-by-side extensibility on SAP BTP allows developers to build their own applications, enhancing and supplementing SAP S/4HANA and SAP S/4HANA Cloud. As shown in the figure, although the coupling of side-by-side extensions is quite loose, tight integration on all architecture layers is still provided by the appropriate APIs. So, side-by-side extensibility is mainly a story of APIs and suitable programming models. All APIs available for side-by-side extensibility are documented on SAP API Business Hub.

The most important use cases for side-by-side extensions are that of building completely new UIs based on the SAP Fiori user experience (UX) or integrating with other SAP applications. You can build completely new applications and business logic that run natively on SAP BTP, the apps being loosely coupled with the ABAP platform back end of SAP S/4HANA or other SAP products.

Both in-app and side-by-side extensibility depend on the stability of the supporting APIs and data models. Therefore, SAP introduced a release concept to ensure the stability of these objects dependent upon the usage contract.

In SAP S/4HANA Cloud, only released APIs are accessible. This applies to both in-app and side-by-side extensibility. To ensure the system's integrity, especially during upgrades, and to decrease operation cost and incidents, compliance with the release concept is enforced for all extensions of SAP S/4HANA Cloud. In on-premise environments with SAP S/4HANA, the use of released APIs is not mandatory but recommended, so you can smoothly bring your custom code toward cloud stability, step-by-step.

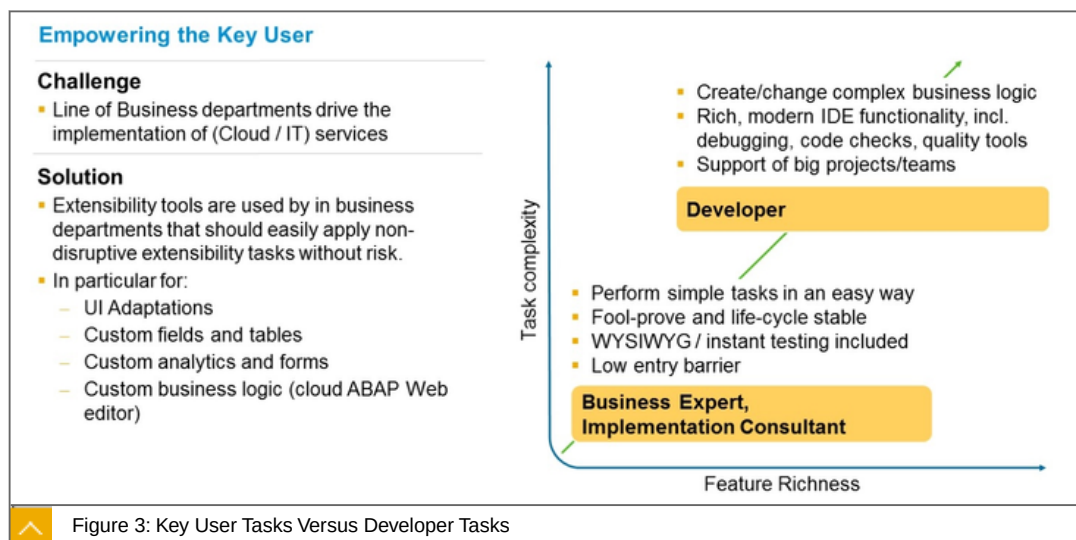
In addition to the release concept, a deprecation concept for APIs exists. APIs that must be deprecated for any reason (for example, a new API with a more complete feature set is available) are marked as being deprecated, and a substitute API is documented. In any case, APIs will be available for a long period of time before being deprecated, and they can still be actively used for two to three releases before they are withdrawn.

In-App Extensibility

In-app – or key-user – extensibility tools for SAP S/4HANA and SAP S/4HANA Cloud (as well as for other LoB solutions such as the SAP Marketing solution) follow the same principles as the leading cloud services. These include, for example, the cloud services provided with SAP SuccessFactors solutions, Ariba Network, SAP Customer Experience solutions, the SAP Business ByDesign solution, and software-as-a-service (SaaS) offerings from SAP's competitors. The tools:

- Are completely Web-based (that is, developed along the principles and technology used in created SAP Fiori apps) and targeted to key users who have a deep knowledge of the product itself but not necessarily a deep technical knowledge

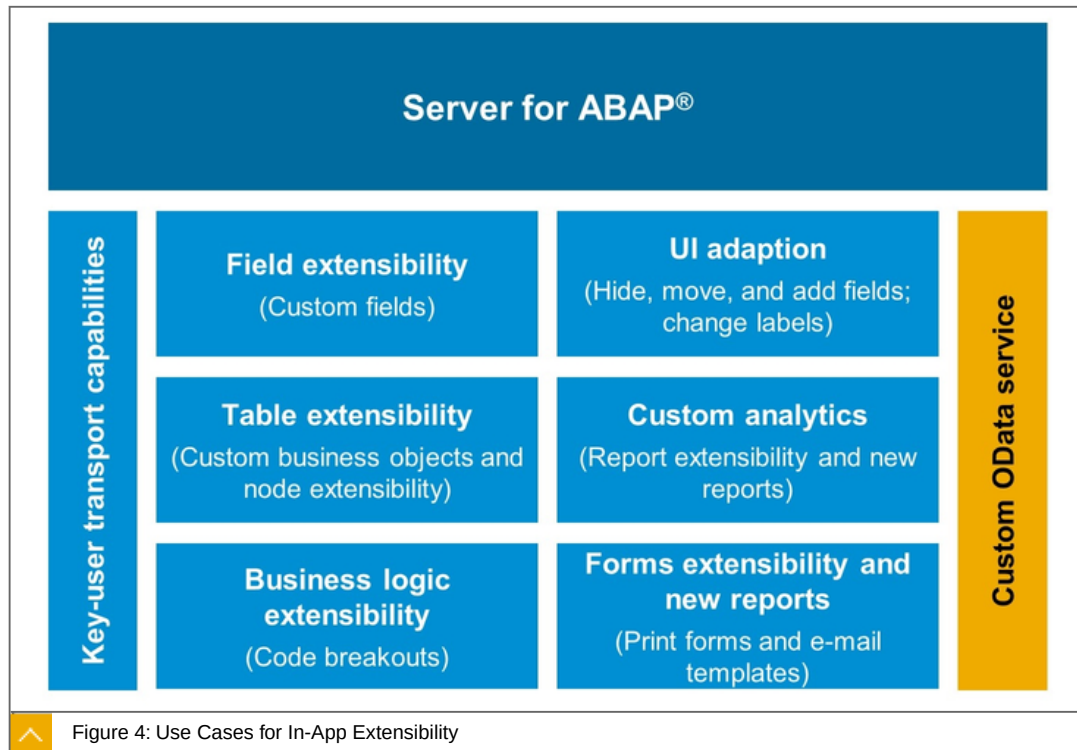
- Create lifecycle-stable artifacts. In other words, SAP software updates do not depend on adoptions of the extensions by the customer or partner. This means the tools are based on public, stable APIs and extension points and technology, so that they will continue to work without requiring manual activities to be performed after the next upgrade



The in-app extensibility options do not use a coding approach, but a what-you-see-is-what-you-get (WYSIWYG) approach. This suits key users who know a process very well but are not very familiar with coding.

Besides, there is the side-by-side extensibility that is done by developers using the ABAP Environment on SAP BTP that allows for coding for the implementation of complex business logic.

Extensibility features are designed for SAP S/4HANA Cloud but are also available for on-premise environments with SAP S/4HANA and have a variety of use cases. Due to its simplicity and strict cloud qualities, key-user extensibility has only limited possibilities. This is especially obvious when comparing the in-app extension features with the development capabilities of SAP BTP. Nevertheless, there are strong use cases for both the in-app and side-by-side approaches to extensibility.



User Interface Adaptation

In the adaptation mode (also called runtime authoring), the key user can adapt the UI layout in a modification-free way as follows:

- Hide fields in a form, table, or filter; hide groups and areas

- Rename labels

- Add a field to a UI from the field repository (OData service)

- Move a form field or UI group, create new groups, and move fields into a group; combine fields into one line and split combined fields

- Define new filter and table variants

Field Extensibility

Field extensibility refers to the capability of adding customer-specific fields (custom fields) to a business context of an application (for example, a sales order item or a customer address). Custom field additions can be made in a one-to-one relation or by adding SAP fields that are available in tables and structures in the “lower levels” of the application to services and UIs.

The tool for adding fields is key-user oriented, being launched directly from the UI of the extended application and providing guidance through a very simple extension dialog.

Field extensibility enables field extensions to be reflected in the APIs of extended applications. This applies for OData services, service-oriented architecture (SOA) services, intermediate documents (IDocs), and the business application programming interface (BAPI).

Custom Business Objects

Table, or node, extensibility refers to the capability of adding customer-specific fields to a business context of an application in a 1:1 or 1:n relationship. (This feature is in planning and not yet available.) In contrast to field extensibility, table extensibility is intended for the creation of new customer specific tables in the database. This would be accompanied by a set of API functionalities supporting create, read, update, and delete (CRUD) services through API classes, CDS views, and OData services.

Custom OData Services

SAP delivers ready-to-use APIs that are listed on SAP API Business Hub. In addition, you can create new OData services that can be used by external clients as follows:

- You can expose your custom business objects as an OData service for external clients (read and write)

- You can create a new custom CDS view and expose it as an OData service for external clients (read only)

Custom CDS views can be created on top of released views of the virtual data model (VDM), which is delivered by SAP, by using a Web-based key-user tool. As of version 1802 of SAP S/4HANA Cloud, SAP offers more than 1,000 released views, including business partner, product, sales order, project, and more. With custom CDS views, you can create read-only OData services that deliver the data tailored for the needs of your external app. In custom CDS views, you can:

- Select elements from the underlying data sources (a process known as projection) and adapt their properties (for example, label, aggregation behavior, and semantics)

- Add one or more associated data sources (also referred to as “joining data sources”)

- Create calculated fields, such arithmetical calculations, case statements, and conversions

- Manage filters and parameters to restrict the result set

A preview functionality for data discovery is included in the Web-based key-user tool.

Forms and E-Mail Template Extensibility

Print forms are maintained through Adobe LiveCycle Designer, which is based on OData services and which might be extended in case of field or table extensions according to the user's choice. Placing a check mark on a print form that's assigned to an extended business context in the key-user extensibility tool makes the extension field available in the field catalog of the corresponding form. Navigating forward to the corresponding editor opens the administration screen for form templates, which in turn allows Adobe LiveCycle Designer to be launched.

Business Logic Extensibility

Enhancements to the behavior of applications and processes include data validation, data calculation (for example, supplying default values), and mapping of standard and extension fields within applications and processes. To make processes more flexible, custom-specific

checks (for example, on approval) or the option of removing process steps or defining additional ones might eventually be offered. Additional examples of use cases are application-domain-specific topics, such as tax calculation or price determination.

In a state-of-the-art Web-based ABAP editor, you can easily add business logic to existing enhancement options. The editor completely hides the technical details of the underlying enhancement technology (such as business add-ins, or BAdIs, from SAP) from you as a key user. It provides a simplified way of exploring APIs with key-user documentation and features such as syntax check, syntax highlighting, and code completion. Testing capabilities are also included. The editor is designed according to cloud qualities: It neither allows usage of obsolete ABAP statements nor direct database access or task handling.

In the Web-based ABAP editor, you can:

- Create simple ABAP program logic supported by syntax highlighting, code completion, syntax checking, and other state-of-the-art editor features.

- Create a draft version of the program logic; test this locally; save the input values of the test as variants; publish, edit, and reset the program logic to the last published version; and delete existing custom logic.

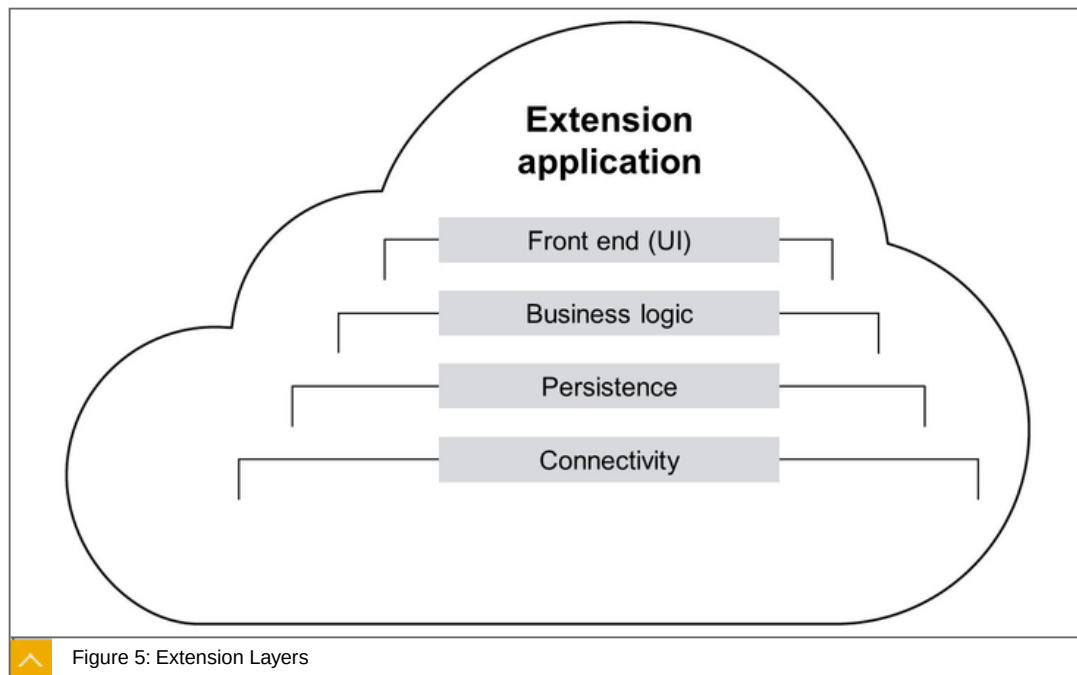
- Use released CDS views and released ABAP classes in the program logic.

- Call an external Web service that runs, for example, on SAP BTP.

Side-By-Side Extensibility

SAP BTP is the platform-as-a-service (PaaS) offering that customers, partners, and developers can use to build side-by-side extensions by leveraging the services available in the cloud environment. SAP BTP comprises infrastructure services, such as cloud operations, data backup, compliance, and service-level agreements (SLAs); database services provided by the SAP HANA business data platform, such as in-memory analytics, text search, planning, predictive, and stored procedures; and application services, such as cloud portal, security, document, administration, and development tools. Powerful integration services are available with the SAP BTP Integration service.

SAP BTP, with its broad range of extensibility services, is the de facto choice for building an extension for any cloud solution from SAP. Developers can use Java, Node.js, ABAP, or the native extended application services of SAP HANA, combined with the underlying open-API layer and powered by the speed of SAP HANA.



An extension application consists of several layers. It usually has a front end, UI layer that is decoupled from the back end by Open Data Protocol (OData) or Representational State Transfer (REST) services. The extension application back end includes existing SAP solution services, or it can expose custom services delivered with the extension application on SAP BTP. It comprises the active business logic, including both the content and the security check, the persistence layer and the connectivity to one or more back-end systems.

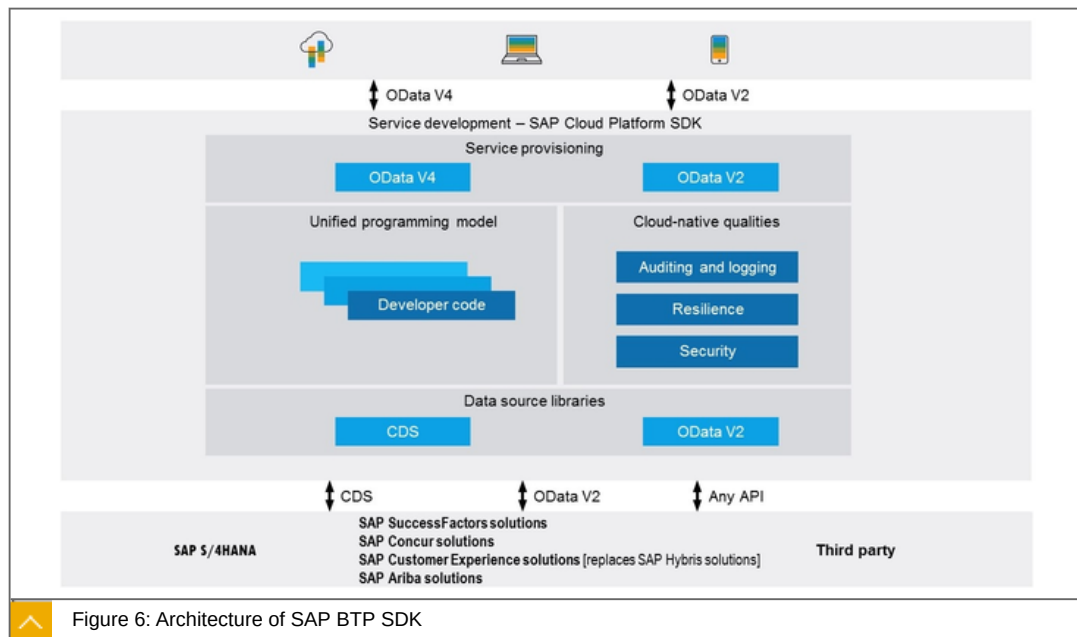
The programming model of a side-by-side extension of offerings in the SAP S/4HANA or SAP S/4HANA Cloud suite follows the approach of an independent microservice that consumes content from SAP S/4HANA Cloud through services exposed by SAP S/4HANA Cloud. An extension application, therefore, consists mainly of the following layers:

Front end – presentation of the enriched or changed business process to the user

Business logic – customer-specific business logic for enriching the business process to be extended

Persistence – customer-specific persistence for data that is used exclusively by the extension application

Connectivity – connection to SAP S/4HANA Cloud for making use of the content and applications offered by SAP S/4HANA Cloud



SAP BTP SDK also offers convenient APIs for consuming either CDS artifacts from an SAP HANA database or any OData V2 service. That way, you can use the cloud application database artifacts, services from SAP S/4HANA, or other services offered through SAP API Business Hub. As a generic tool, SAP BTP SDK is not restricted to the consumption of SAP S/4HANA Cloud services alone, but it can be used to consume any OData service offered by SAP. Furthermore, it can be used to expose data through OData V2 or OData V4, which is consumed from any API such as other REST-based services.

Among others, the following are some of the functionalities that SAP S/4HANA Cloud SDK provides to support applications:

Connectivity– establishment and management of connections to instances of SAP S/4HANA Cloud.

Data modeling– creation of a typed data model of services in SAP S/4HANA Cloud.

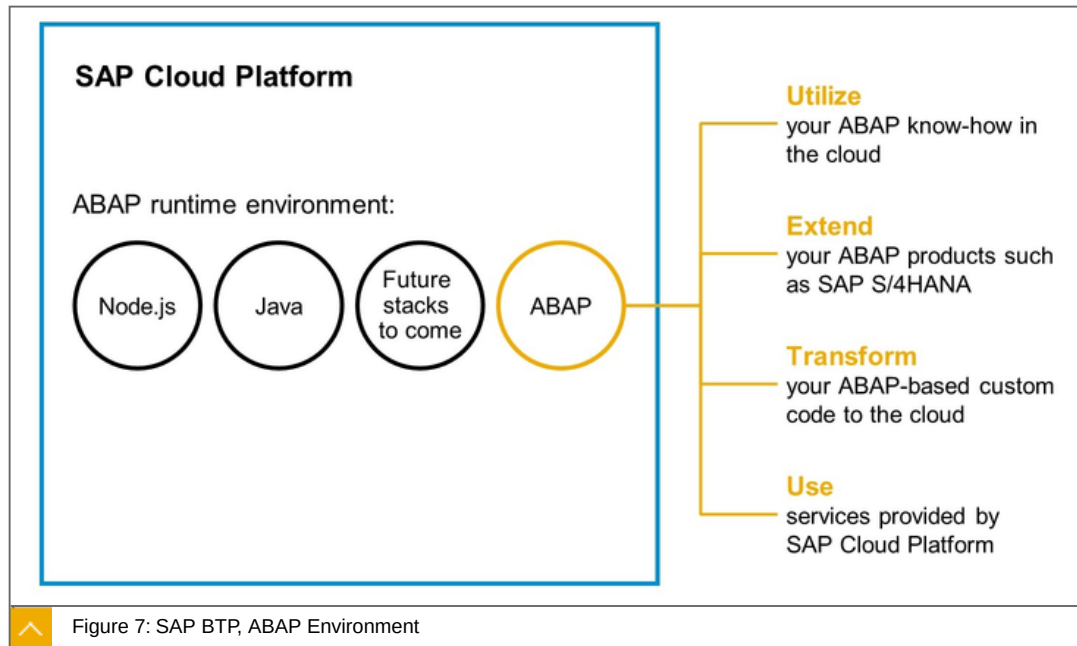
Audit logging– simple access to the audit logger in SAP BTP.

Security– abstraction of security concepts for SAP BTP.

Tenant management– simple access to tenant information for multi tenant applications.

Cloud quality– resilience and fault tolerance through Hystrix integration.

Persistence– multitenant awareness persistence.



A huge number of our customers and partners are running their mission-critical businesses with solutions based on the ABAP programming language, such as SAP ERP, SAP Business Suite, and SAP S/4HANA. Consequently, a high number of custom ABAP programs and a tremendous ABAP community has emerged over the last decades. SAP customers transitioning to SAP S/4HANA Cloud are looking for ways to leverage their investments in on-premise ABAP extensions. With SAP BTP, ABAP environment, SAP provides an additional development and runtime environment to use next to existing environments such as Java or Node.js.

While a complete rewrite of customer or partner extensions on a side-by-side, non-ABAP environment requires huge efforts and investments, you can significantly reduce the efforts required for a transition to the cloud by reusing ABAP code and skills, at least partially.

The ABAP environment is based on released APIs and offers a cloud-optimized ABAP language that uses a programming model based on SAP Fiori, and include the creation and consumption of CDS views. The strict decoupling of customer and partner extensions from the digital core (SAP S/4HANA in the on-premise environment or SAP S/4HANA Cloud) requires a restriction to released APIs of SAP S/4HANA and the ABAP environment itself. This approach enables agile extension development and fast upgrades of SAP S/4HANA, in contrast to earlier modifications, which have often required large upgrade efforts.



LESSON SUMMARY

You should now be able to:

Explain the SAP BTP and ABAP Environment

Unit 2

Lesson 2

Introducing a Modern ABAP Language



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Introduce a modern ABAP language

A Modern ABAP Language



Enablement of smooth ABAP PaaS operation through restricted ABAP language feature set intended for developments on the SAP Cloud Platform ABAP Environment

Object-oriented

ABAP language extensions provided for the development of transactional service-based SAP Fiori business applications

Access only allowed to released development objects and programming interfaces

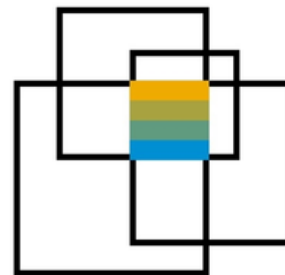


Figure 8: ABAP Language Feature Scope

The figure illustrates the following objects:

Restricted ABAP language feature set

For example, access to data of other tenants is not possible – SQL interface is adapted accordingly.

ABAP language extensions for transactional SAP Fiori apps

This is implemented with ABAP Restful Programming model with new development objects like Business Object, Service Definition, and Service Binding.

Released development objects

Some development objects are not useful in ABAP Environment, for example Type-1-Programs (Classical Report), or Classical Screens (Dynpros).

Released ABAP Development Objects are:

Table 1: Released ABAP Development Objects

Topic	Content
ABAP Core Development	Package, Class, Interface, Function Group, Function Module, Include, Simple Transformation, Transformation, Message Class, Authorization Object, Authorization Object Class

ABAP Dictionary	Domain, Data Element, Lock Object, Database Table, Structure, Type Groups, Table Types
ABAP Core Data Services (CDS)	CDS Data Definition, CDS Access Control
Business Objects	Behavior Definition, Behavior Implementation
Business Services	Service Definition, Service Binding
Identity Management	App, Business Catalog
Connectivity	HTTP Service

You can find more details in the SAP Help Documentation for SAP BTP, ABAP environment.

The table shows the restricted development object set released for use in SAP BTP, ABAP environment.

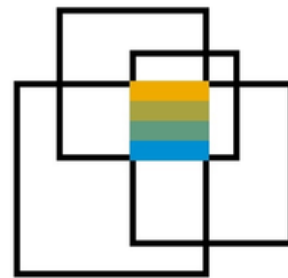
The list of development objects includes useful objects for the main purpose of ABAP environment, which is to allow side-by-side extensions for cloud-based SAP software solutions. Some development objects are not useful in ABAP environment, for example Type-1-Programs (Classical Report) or Classical Screens.



Enablement of smooth ABAP PaaS operation through restricted ABAP SQL feature set support on SAP Cloud Platform ABAP Environment

Major restrictions (i.e. not supported)

- SQL-CONNECTION and SELECT-CONNECTION to avoid unwanted secondary database connection
- SQL-CONNECTION-DYNAMIC and SELECT-FROM-DYN due to no support for dynamic performance
- SELECT-CLIENT-SPECIFIED and SELECT-USING-CLIENT to enforce client separation
- Obsolete statements like SELECT-FOR-UPDATE



Find more details in the SAP Help Documentation for SAP Cloud Platform Environment

Figure 9: ABAP SQL Feature Scope

Secondary database connections are not possible. Direct access is only possible to the SAP BTP's HANA database. Access to other databases is only possible indirectly. Using whitelisted APIs, you can access data services of other software systems.

Access to other tenant's data (comparable to access to other client's data in classical systems) is not wanted and not possible.

Released APIs

Facts about released APIs:

Restricted API set released for use in SAP BTP, ABAP environment.

Can easily be found with Eclipse ABAP Development Tools (ADT).

Checks against the language scope and released APIs for ABAP Test Cockpit (ACT) on 7.52 SAP_BASIS available.

There is only a restricted **external content** available in ABAP environment. External content is the set of all development objects that are imported into the ABAP Environment (meaning

they come from SAP, not from the customer). While in a classical on-premise system there was a lot of SAP content available to the customer developer, in an ABAP environment this is restricted to some generally needed development objects and very few application-specific development objects.

In ABAP Development Tools, this external content is shown under External Content in the Repository Browser.



LESSON SUMMARY

You should now be able to:

- Introduce a modern ABAP language

Unit 2

Lesson 3

Obtaining an ABAP Environment



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Obtain an ABAP environment

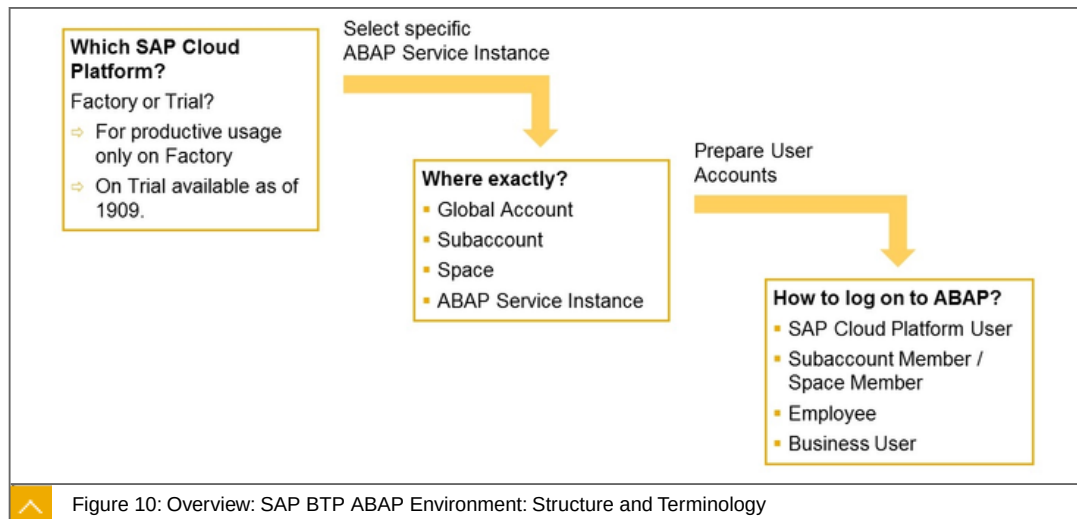
Obtain an ABAP Environment

Contents in the landscape of this training – What the administrator did for you:



Creation of ABAP Service.

Creation of Business Customers.



The ABAP Environment is also called an *ABAP Service Instance* when looked upon from SAP BTP from an administrator's perspective.

In order to create such an ABAP Service Instance, you have to use *SAP BTP Factory Version* for productive usage. This means the productive usage is not free of charge. Within SAP BTP, there is a structure of *Global Accounts* and *Subaccounts*. While a global account relates to an SAP Customer (an organizational unit), a subaccount is a technical division within the global account that allows to dispatch quotas, for example, between departments or sub-organizations that compete for resources like runtime or disc space.

Within a subaccount, there is a further entity, the *Space*. Service instances are assigned to spaces. It is possible that in one space several instances of the same service kind exist, for example, several instances of ABAP Service. A space is also assigned quotas. ¹

¹ In cloud computing, resources come out of the net but they are not necessary for free. A customer may have to book resources and may have to pay for it.

For each of these entities exists a user concept: subaccount member, space member, and so on.



SAP Cloud Platform is available as

- Factory Version
- Trial Version

Factory

- <https://account.hana.ondemand.com>
- Licensed
- All Features available
- ABAP Service available

Trial

- <https://account.hanatrial.ondemand.com>
- Free of charge
- Reduced Feature Set (but still very mighty)
- ABAP Service Trial Version available

Figure 11: ABAP Service on SAP BTP "Factory Version"

Since 1909 (September 2019), the Trial Account also offers the ABAP Environment.



- Get a **global account** to develop and deploy applications on SAP Cloud Platform.
- Set up your account model to get your SAP Cloud Platform **Global Accounts** and **Subaccounts** that enable you to access the relevant SAP Cloud products within the production landscape.
- Create a space.
- Set up instance of ABAP Service.
- Configure your environment to adapt the service instance that represents the ABAP system to your company's needs. For example, you might want to provide user access for the application.

Name	Service	Plan	Referencing Applications	Service Keys	Last Operation	Action
abap	destination	dev	None	1	Created	
CloudStore-Instance-0002	hanacloud	Ind-shared	rgl007-hanacloud-hanacloud	2	Created	
CloudStore-Instance-0003	hanacloud	Ind-shared	rgl007-hanacloud-hanacloud	1	Created	
com	connectivity	dev	address-manager	1	Created	
cp000abap	abap	SL_abap_54_00	None	2	Created	

Figure 12: Log on Credential Prerequisites (To Log on to your ABAP Service Instance)

To get an instance of an ABAP Service instance, complete the following steps:

1. Get a global account.
2. Get a subaccount.
3. Get a space.
4. Create an ABAP Service instance.
5. Create users (employees and business users).

There are several kind of users needed before you can actually develop with ABAP Environment:

Users for Administrative Tasks

1. An SAP BTP User (Factory Account).
2. A subaccount member or a space member.

Users for Developer Tasks

1. An Employee (an entity within the ABAP Service). The existence of an employee is a precondition for the creation of a Business User.

2. An Business User (an entity within the ABAP Service). This is the actual user for development. To this, user appropriate roles can be assigned.

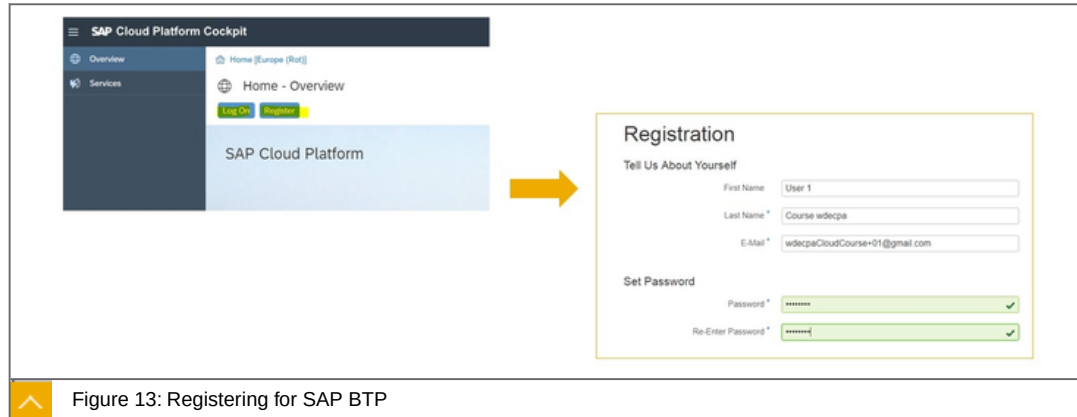


Figure 13: Registering for SAP BTP

The *SAP BTP User for Factory Account* has to be created for each user. This means it is a personalized user. The user is identified by an email address. The user needs to have access to the email account because they will have to answer a confirmation email when registering.

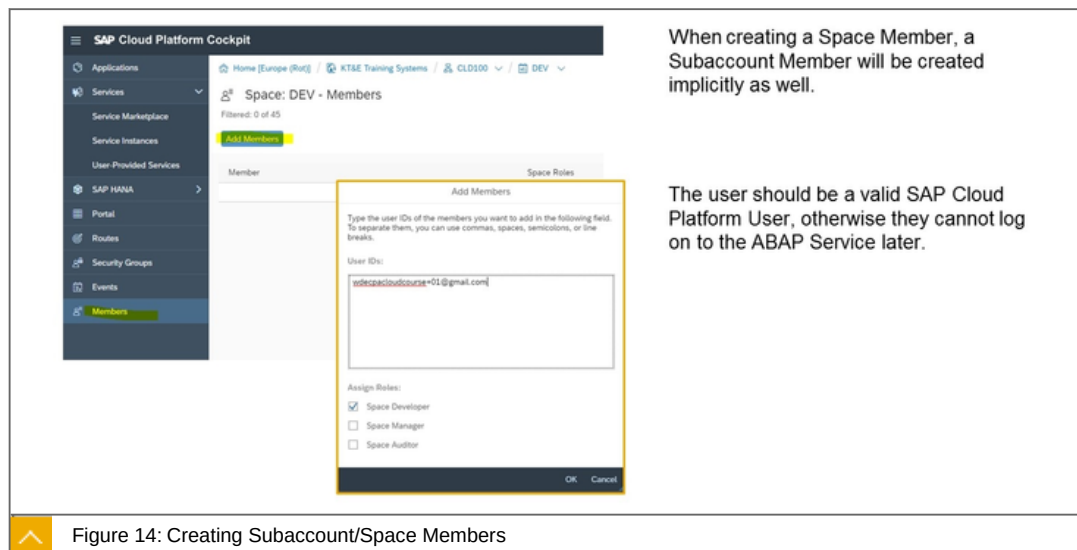


Figure 14: Creating Subaccount/Space Members

Space members should be created by an administrator. A space member is identified by their email address.



Go to ABAP Service Instance Dashboard

The screenshot shows the SAP Cloud Platform Cockpit interface. On the left, a sidebar contains navigation links: Applications, Services, Service Marketplace, Service Instances, User-Provided Services, SAP HANA, and Portal. The main area displays 'Space: DEV - Service Instances' with a table of service instances. The table has columns: Name, Service, Plan, Referencing Applications, Service Keys, Last Operation, and Actions. One instance, 'cp100abap', is highlighted. An arrow points from this instance to the 'Create Employee' form.

The 'Create Employee' form is shown with the following fields:

- Basic Data: Employee ID (wdecpacloudcourse01), Form of Address (dropdown), First Name (Textuser), Last Name (wdecpacloudcourse01), Workplace Data.
- Contact Data: E-Mail (wdecpacloudcourse01@gmail.com), Telephone, Mobile Phone, Language.

Figure 15: Creating Employees

The next step is to create an Employee for the ABAP Service instance. The existence of an Employee is a precondition for the creation of a Business User. Again, the usage of the email address is necessary.



The screenshot shows the Identity and Access Management (IAM) console. On the left, a sidebar contains navigation links: Maintain Business Users, Display Technical Users, and Display Business Users. The main area displays the details for a user named 'wdecpa WDECPA01'. The user details are organized into sections: Personal Data, User Data, Regional Settings, and Assigned Business Roles (1).

The 'Assigned Business Roles (1)' section shows the following roles:

Business Role	Business Role ID	Price Category
Developer	BR_DEVELOPER	

Figure 16: Creating Business User

Finally, there is the Business User. This user is the one that is actually necessary to log on via ADT to the ABAP Service and perform developer tasks. It is here where the administrator of the ABAP Service Instance assigns appropriate roles. The most important role would be standard role **BR_DEVELOPER**. The administrator creates this role from a template role. The administrator will have to assign Business Users to this role and maintain access restrictions for this role.



Note:

In the role **BR_DEVELOPER** the default for **Write Access** is *no value* (that is, forbidden), therefore, the administrator must not forget to change this to *Unrestricted*.



When Creating Cloud Project

System Connection to SAP Cloud Platform ABAP Environment

Select how to establish the system connection for the new ABAP cloud project.

Service Instance Connection

☒ SAP Cloud Platform Cloud Foundry Environment
Select this option to enter the login data of your account. The service key is provided by the Cloud Foundry Environment.

☐ Service Key
Select this option to enter your service key in JSON format.

When Logging On otherwise / as well final step of creating Cloud Project

Connection Settings to SAP Cloud Platform Cloud Foundry Environment

Specify the connection settings of the service instance using the drop-down list boxes

Service Instance Details

Organization: KT&E Training Systems_cld100

Space: DEV

Service Instance: cp100abap

When Logging On otherwise / as well final step of creating Cloud Project

System Connection Using a Service Key

Enter the login credentials of the SAP Cloud Platform Identity Authentication Service (IAS) tenant configured for your SAP Cloud Platform Cloud Foundry subaccount.

Service Instance Login

Welcome to CLD100!

sven@1234567890.com

Log On

Figure 17: Logging On to ABAP Service from Eclipse with Logon Credentials

After all is set, the developer creates an ABAP Cloud Project from ADT. When doing so, they have to log on to the ABAP Service instance using their email address and the other parameters as shown in the figure.



When Creating Cloud Project

System Connection to SAP Cloud Platform ABAP Environment

Select how to establish the system connection for the new ABAP cloud project.

Service Instance Connection

☐ SAP Cloud Platform Cloud Foundry Environment
Select this option to enter the login data of your account. The service key is provided by the Cloud Foundry Environment.

☒ Service Key
Select this option to enter your service key in JSON format.

When Logging On otherwise / as well final step of creating Cloud Project

System Connection Using a Service Key

Enter the login credentials of the SAP Cloud Platform Identity Authentication Service (IAS) tenant configured for your SAP Cloud Platform Cloud Foundry subaccount.

Service Instance Login

Welcome to CLD100!

sven@1234567890.com

Log On

Final Logon still requires User / Password.

Figure 18: Using the Service Key

Instead of using an SAP BTP user, you can use the ABAP Service Key for the first part of the log on procedure when creating an ABAP Cloud Project.



Global Account ⇒ Subaccount ⇒ Space ⇒ ABAP Service Instance ⇒ Service Key

Figure 19: How To Retrieve Service Key

With the help of the ABAP Service Key, you can avoid the log on of the SAP BTP User when creating an ABAP Cloud Project. You still need an Employee and a Business User for the actual log on.



LESSON SUMMARY

You should now be able to:

- Obtain an ABAP environment

Unit 2

Lesson 4

Explaining ABAP Development Tools in Eclipse (ADT)



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explain ABAP Development Tools in Eclipse (ADT)

ABAP Development Tools (ADT) in Eclipse



Full ABAP for SAP HANA support
SQL tools, incl. Data Preview and SQL console
Troubleshooting tools like debugger, tracing, dump analysis
Quality tools for static code checks and unit testing
...

Superior ABAP toolset provided in Eclipse

Open and extensible

SAP's central eclipse update site on
<https://tools.hana.ondemand.com>

SDK for partner and customer extensions

Eclipse Marketplace & Community

Figure 20: ABAP Development Tools in Eclipse: Greater Productivity for Developers

ABAP Development Tools is the state-of-the-art Integrated Development Environment (IDE) for the ABAP developer.

It comprises full ABAP for SAP S/4HANA support, for example, for maintenance of CDS Views (Core Data Services views) and other CDS entities. It includes, for example, a data preview which is similar with the classical data browser. Debugging, tracing, ABAP Unit support and ABAP Test Cockpit (ATC) are also included.

ABAP Development Tools offers editor tools for each development object. While in the beginning many ADT editors were simply rendered as classical screens with SAP GUI for HTML, now more and more native editors are offered (*native* meaning specifically created for ADT usage).



Figure 21: ABAP Development Tools in Eclipse: Major Enhancements in ABAP 7.50

Previously, an SAP GUI editor was used to edit a dictionary structure from ADT. Now there is an ADT specific editor. As is typical for many ADT editors, it is source-based. In the SAP GUI, many editors are form-based.

Core Data Services (CDS) comprise several entities, among them CDS views, which is a very powerful new kind of view that allows a very powerful SQL and hierarchical views (view-on-view concept). CDS entities cannot be maintained with the classical SAP GUI based workbench transaction SE80. Therefore, if one has to maintain CDS entities, then the usage of ADT is a must.

AMDP: ABAP managed database procedures: ABAP procedures executed on the database. With SAP S/4HANA “push down to data” has become a very popular paradigm for programming. This means a lot of calculations are performed on the database and not in the program anymore. AMDPs have the advantage that they still can be coded from within the ABAP Environment – though performed on the database.

BOPF: Business Object Processing Framework. BOPF is the programming model for SAP Fiori UI5 apps.



Figure 22: ABAP Development Tools in Eclipse: Major Enhancements in ABAP 7.51

Domains are an entity from ABAP dictionary. Since this release, they can be maintained with a native editor.

The Project Explorer provides an overview of the development objects of a selection, for example, of a package. You can organize your view in different ways according to your needs.

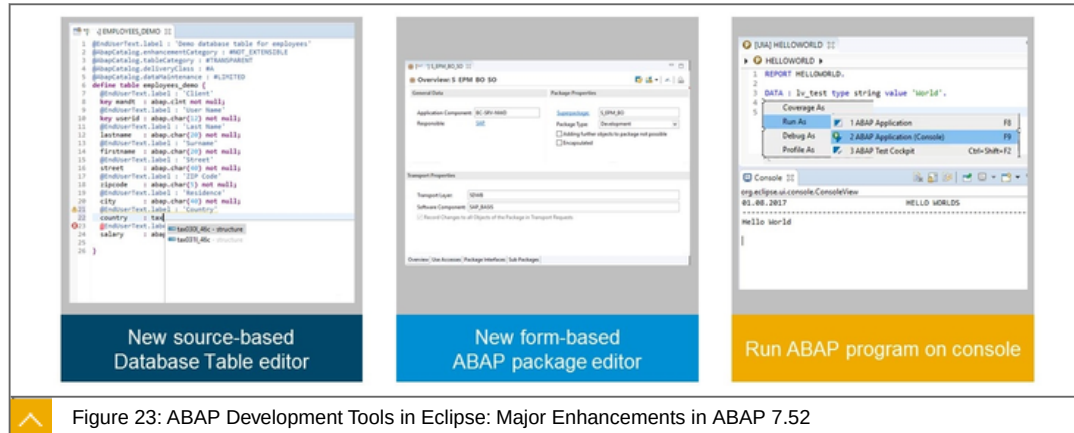
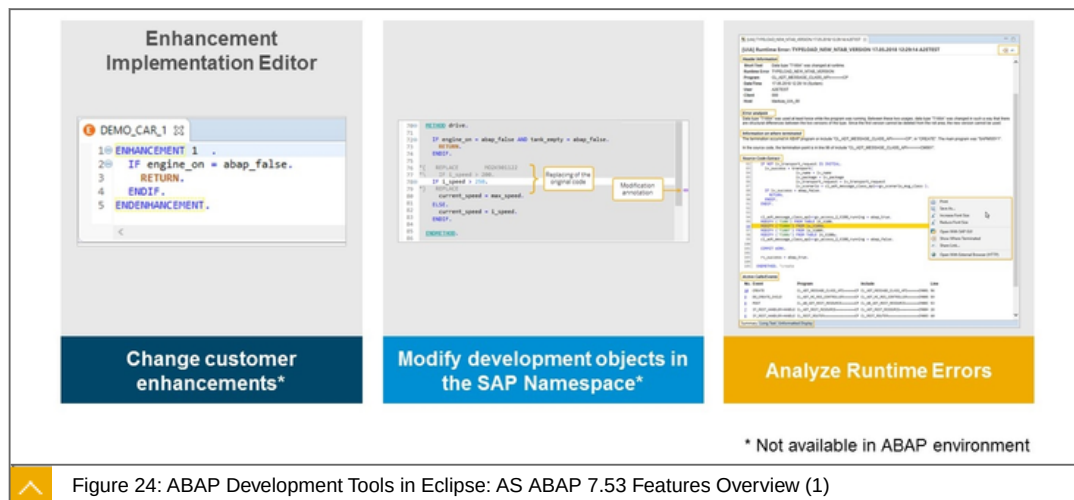


Figure 23: ABAP Development Tools in Eclipse: Major Enhancements in ABAP 7.52

Previously, an SAP GUI editor was used to edit a database table from ADT. Now there is an ADT specific editor. As is typical for many ADT editors, it is source-based. In SAP GUI, many editors are form-based.

Since this release, packages can be maintained with a native editor.

Since this release, a console based output is possible with help of the `IF_OO_ADT_CLASSRUN` interface. The output is comparable to an easy write statement from ABAP or a similar output statement of other programming languages. You would use this for immediate easy tests and checks done by the developer.



* Not available in ABAP environment

Figure 24: ABAP Development Tools in Eclipse: AS ABAP 7.53 Features Overview (1)

As of release 7.53, from ADT a Customer Enhancement can be changed. A restriction is that it cannot be created. (This means they have to be created before elsewhere – namely in SAP GUI based Object Navigator.)

As of release 7.53, you can start a modification from ADT.



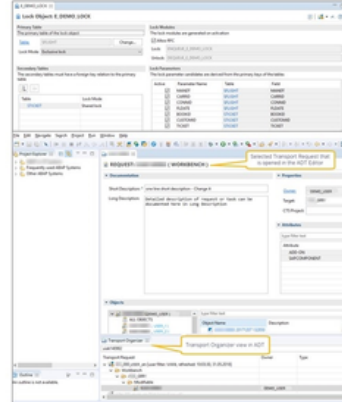
Classic objects in ABAP Dictionary

- Create and edit Lock Objects

Transport Organizer

- View and edit details of a Transport Request

Lock Object Editor



Transport Request Editor

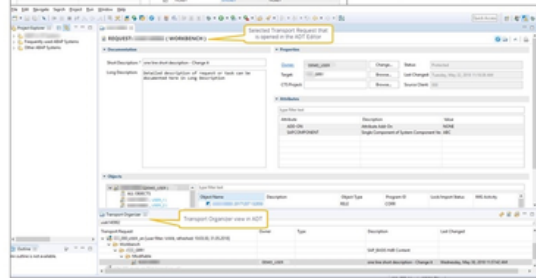


Figure 25: ABAP Development Tools in Eclipse: AS ABAP 7.53 Features Overview (2)

To maintain a lock object in the beginning, an SAP GUI editor was used. Now there is an ADT specific editor.

Now there is a native editor for the Transport Organizer available.

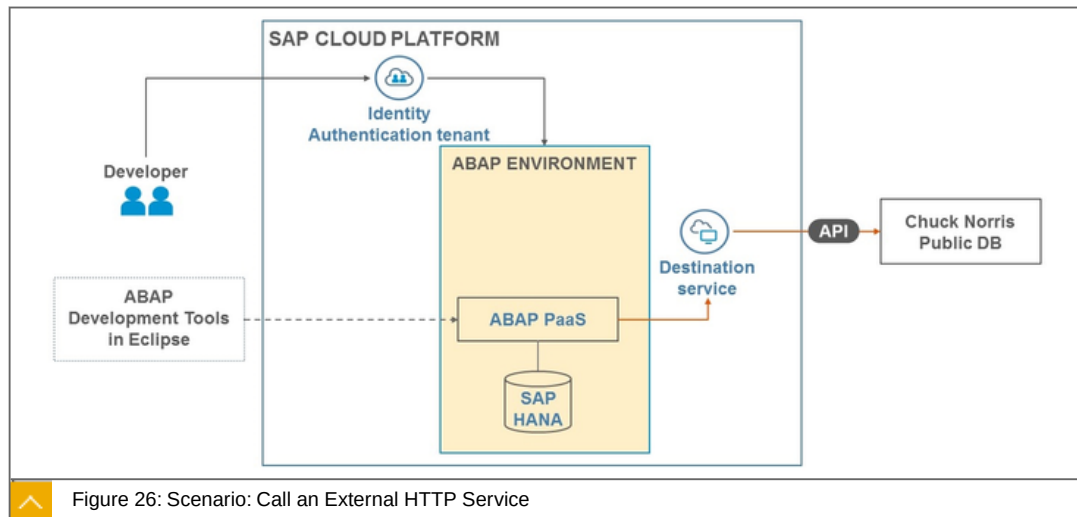


Figure 26: Scenario: Call an External HTTP Service

A typical scenario for ABAP Environment would be the call of an external service via HTTP. There is an API (Application Programming Interface) delivered with ABAP Environment (External Content) so that a developer can use the SAP BTP's Destination Service. The Destination Service can be used throughout the whole SAP BTP.

On the other hand, the ABAP Environment can offer an HTTP service to the outside world (not shown in the figure). To facilitate this, the ABAP Environment includes the HTTP Service Development Object.

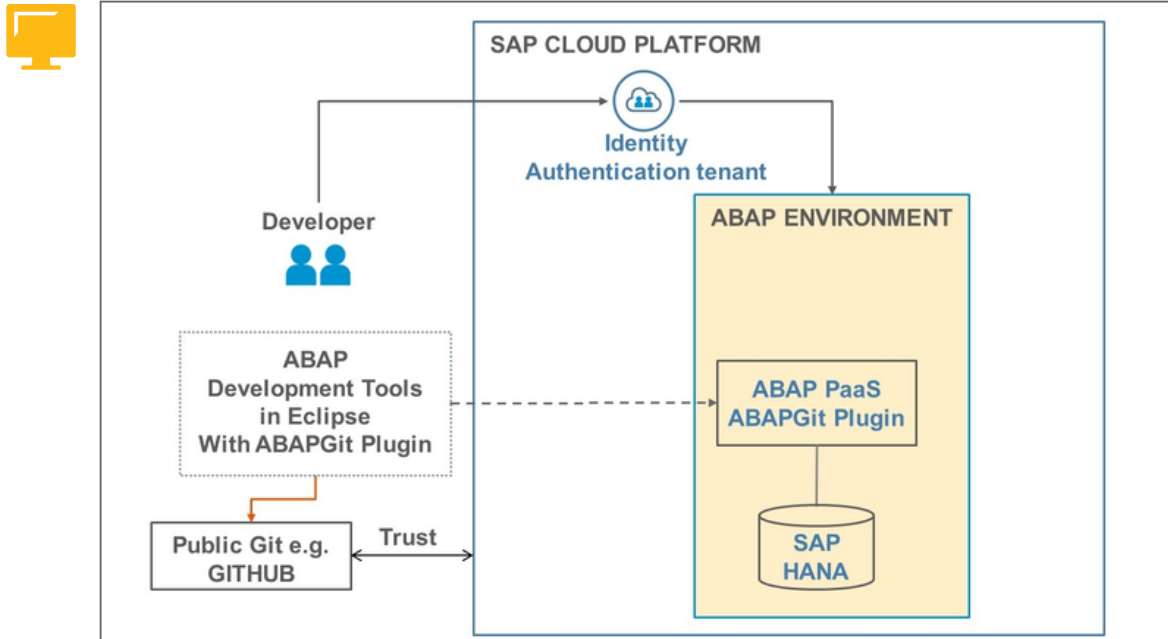


Figure 27: Scenario: Import Sources via ABAPGit

Git /GitHub is a broadly accepted way of sharing sources on the net. There is a specific method for exposing ABAP sources in a git repository. With the `ZABAPGIT` report running on an on-premise system, you can create a repository in the web in a specific format. This format in turn can be consumed from **abapGit** plug-in in the ABAP Development Tools. It creates corresponding development objects in ABAP Environment. So with the `ZABAPGIT` report and the `abapGit` plugin, you have an infrastructure that is similar to the classical transport system. You can copy development objects from a source system into the ABAP Environment. But unlike with the classical transport system, the development objects are created as originals in the ABAP Environment – this means that you can edit them. You can also make use of GitHub's features to administrate several versions of a source.

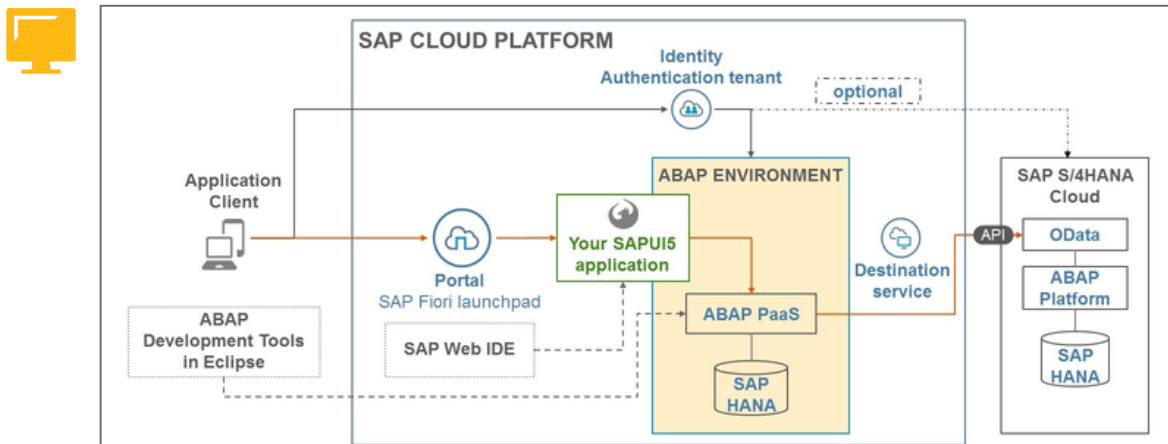
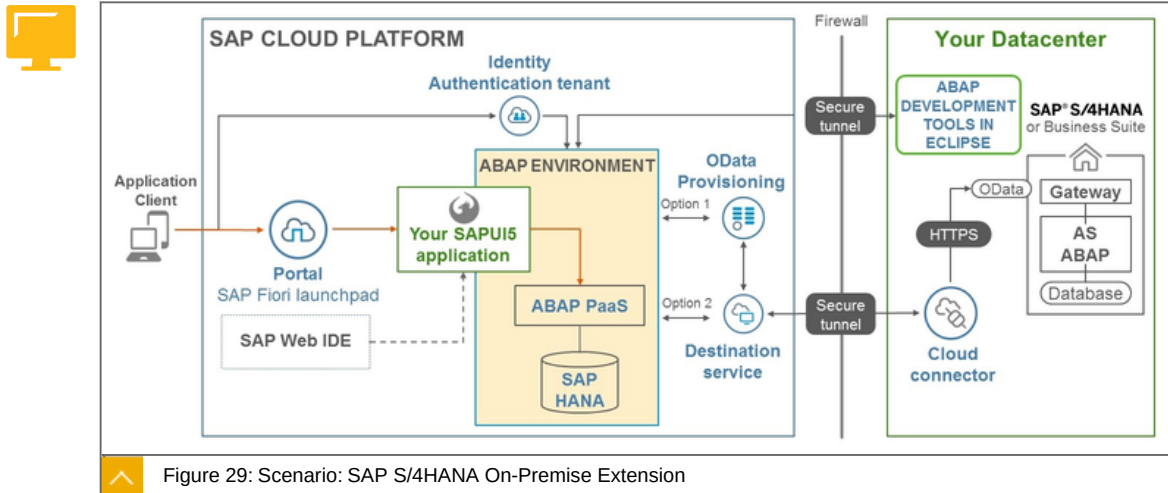


Figure 28: Scenario: SAP S/4HANA Cloud Extension

Development for ABAP Environment takes places in ABAP Development Tools. For example, you might code a method that calls an OData Service on your SAP S/4HANA Cloud system. This method in turn might be called in the implementation of an HTTP service that is offered by one's ABAP environment. A customer SAPUI5 application might call this HTTP service.

This shows that a user can call an SAPUI5 application and this application may make use of the services offered with ABAP Environment.

An SAPUI5 application might as well call the OData service directly. To use the option via ABAP Environment is especially relevant if the developer wishes to implement an additional function in the ABAP Environment.



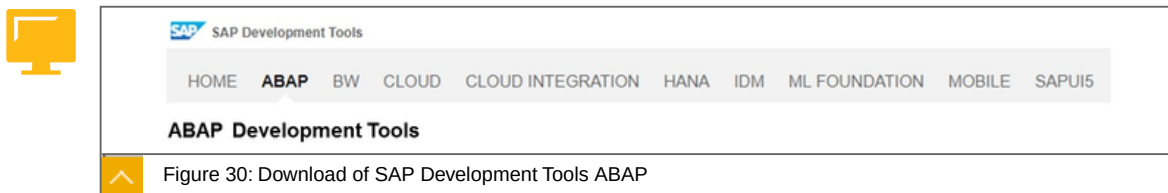
Since 2019 (Release 1902), it is also possible to develop side-by-side extensions for on-premise SAP systems like SAP S/4HANA. In this case, an OData service call is directed to an on-premise system. As SAP BTP is cloud-based, there must be a way to establish a connection between cloud and an on-premise system. This can be achieved with the help of the Cloud Connector.

Eclipse and ABAP Development Tools Plug-In

Download of Eclipse and the ABAP Development Tools Plug-In

To use the ABAP Environment, you need to have the Eclipse IDE with the ABAP Development Tools (ADT) plug-in installed on your client computer. Depending on whether you have already installed Eclipse from another usage (for example, Java Development) or not, you must either first install Eclipse and then add the plug-in, or just add the plug-in.

If this is your first time, go to <https://tools.hana.ondemand.com/#abap> for a detailed description of the procedure.



The figure introduces to the procedure to install the front-end component of ADT.

Procedure

To install the front-end component of ADT, proceed as follows:

1. Get an installation of Eclipse 2019-06 (for example: Eclipse IDE for Java Developers).
2. In Eclipse, choose in the menu bar Help → Install New Software...

3. For Eclipse 2019-06 (4.12), add the URL <https://tools.hana.ondemand.com/2019-06>
4. Press Enter to display the available features.
5. Select ABAP Development Tools and choose Next.
6. On the next wizard page, you get an overview of the features to be installed. Choose Next.
7. Confirm the license agreements and choose Finish to start the installation.

Via this procedure, you are guided through the whole installation process.



Note:

You must use the *Oxygen* release or later. If, after some time, you install a more recent version of Eclipse, you must install the plug-in from new, because the installation of the new Eclipse version is independent from the former (Eclipse) installation.



Note:

Within your Eclipse installation, you should update the ABAP Development Tools Plug-In from time to time as new versions are shipped frequently.

Tutorials on the Internet

To find tutorials, go to <https://developers.sap.com/tutorial-navigator.html>. In the navigator, choose the ABAP Development topic, then choose the SAP Cloud Platform, ABAP Environment product.



LESSON SUMMARY

You should now be able to:

Explain ABAP Development Tools in Eclipse (ADT)

Learning Assessment

1. What is correct with regards to in-app extensibility?

Choose the correct answers.

- ☐ **A** The key user can perform simple extensibility tasks in an easy-to-do way.
- ☐ **B** UI adaptations are feasible for fields, groups, and field labels.
- ☐ **C** The key user may adapt the behavior of the kernel functions of the application server.
- ☐ **D** The key user may reorganize the sequence of the Logical Unit of Work processing.
- ☐ **E** The key user may add custom business logic at predefined enhancement points.

2. What is true concerning the new extensibility concept for SAP S/4HANA?

Choose the correct answers.

- ☐ **A** Only such an extensibility is allowed that does not block updates and permits to continue to work after an update.
- ☐ **B** No modifications are allowed.
- ☐ **C** Custom side-by-side extensions are tightly coupled with the digital core.

3. ABAP Environment uses a specific ABAP language version. Which of the following is correct?

Choose the correct answer.

- ☐ **A** ABAP Environment uses ABAP version "Standard ABAP" just like an on-premise system.
- ☐ **B** ABAP Environment can use all external repository objects arbitrarily.
- ☐ **C** In ABAP Environment 1809 (1st version) it was prohibited to specify data objects dynamically.
- ☐ **D** In ABAP Environment, usage of secondary database connections and Native SQL is allowed.

4. An ABAP Environment is a service of SAP Cloud Platform. Which of the following are correct?

Choose the correct answers.

- ☐ **A** In a technical sense, the ABAP Service is associated directly with a Global Account; there is no need for a subaccount and a space.
- ☐ **B** The ABAP Service is instantiated as part of a space. A space in turn can comprise several ABAP services.
- ☐ **C** To perform administrative tasks for the ABAP Service with ABAP Service Launchpad (Dashboard), you need to have a Business User.
- ☐ **D** To perform development tasks with ABAP Development Tools (ADT) on ABAP Environment, you need to have a space user.
- ☐ **E** To perform development tasks with ABAP Development Tools (ADT) on ABAP Environment, you need to have a Business User.

5. To log on to ABAP Environment coming from ABAP Development Tools (ADT), using the service key is sufficient.

Choose the correct answer.

- ☐ **A** Yes, you do not need any other credentials.
- ☐ **B** No. Besides this, you need a business user.

6. For side-by-side extensions, SAP BTP offers several programming languages. Which of the following are supported?

Choose the correct answers.

- ☐ **A** HTML5
- ☐ **B** Cobol
- ☐ **C** ABAP
- ☐ **D** Assembler

7. SAP BTP, ABAP environment allows Agile Innovation on the newest technology. Which of the following are correct?

Choose the correct answers.

- ☐ **A** Developers can work from everywhere.
- ☐ **B** The newest SAP S/4HANA capabilities always can be exploited.
- ☐ **C** Innovations from SAP must be pulled explicitly by the customer; there is no automatism that they are available.
- ☐ **D** The newest ABAP Programming Model is always available.

8. ABAP Environment is a Platform-as-a-Service (PaaS). Which of the following is correct?

Choose the correct answer.

- ☐ **A** It cannot be part of SAP BTP because SAP BTP itself is already a PaaS.
- ☐ **B** It is always part of SAP BTP.
- ☐ **C** It can be part of SAP BTP but it is available stand-alone also.

9. ABAP Development Tools (ADT) are the Integrated Development Environment (IDE) for ABAP Environment. Which of the following are correct?

Choose the correct answers.

- ☐ **A** ADT is part of SAP BTP.
- ☐ **B** ADT can be downloaded at eclipse.org and works for ABAP development out-of-the-box without any further preparations.
- ☐ **C** ADT is an Eclipse platform plus a Plug-In from SAP.
- ☐ **D** ADT can be used for on-premise development and for cloud-based development.

Learning Assessment - Answers

1. What is correct with regards to in-app extensibility?

Choose the correct answers.

- ☒ **A** The key user can perform simple extensibility tasks in an easy-to-do way.
- ☒ **B** UI adaptations are feasible for fields, groups, and field labels.
- ☐ **C** The key user may adapt the behavior of the kernel functions of the application server.
- ☐ **D** The key user may reorganize the sequence of the Logical Unit of Work processing.
- ☒ **E** The key user may add custom business logic at predefined enhancement points.

You are correct! The key user can perform simple extensibility tasks in an easy-to-do way. UI adaptations are feasible for fields, groups, and field labels. The key user may add custom business logic at predefined enhancement points.

2. What is true concerning the new extensibility concept for SAP S/4HANA?

Choose the correct answers.

- ☒ **A** Only such an extensibility is allowed that does not block updates and permits to continue to work after an update.
- ☒ **B** No modifications are allowed.
- ☐ **C** Custom side-by-side extensions are tightly coupled with the digital core.

You are correct! Only such extensibility options are allowed that do not block updates and permit to continue to work after an update. No modifications are allowed.

3. ABAP Environment uses a specific ABAP language version. Which of the following is correct?

Choose the correct answer.

- ☐ **A** ABAP Environment uses ABAP version "Standard ABAP" just like an on-premise system.
- ☐ **B** ABAP Environment can use all external repository objects arbitrarily.
- ☒ **C** In ABAP Environment 1809 (1st version) it was prohibited to specify data objects dynamically.
- ☐ **D** In ABAP Environment, usage of secondary database connections and Native SQL is allowed.

You are correct! In ABAP Environment 1809 (1st version) it was prohibited to specify data objects dynamically.

4. An ABAP Environment is a service of SAP Cloud Platform. Which of the following are correct?

Choose the correct answers.

- ☐ **A** In a technical sense, the ABAP Service is associated directly with a Global Account; there is no need for a subaccount and a space.
- ☒ **B** The ABAP Service is instantiated as part of a space. A space in turn can comprise several ABAP services.
- ☐ **C** To perform administrative tasks for the ABAP Service with ABAP Service Launchpad (Dashboard), you need to have a Business User.
- ☐ **D** To perform development tasks with ABAP Development Tools (ADT) on ABAP Environment, you need to have a space user.
- ☒ **E** To perform development tasks with ABAP Development Tools (ADT) on ABAP Environment, you need to have a Business User.

You are correct! In a technical sense, an ABAP service is part of a space, a space in turn belongs to a subaccount, and a subaccount belongs to a Global Account. A subaccount can have several ABAP Service instances. To perform administrative tasks, you need a space member. To perform development tasks, you need a Business User.

5. To log on to ABAP Environment coming from ABAP Development Tools (ADT), using the service key is sufficient.

Choose the correct answer.

- ☐ **A** Yes, you do not need any other credentials.
- ☒ **B** No. Besides this, you need a business user.

You are correct! Logging on with the service key only replaces the cloud platform user, but not the Business User.

6. For side-by-side extensions, SAP BTP offers several programming languages. Which of the following are supported?

Choose the correct answers.

- ☒ **A** HTML5
- ☐ **B** Cobol
- ☒ **C** ABAP
- ☐ **D** Assembler

You are correct! Cobol and Assembler are not supported.

7. SAP BTP, ABAP environment allows Agile Innovation on the newest technology. Which of the following are correct?

Choose the correct answers.

- ☐ **A** Developers can work from everywhere.
- ☒ **B** The newest SAP S/4HANA capabilities always can be exploited.
- ☐ **C** Innovations from SAP must be pulled explicitly by the customer; there is no automatism that they are available.
- ☒ **D** The newest ABAP Programming Model is always available.

You are correct! Though developers can work from everywhere, this is not a specific characteristic of Agile Innovation. Innovations from SAP are delivered automatically.

8. ABAP Environment is a Platform-as-a-Service (PaaS). Which of the following is correct?

Choose the correct answer.

- ☐ **A** It cannot be part of SAP BTP because SAP BTP itself is already a PaaS.
- ☒ **B** It is always part of SAP BTP.
- ☐ **C** It can be part of SAP BTP but it is available stand-alone also.

You are correct! ABAP Environment is always a component of SAP BTP.

9. ABAP Development Tools (ADT) are the Integrated Development Environment (IDE) for ABAP Environment. Which of the following are correct?

Choose the correct answers.

- ☐ **A** ADT is part of SAP BTP.
- ☐ **B** ADT can be downloaded at eclipse.org and works for ABAP development out-of-the-box without any further preparations.
- ☒ **C** ADT is an Eclipse platform plus a Plug-In from SAP.
- ☒ **D** ADT can be used for on-premise development and for cloud-based development.

You are correct! ADT is not part of SAP BTP. It is installed independently and can then be used to develop against ABAP Environment. It is not sufficient to download the eclipse IDE to develop with ABAP, but it is also necessary to download and install the ADT Plug-In from SAP.

UNIT 3

ABAP Language Versions

Lesson 1

Explaining ABAP Language Versions	42
-----------------------------------	----

Lesson 2

Explaining ABAP Features not Supported in SAP BTP, ABAP Environment	46
---	----

Lesson 3

Explaining Released APIs in ABAP Environment	48
--	----

UNIT OBJECTIVES

Explain ABAP language versions

Explain ABAP features not supported in SAP Cloud Platform, ABAP environment

Explain released APIs in ABAP environment

Unit 3

Lesson 1

Explaining ABAP Language Versions



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explain ABAP language versions

ABAP Language Versions

ABAP Language Versions (as of AS ABAP 7.52) are:

Table 2: ABAP Language Versions

Language Version	Meaning	Version ID
Standard ABAP (Unicode)	Universal basic version; covers the entire language scope for Unicode systems.	X
ABAP for Key Users	Designed for secure implementations of enhancements as part of enhancement points; only a very restricted set of language elements are supported.	2
ABAP for SAP Cloud Platform	This version is designed for development in SAP Cloud platform with restricted set of ABAP language elements.	5

There is one ABAP but different versions.

When Unicode ability was introduced this was implemented as the new standard version. Since then, programs have a Boolean attribute "Unicode checks" – those programs that have Unicode checks enabled are of Standard ABAP and the others were "obsolete ABAP".

With release 7.52, this has changed. Now there are ABAP language versions. Apart from the Standard Version (Version X), there is now version 2 that is used for in-app extensibility and version 5 that is used for ABAP in the Cloud Environment.

Version 5 is characterized by a reduced instruction set and restricted access to external repository objects (meaning development objects that were not created within the ABAP Environment Service Instance but are available through the infrastructure).



Figure 31: Selecting the ABAP Language Version

The figure illustrates the possible selection of the ABAP Language version in a program.



Note:

The numbers: 3, 4, and <empty> are obsolete, so no explanation is given here.

ABAP language versions – useful tools are:



Program **ABAP_DOCU_VERSION_WHITELIST** shows the external repository objects that are allowed for the restricted ABAP language versions.

Program **DEMO_ABAP_VERSIONS** makes it possible to check ABAP source code using the syntax rules of the different ABAP language versions.

From a technical perspective, the version ID is in **column UCCHECK of database table TRDIR** for program attributes and is usually set by the associated tools.

The set of the allowed external repository objects can be displayed with the **ABAP_DOCU_VERSION_WHITELIST** program. SAP may extend this set with new releases.



Examples for restrictions

```

31 data lr_data TYPE Ref to data.
32 data lv_type TYPE string value 'ABC'.
33 CREATE DATA lr_data TYPE (lv_type).
34
35
36 data lv_func TYPE string value 'FM_TEST'.
37 call function lv_func.
38
39
40 data lv_carrid TYPE s_carr_id.
41
42
43 select * from t001 into TABLE @data(lt_t001).
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

The addition "DATA TYPE <dynamic-type>" is not valid in the restricted language scope.

This variant of the command "CALL" is not allowed in the restricted language scope.

The addition "FUNCTION <dynamic-function>" is not valid in the restricted language scope.

The statement "EXEC" is invalid. Check the spelling, spelling.

"Data Element S_CARR_ID": The use of this object is not permitted.

"T001" is not declared as a table, projection view, or database view in ABAP Dictionary or does not exist in an active version.

Figure 32: ABAP Language Version 3: Static ABAP with Restricted Object Use

The following statements are not allowed with ABAP Environment 1809 and therefore an error is issued:

Dynamic typing (for example CREATE DATA ... TYPE (dynamic)).

Usage of prohibited external objects (for example ...TYPE s_carr_id).

Forbidden statement (for example CALL SCREEN – screens or dynpros not supported).

Usage of Native SQL forbidden.



Database table **SYCH_VERSION** shows which statements are supported (whitelisted) and which are not (blacklisted)!

Data Browser: Table SYCH_VERSION: Selection Screen

Number of Entries: 1

VERSION: 5 to 5

KEYWORD: to

SUPER: to

RESTRICTION: to

List

VERSION	KEYWORD	SUPER	RESTRICTION
3	ASSIGN-CASTING-DYNAMIC-TYPE	CALL	W
3	ASSIGN-DYNAMIC	ASSIGN	B
3	CALL-CFUNCTION	CALL	B
3	CALL-DATABASE-PROCEDURE	CALL	B
3	CALL-DIALOG-DYNAMIC	CALL	B

Restriction Type

Restriction Type	Short Descript.
W	White
w	Preliminary white
B	Black
b	Preliminary black
R	Read-only (SY component)

Figure 33: Database Table SYCH_VERSION

The report has to be started with value **5** for the Version field. The list can be read this way:

All statements are allowed (whitelisted) apart from the following:

Keyword **assign-casting-dynamic-type** Though there is not exactly a statement of that name, it is easy to understand that the usage of statement ASSIGN (used to assign data objects to field-symbols) with dynamic typing (ASSIGN dataobject TO <fieldsymbol> CASTING TYPE (typename).) is not allowed (blacklisted).

A Simple Way to View the Available Repository Objects for ABAP in the Cloud

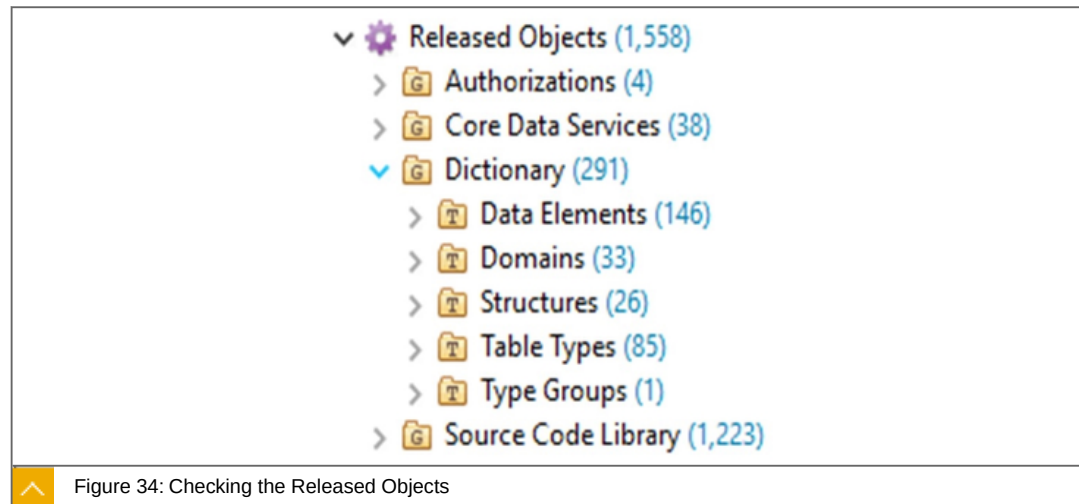


Figure 34: Checking the Released Objects

A simple way to view the available repository objects for ABAP in the cloud, that means version 5, is to check in the ADT of Eclipse the Released Objects.



LESSON SUMMARY

You should now be able to:

Explain ABAP language versions

Unit 3

Lesson 2

Explaining ABAP Features not Supported in SAP BTP, ABAP Environment



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explain ABAP features not supported in SAP Cloud Platform, ABAP environment

ABAP Features not Supported in SAP Cloud Platform, ABAP Environment

All dynamic functions are prohibited with ABAP Environment 1809. This has changed with later versions. File handling is not possible. The usage of cursor access to the database is not possible. Everything that is related to screen (dynpro) programming and programs of type 1 (Executable program – implicitly linked to screen programming) is not possible.

As of release AS ABAP 7.52, in a normal on-premise environment, you can set the language version of your program. In an on-premise environment, the default value is *Standard ABAP*.

This feature allows a developer to check whether their program would work with other ABAP language versions, for example, for ABAP Environment (Version 3, *Static ABAP with Restricted Object Use*).



- In an on-premise environment one can already test whether one's code will be possible in ABAP Environment.
- Of course one can as well test directly in ABAP Environment.

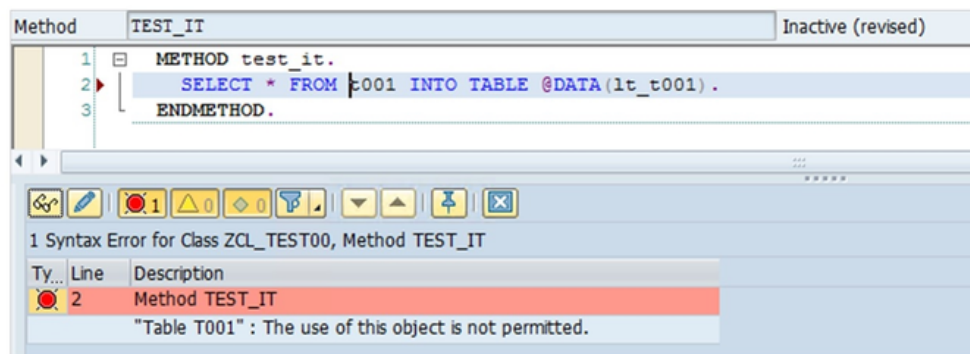


Figure 35: Example: Using Language Version "X" for a Global Class

In an on-premise environment, a developer may already test whether their code can be transferred to ABAP Environment by using ABAP Language Version 3. Of course, it is not mandatory to do this test in an on-premise environment, it is only useful sometimes. If you code in the ABAP Environment, you always implicitly use ABAP language version 3.



Program **DEMO_ABAP_VERSIONS** makes it possible to check ABAP source code using the syntax rules of the different ABAP language versions (on a on-premise system).

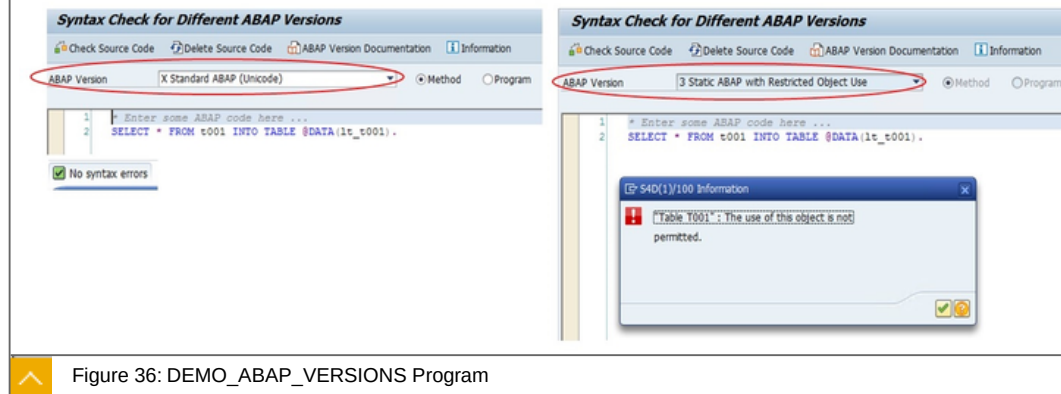


Figure 36: DEMO_ABAP_VERSIONS Program

The DEMO_ABAP_VERSIONS program allows the developer to check out the behavior of the different ABAP Language versions. The developer can easily set and change the ABAP Language version that is to be applied for syntax checking.



LESSON SUMMARY

You should now be able to:

Explain ABAP features not supported in SAP Cloud Platform, ABAP environment

Explaining Released APIs in ABAP Environment



LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Explain released APIs in ABAP environment

Released External Repository Objects in ABAP Environment

Only a very limited number of external repository objects are allowed in ABAP Environment. These repository objects comprise:



Global Classes

A set of basic features for technical, mathematical, and general functions (for example CL_ABAP_MATH, CL_ABAP_RANDOM, CL_ABAP_REGEX).

Interfaces

Same as with Global Classes.

Core Data Services Views

A set of Data Definition Sources including application data (for example I_PLANT, I_SUPPLIER).

Domains

A basic set.

Data Elements

A basic set, some are application specific.

Structure Types

A basic set, some are application specific.

Table Types

A basic set.

External Repository Objects are Development Objects that are not created by the customer but by SAP and that are available in an ABAP Environment. Their availability is made sure by the infrastructure.

It is characteristic for ABAP Environment that there is only a comparatively small number of external objects. While in an on-premise system there can be nearly endless SAP objects (which can be compared to external objects), in an ABAP Environment, this set is very restricted.



Program **ABAP_DOCU_VERSION_WHITELIST** shows the external repository objects that are allowed for the restricted ABAP language versions.

Whitelist of Repository Objects for ABAP Versions			
ABAP Version			
	CLAS	DDLS	TABL
	OBJECTS	OBJECTS	OBJECTS
	/CPD/CL_SC_BADI_UTILITY	C_CUSTOMERSUPPLIERCOMPAN	
	CL_ABAP_CHAR_UTILITIES	C_ENGAGEMENTPROJECTROLES	
	CL_ABAP_CODEPAGE	C_MATERIALSTOCKACTUAL	/CPD/S_SC_PROJECTID
	CL_ABAP_CONTAINER_UTILITIES	C_MATERIALSTOCKBYKEYDATE	ABAP_SORTORDER
	CL_ABAP_CONTEXT_INFO	C_MATERIALSTOCKTIMESERIE	BADI_CHAR_COND
	CL_ABAP_CONV_IN_CE	C_TRIALBALANCEQ0001	BADI_FILTER
	CL_ABAP_CONV_OUT_CE	C_TRIALBALANCEQ0002	BADI_FILTER_BINDING
	CL_ABAP_CONV_X2X_CE	E_INSPECTIONCHARACTERIST	BADI_FILTER_DECLARATION
	CL_ABAP_CORRESPONDING	E_INSPECTIONLOT	BADI_IMPL
	CL_ABAP_DATFM	E_INSPECTIONOPERATION	BADI_INT_COND
	CL_ABAP_DECFLOAT	E_INSPECTIONRESULT	BADI_MAIN
	CL_ABAP_EXCEPTIONAL_VALUE	E_INSPECTIONRESULTVALUE	
	CL_ABAP_FORMAT	E_INSPECTIONSUBSET	
	CL_ABAP_MATCHER	E_INSPLOTUSAGEDECISION	
	CL_ABAP_MATH	I_ACCDOCITEMCATEGORY	
	CL_ABAP_RANDOM		
	CL_ABAP_RANDOM_DECFLOAT16		

(Note: These are arbitrary examples only, list is longer and will be extended in coming releases!)

Figure 37: Program ABAP_DOCU_VERSION_WHITELIST

Which external objects actually are available best can be found out directly in ABAP Environment. In the project explorer, simply open the branch "External content".

Besides, in an on-premise system with the program **ABAP_DOCU_VERSION_WHITELIST**, you can check which repository objects are external objects for which version.

About the Term "Whitelisted", please distinguish between...



Released External Repository Objects for ABAP Language Versions (topic of this lesson).

Released Development Objects (for example, Global Classes are released, Programs of Type 1 are not).

Released ABAP Statements.

Released APIs of External Systems like SAP S/4HANA.

The term **released** appears in several contexts, so there might easily be confusion.

In ABAP Environment there are released External Repository Objects, released Development Objects and released ABAP statements. Other systems like SAP S/4HANA offer released API (see <https://api.sap.com/>).



LESSON SUMMARY

You should now be able to:

Explain released APIs in ABAP environment

Learning Assessment

1. There is one ABAP but different versions.

Determine whether this statement is true or false.

☐ True

☐ False

2. which program is used, to allow developers to check, out the behavior of the different ABAP language versions?

Choose the correct answer.

☐ **A** ABAP_VERSIONS_DEMO

☐ **B** COMPARE_ABAP_VERSIONS

☐ **C** DEMO_ABAP_VERSIONS

3. External Repository Objects are Development Objects that are not created by the customer.

Determine whether this statement is true or false.

☐ True

☐ False

Learning Assessment - Answers

1. There is one ABAP but different versions.

Determine whether this statement is true or false.

☒ True

☐ False

This is correct.

2. which program is used, to allow developers to check, out the behavior of the different ABAP language versions?

Choose the correct answer.

☐ A ABAP_VERSIONS_DEMO

☐ B COMPARE_ABAP_VERSIONS

☒ C DEMO_ABAP_VERSIONS

This is correct.

3. External Repository Objects are Development Objects that are not created by the customer.

Determine whether this statement is true or false.

☒ True

☐ False

This is correct.

UNIT 4

ABAP RESTful Programming Model

Lesson 1

Explaining ABAP RESTful Programming Model	53
---	----

Lesson 2

Explaining ABAP Core Data Services (CDS)	56
--	----

Lesson 3

Creating Data Models	61
----------------------	----

Lesson 4

Explaining Business Services	65
------------------------------	----

Lesson 5

Defining Business Object Behavior	68
-----------------------------------	----

Lesson 6

Enabling Draft	75
----------------	----

UNIT OBJECTIVES

Explain ABAP RESTful programming model

Explain ABAP Core Data Services (CDS)

Define a data model in the ABAP RESTful Programming Model and create and test an OData service for the data model

Explain business services

Define business object behavior and its implementation to enable transactional processing for business objects

Draft-enable a business object to automatically persist transactional data in the backend

Unit 4

Lesson 1

Explaining ABAP RESTful Programming Model

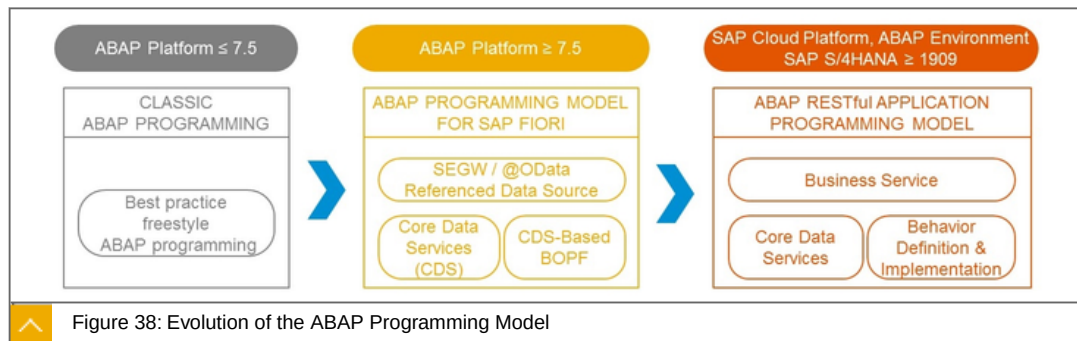


LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explain ABAP RESTful programming model

ABAP RESTful Programming Model, Overview



Classic ABAP Programming

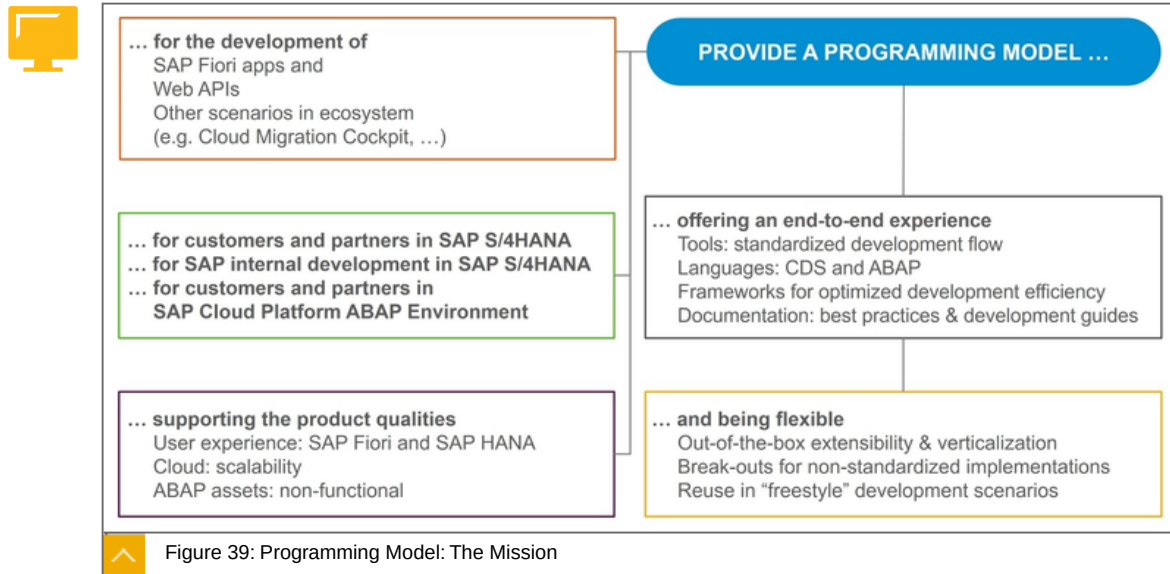
The developer is completely free in their decision on how to access their data model. A good practice is the use of BAPIs wherever it is possible. But generally, there is no decoupling from the data model and no guarantee that the data model does not change with the next software version.

Programming Model for SAP Fiori

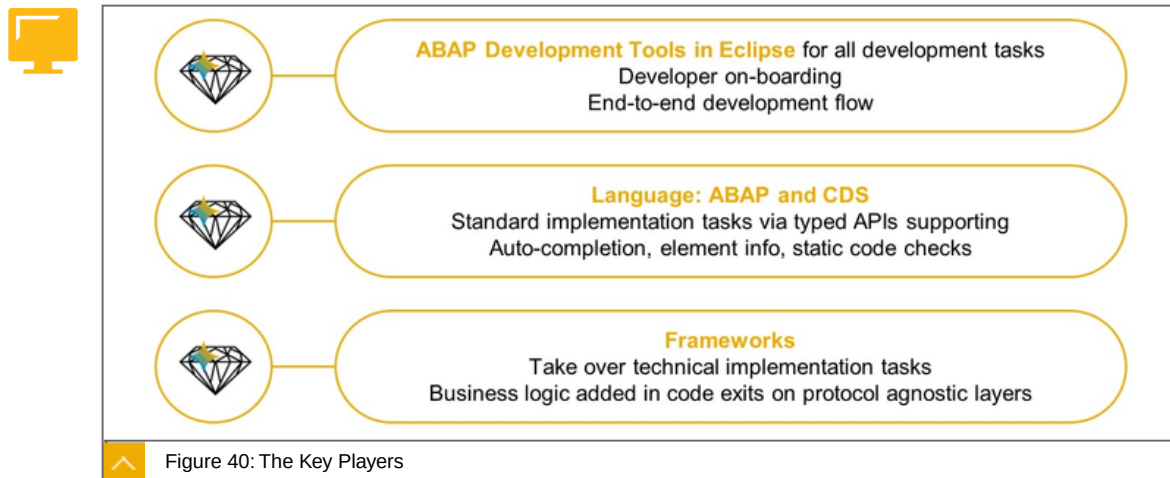
SEGW is a transaction code. It represents the explicit creation and maintenance of OData Services. Business Object Processing Framework (BOPF) is a framework with which the developer can create reusable methods to access the data model. The model is organized along Business Objects which makes it possible to methodically reuse the methods.

ABAP RESTful Programming Model

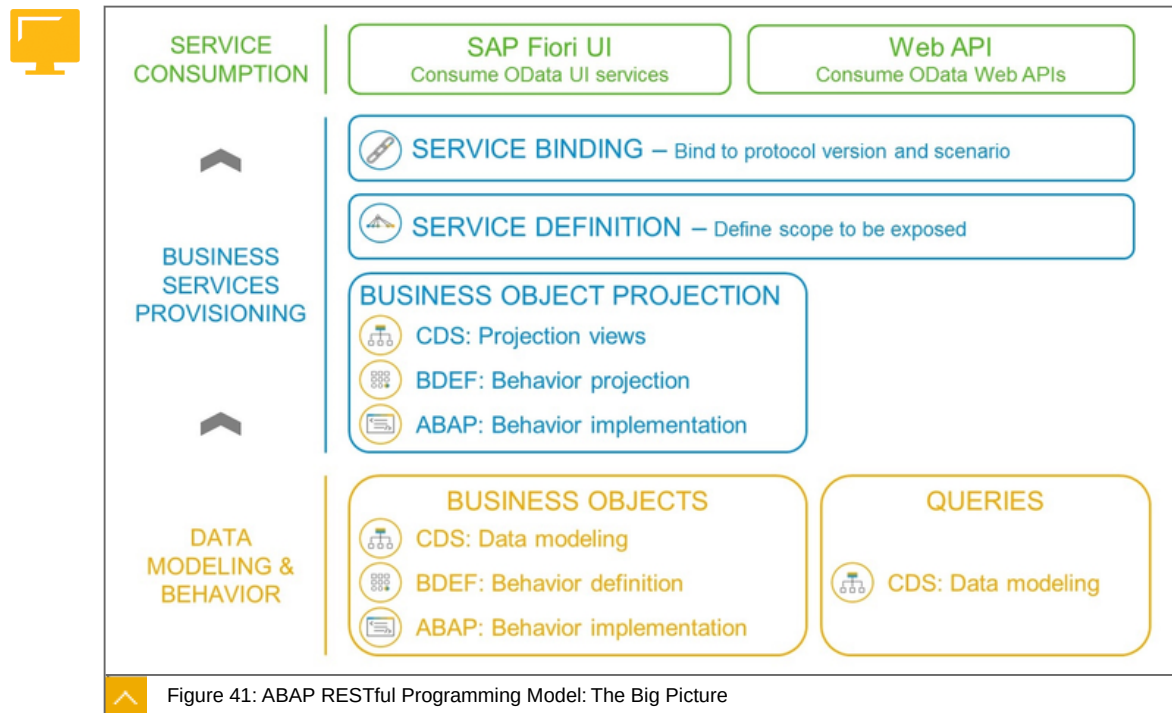
These "new" Business Objects are *first-class ABAP citizens*, meaning they are a known entity to ABAP (unlike BOPF Business Objects). Corresponding OData Services are created automatically.



The objective was to create a programming model that fulfills all aspects that are mentioned here. Note that Web APIs means that an app can be called arbitrarily from somewhere in the cloud.



Everything that is part of the Programming Model can be maintained with ABAP Development Tools (ADT). All relevant entity types (especially Business Object Types) can be addressed directly with ABAP. Frameworks are to perform standardized development tasks at design time.



The following entity types combined with each other implement the ABAP RESTful Programming Model:

Business Objects They are derived from a CDS view. With a Business Object, a Behavior Definition can be defined (which database operations are supported, like update, delete, insert) and a Behavior Implementation can be defined (how exactly is a database operation carried out – actually an implementation of a handler method). **Queries** (Select, Reporting) are possible without any further effort as they directly can make use of the CDS view.

Business Object Projection Used to project and to alias a subset of the business object for a specific business service. The projection enables flexible service consumption as well as role-based service designs. The projection layer exposes the service specific subset of the general business object. The projection layer also enables one business object to be exposed in an OData service for an SAP Fiori UI and for a stable Web API. The service-specific differences can then be implemented in the respective projection layers.

Service Definition The service definition is a projection of the data model and the related behavior to be exposed. A service definition refers to one or several CDS views.

Service Binding Defines a specific communication protocol, such as OData V2 or OData V4, and the kind of service to be offered for a consumer.

Service Consumption OData client consuming the OData service of the service binding, like an SAP Fiori UI or a Web API.



LESSON SUMMARY

You should now be able to:

Explain ABAP RESTful programming model

Unit 4

Lesson 2

Explaining ABAP Core Data Services (CDS)

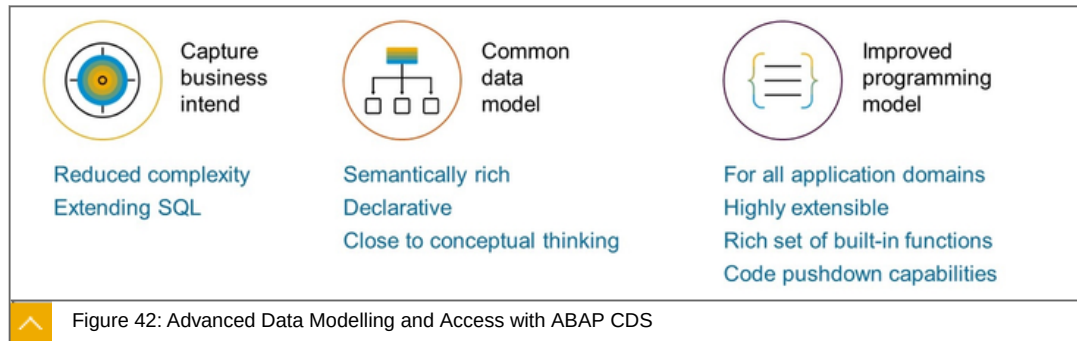


LESSON OBJECTIVES

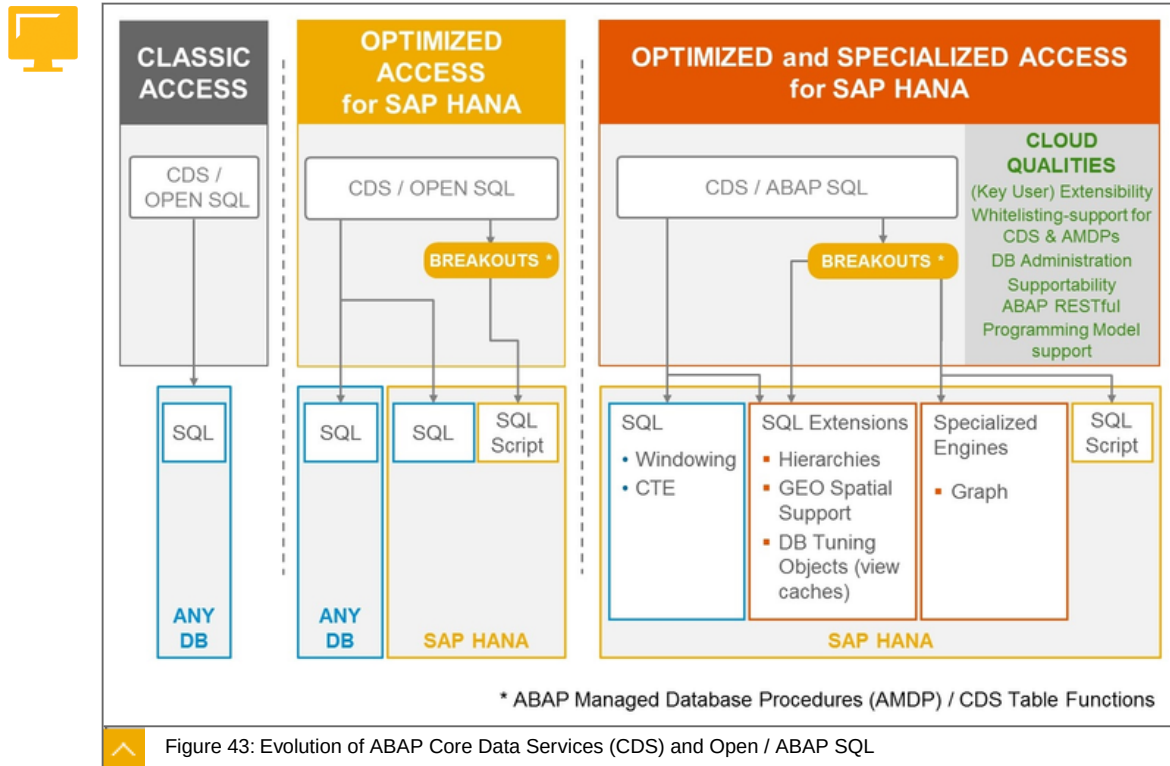
After completing this lesson, you will be able to:

Explain ABAP Core Data Services (CDS)

ABAP Core Data Services (CDS)

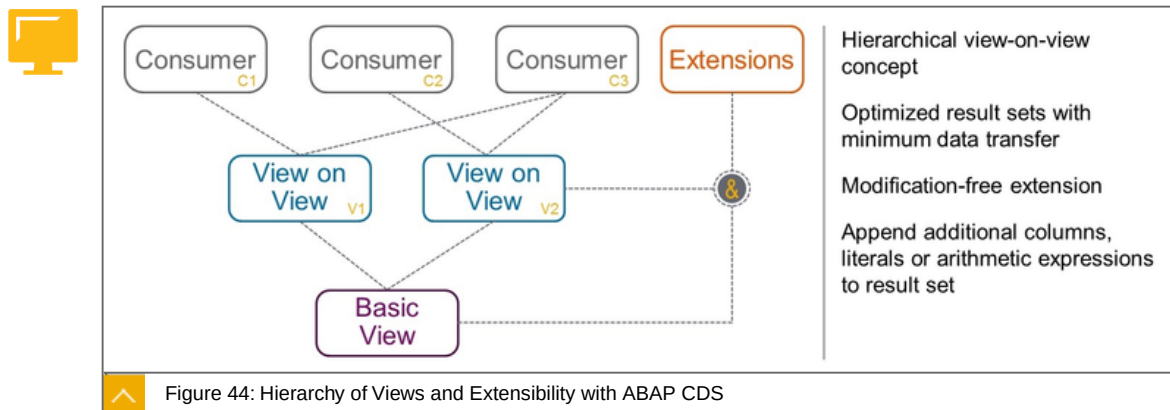


Core Data Services (CDS) Views with their extended SQL options allow you to create complex queries. But their usage is not limited to queries – they can also be the foundation for a complex data model. With annotations, it is possible to add definitions for specific domains (therefore Core Data Services are called semantically rich). For example, it is possible to add information for the generation of SAP Fiori UIs (in this case, the SAP Fiori UI is the specific domain). Core Data Services have more entity types apart from the CDS view, for example, CDS roles. The usage of CDS will improve the way of ABAP programming in many ways, as many aspects are now handled outside of the actual ABAP coding. A complete reuse of CDS entities throughout a system is possible.



Core Data Services are available for *anyDB*. This means in SAP Business Suite they can be used with every Database Management System which is supported by the Database Interface of the ABAP Application Server.

In combination with SAP HANA DB, many special features are supported which are not available with *anyDB*. SQL Script is the Native SQL of SAP HANA DB. It offers options and features that go beyond Open SQL. It is mandatory to use SQL Script when working with ABAP Managed Database Procedures. These are procedures which are executed on the database but are maintained with the ABAP Workbench (ABAP Development Tools – ADT).



With CDS views it is possible that one view can use another view as a data source. This allows for view hierarchies and a better reuse of definitions. This feature was not possible with classical ABAP Dictionary database views. As the result of a view is supplied by the database, there is less need to transport the content of several database tables to the program with the intent to join them there. Instead, the database will supply the information in a way in which the consumer needs it.

CDS views can easily be enhanced (for example, a customer can enhance an SAP CDS view). With an enhancement, it is possible to append additional columns, arithmetic expressions, and so on, to the result set.



Capturing the relationships between business entities in the data model

Used associations are implicitly translated into SQL joins

Reuse of generated joins when semantically identical

Can be queried in CDS views and ABAP SQL

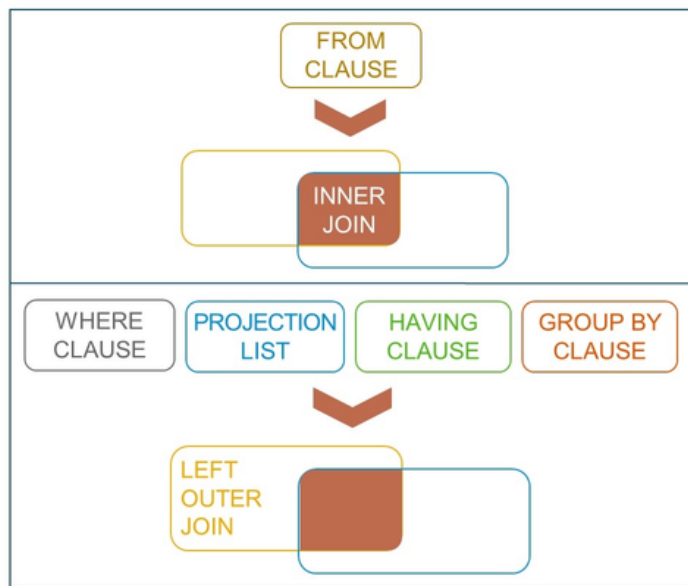


Figure 45: CDS Associations

When defining an ABAP join, the programmer has to describe a join condition, that is, define which fields of the left table are to be compared with which fields of the right table. With associations, this relation between data sources can be described in a more abstracted way. Working with associations has several advantages when consuming the CDS view – in particular, you can describe an evaluation path, that is, which associations are to be used going from one CDS view to the next.

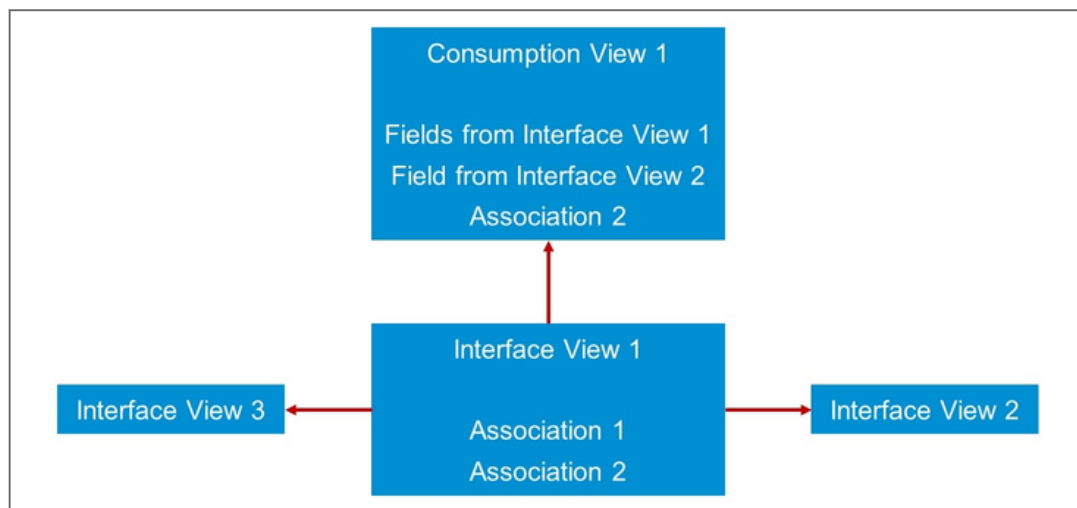


Figure 46: Associations

A CDS view can define associations. An association is a declaration of the relationship of views which can be consumed by itself or by the consumer of this CDS view. Usually, you need to define some reusable CDS views, which define associations as a representation of the

business entity. Then you can write a consumption CDS view, which uses the interface view as its data source, to consume the data and associations.

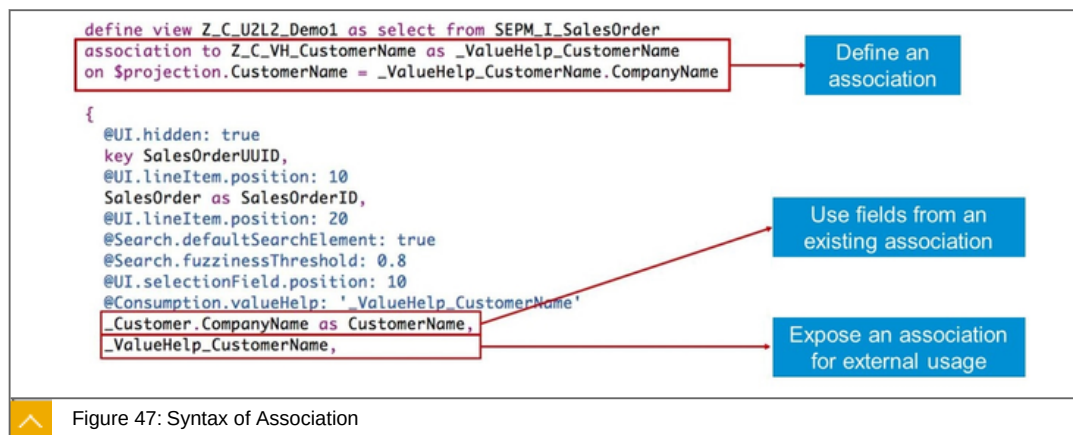
In the figure, Associations, Interface View 1 has two associations connected to Interface View 3 and Interface View 2.

When consumption view 1 accesses data of interface view 1, it can:

Access fields of Interface View 1

Access fields of Interface View 2 through association 1 of interface view 1

Expose Association 2 of Interface View 1 again, to let the consumer of this view have the ability to access fields of Interface View 3



The figure, Syntax of Association, shows the 3 main areas of an association.

These are:

Define an association:

The definition of association is quite like SQL JOIN syntax. A simple way to reference fields of the current projection list is `$projection.<fieldname>`.

Use fields from an existing association:

You can access fields from associated CDS views by connecting association name and field names with a period `'.'`. The association can be defined either in current CDS view or the data source view. (In the figure, **CDS SEPM_I_SalesOrder** exposes the association `_Customer`)

Expose an association for external usage:

To expose an association for consumer of this view, write the name of association in the projection list. Since we cannot distinguish association and fields, usually we use `'_'` as the first character for the association name.



- Enrich meta model with consumer-specific semantics and vocabularies
- Reusable and unified view model for all use cases
- Enablement for flexible usage in different contexts
- Vital part of the ABAP RESTful Programming Model



Figure 48: CDS Annotations for Domain-Specific Frameworks

CDS annotations are possible for different consumers. For example, an SAP Fiori app can be a consumer of a CDS view. So it is possible to annotate in the CDS view how an SAP Fiori UI view shall be rendered at runtime.

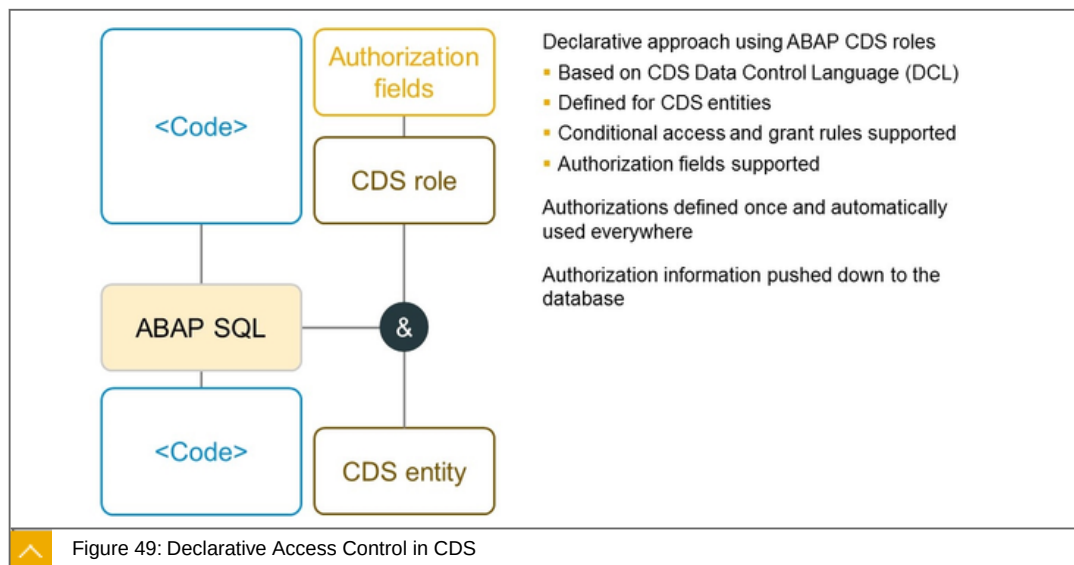


Figure 49: Declarative Access Control in CDS

Before there were CDS roles, it was not possible to link a database object directly to any user authorizations. The classical SAP approach uses authorization objects and authorizations which are linked only indirectly and in an abstracted way to database objects.

With CDS roles, it is possible to store authorization data directly on the database, and of course this information is evaluated directly at runtime. This concept can be combined with the classical approach of the ABAP authorization concept.



LESSON SUMMARY

You should now be able to:

Explain ABAP Core Data Services (CDS)

Creating Data Models



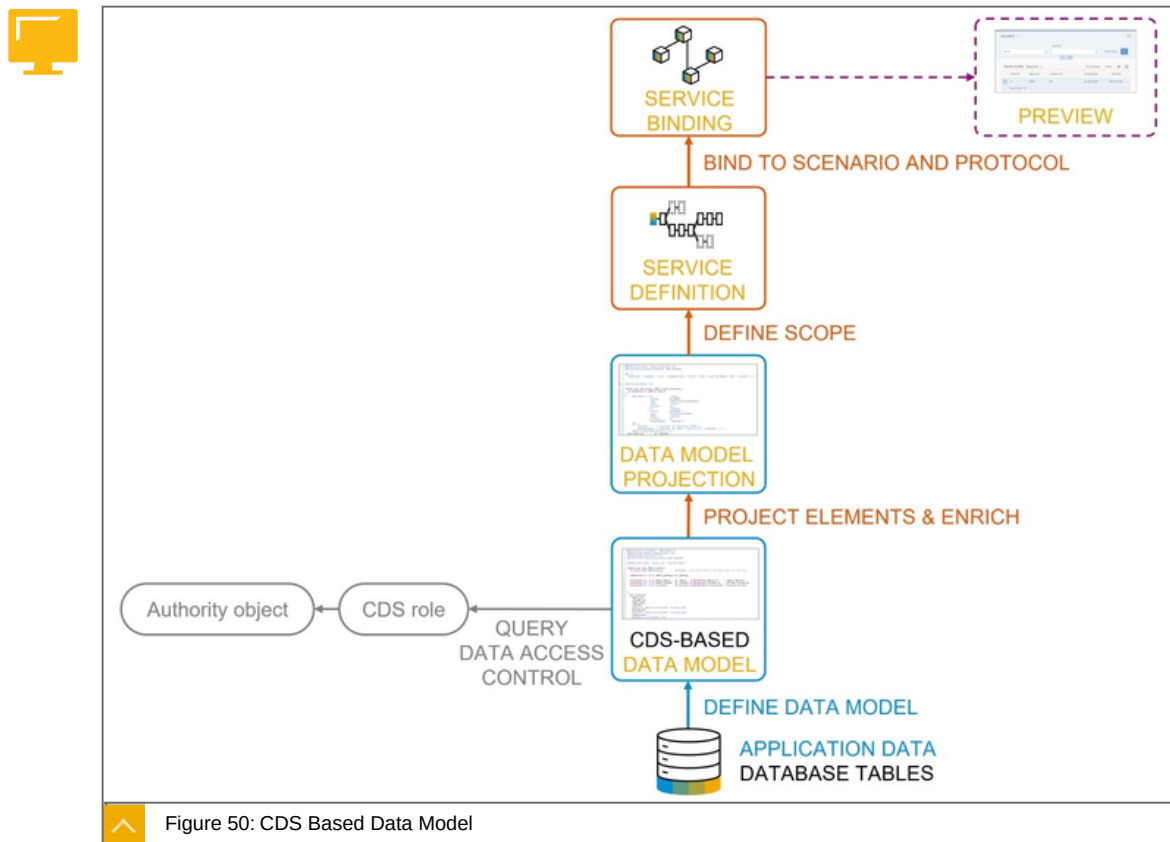
LESSON OBJECTIVES

After completing this lesson, you will be able to:

- Define a data model in the ABAP RESTful Programming Model and create and test an OData service for the data model

Data Model

The data model comprises the description of the different entities involved in a business scenario and their relationships, for example, the parent-child relationship between the entities. The ABAP RESTful Programming Model uses CDS to define and organize the data model. CDS provides a framework for defining and consuming semantic data models. These data models have a physical representation in the database in the form of database views which are automatically created based on a CDS data model. Every real-world entity is represented by one CDS entity. View building capabilities allow you to define application-specific characteristics in the data model. This means that CDS entities are the fundamental building blocks for your application. When using the CDS entity for a data selection, the data access is executed by the SQL-view, which is defined in the CDS entity.



CDS offer a rich set of built-in functions and expressions and have their own data control language to restrict access to the data. The relations between business entities on the data model level can be defined using CDS associations. Domain-specific semantics can be added to data models using CDS annotations.

Depending on the use case, data models support transactional access or query access to the database. Thus, data models are used in business objects or queries respectively. To support different use cases, data model projections are defined. CDS based data model projections allow de-normalization and enrichment of the underlying data model. While the underlying data model is service agnostic and intended for general use, projection views are intended for a specific service and expose only those elements relevant for that service. Projection views de-normalize the underlying data model and define fine-tuning such as value helps, search, and UI semantics.

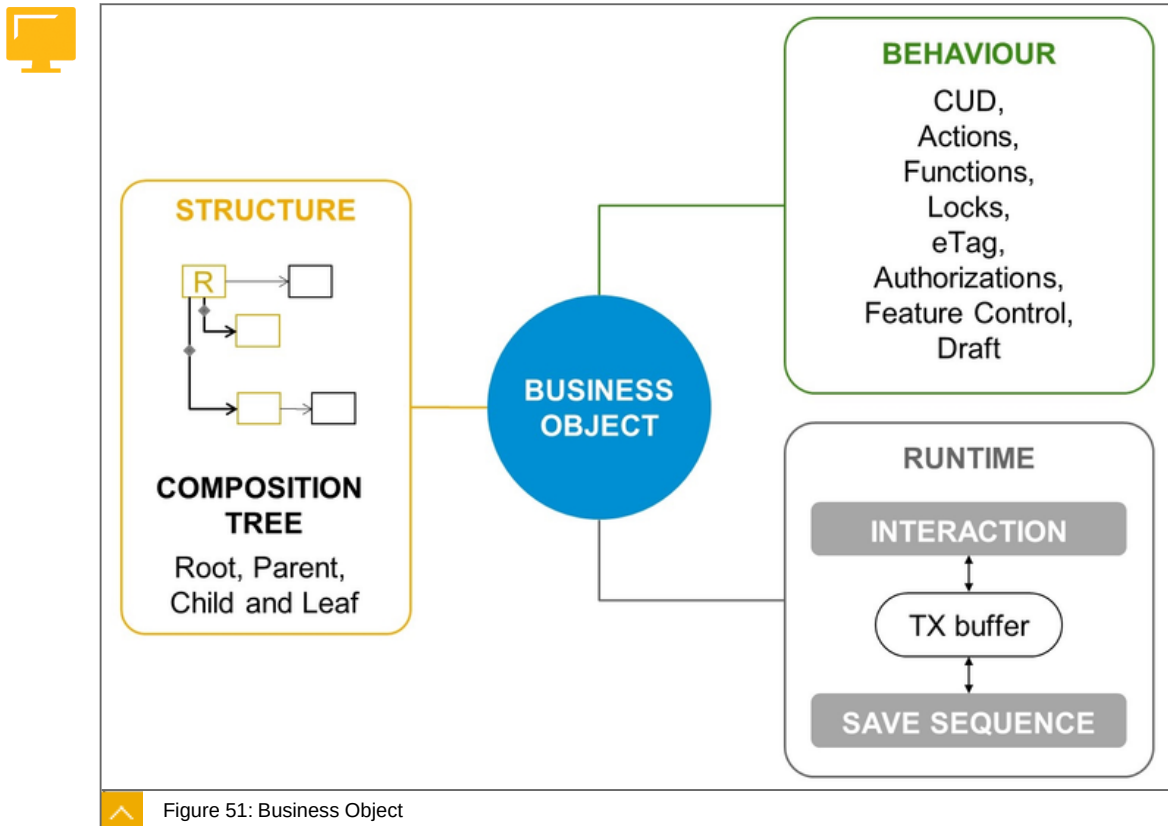
Business Object

A business object (BO) is a common term to represent a real-world artifact in enterprise application development such as the Product, the Travel, or the SalesOrder. In general, a business object contains several nodes such as Items and ScheduleLines and common transactional operations such as for creating, updating, and deleting business data. An additional application-specific operation in the SalesOrder business object might be, for example, an Approve action allowing the user to approve the sales order. All changing operations for all application-related business objects form the transactional behavior in an application scenario.

When going to implement an application scenario based on business objects, we may distinguish between the external, consumer-related representation of a business object and the internal, provider-related perspective:

The **external perspective** hides the intrinsic complexity of business objects. Developers who want to create a service on top of the existing business objects for role-based UIs do not need to know in detail on which parts of technical artifacts the business objects are composed of or how runtime implementations are orchestrated internally. The same also applies to all developers who need to implement a consumer on top of the business object's APIs.

The **internal perspective** exposes the implementation details and the complexity of business objects. This perspective is required for application developers who want to provide new or extend existing business objects for the industries, the globalization and partners.

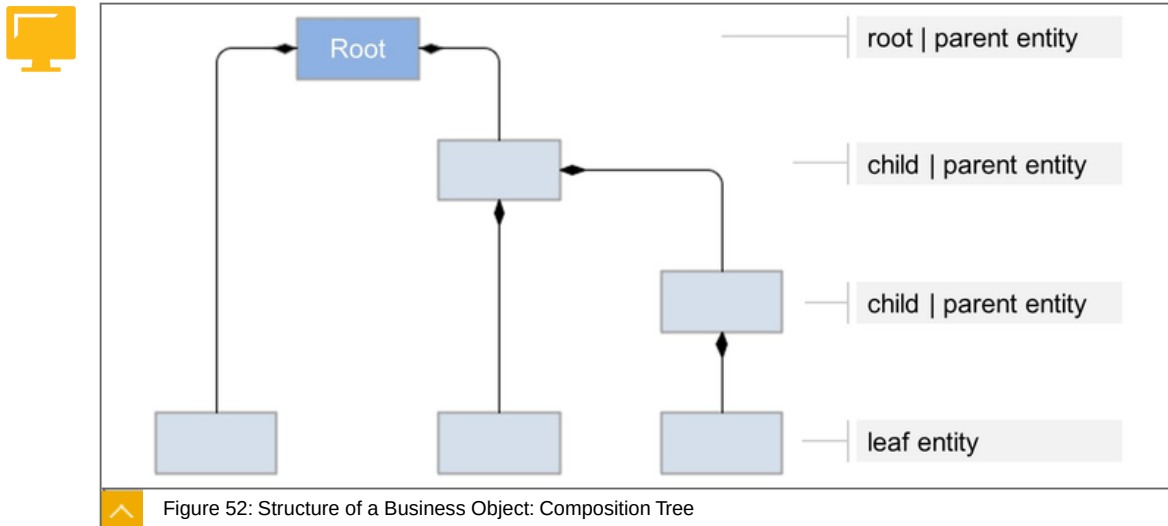


From a formal point of view, a business object is characterized by the following:

- BO Structure
- BO Behavior definition
- BO Runtime implementation

Business Object Structure

From a structural point of view, a business object consists of a hierarchical tree of nodes (SalesOrder, Items, ScheduleLines) where the nodes are linked by special kinds of associations, namely by compositions. A composition is a specialized association that defines a whole-part relationship. A composite part only exists together with its parent entity (whole).



Each node of this composition tree is an element that is modeled with a CDS entity and arranged along a composition path. As depicted in the figure, a sequence of compositions connecting entities with each other builds up a composition tree of an individual business object.

The root entity is of particular importance in a composition tree: The root entity serves as a representation of the business object and defines the top node within a hierarchy in a business object's structure. This is considered in the source code of the CDS data definition with the keyword `ROOT`.

The root entity serves as the source of a composition which is defined using the keyword `COMPOSITION` in the corresponding data definition. The target of this composition defines a direct child entity. On the other hand, CDS entities that represent child nodes of the business object's composition tree must define an association to their compositional parent or root entity. This relationship is expressed by the keyword `ASSOCIATION TO PARENT`. A to-parent association in ABAP CDS is a specialized association which can be defined to model the child-parent relationship between two CDS entities.

In a nutshell: both a sequence of compositions and to-parent associations between entities define the structure of a business object with a root entity on top of the composition tree.

All entities – except the root entity – that represent a node of the business object structure serve as a:

Parent entity – if it represents a node in a business object's structure that is directly connected to another node when moving towards the root.

Child entity – if it represents a node in a business object's structure that is directly connected to another node (parent node) when moving away from the root.

Leaf entity – if it represents a node in a business object's structure without any child nodes. A leaf entity is a CDS entity, which is the target of a composition (a child entity node) but does not contain a composition definition.



LESSON SUMMARY

You should now be able to:

Define a data model in the ABAP RESTful Programming Model and create and test an OData service for the data model

Unit 4

Lesson 4

Explaining Business Services



LESSON OBJECTIVES

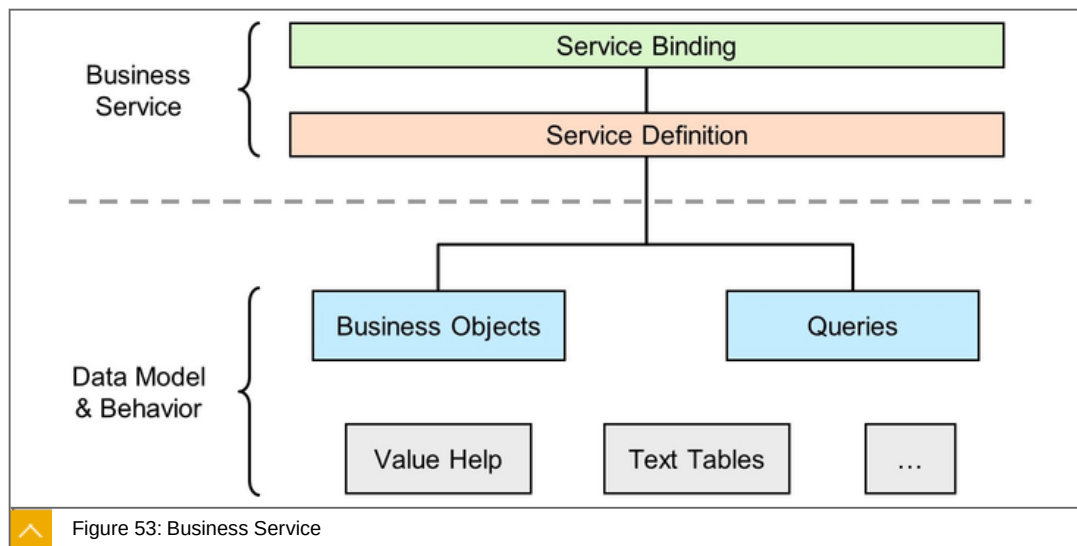
After completing this lesson, you will be able to:

Explain business services

Business Services

The ABAP development platform can act in the roles of a service provider and a service consumer (such as SAP Fiori UI client).

In the context of the ABAP RESTful application programming model, a business service is a RESTful service which can be called by a consumer. It is defined by exposing its data model together with the associated behavior. It consists of a service definition and a service binding.



The programming model distinguishes between the data model and behavior and the service that is defined by exposing these data models together with the behavior. The data model and the behavior layer contain domain-specific semantic entities like business objects, list views, and analytical queries, and, in addition, related functionality such as value help, feature control, and reuse objects.

Separation Between the Service Definition and the Service Binding

In an SAP Fiori UI, many role-based and task-oriented apps are based on the same data and related functionality must be created to support end users in their daily business and in their dedicated roles. This is implemented by reusable data and behavior models, where the data model and the related behavior is projected in a service-specific way. The service definition is a projection of the data model and the related behavior to be exposed, whereas the service binding defines a specific communication protocol, such as OData V2 or OData V4, and the

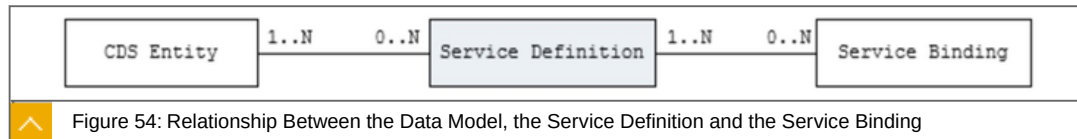
kind of service to be offered for a consumer. This separation allows the data models and service definitions to be integrated into various communication protocols without the hassle of re-implementation.

Service Definition

A business service definition describes which CDS entities of a data model are to be exposed so that a specific business service can be enabled. It is an ABAP Repository object that describes the consumer-specific but protocol-agnostic perspective on a data model. A service definition represents the service model that is generically derived from the underlying CDS-based data model. A service definition is used to define which data is to be exposed as a business service using one or more business service bindings. A service definition itself is independent from the version or type of the protocol that is used for the business service.

Service Binding

The business service binding is used to bind a service definition to a client-server communication protocol such as OData.



A service binding relies directly on a service definition that is derived from the underlying CDS-based data model. Based on an individual service definition, a plurality of service bindings can be created. The separation between the service definition and the service binding enables a service to integrate a variety of service protocols without any kind of re-implementation. The services implemented in this way are based on a separation of the service protocol from the actual business logic.



	Web API	SAP FIORI UI
UI annotation exposure	Not required	Required
Value help	Not required (but replicated code lists)	Required (for UI end-user interaction)
Dynamic feature control	Not required	Required
Draft	Not required	Required
ContentIDs for complex \$batch requests	Required	Not required
Release, versioning and deprecation	Required	Not required

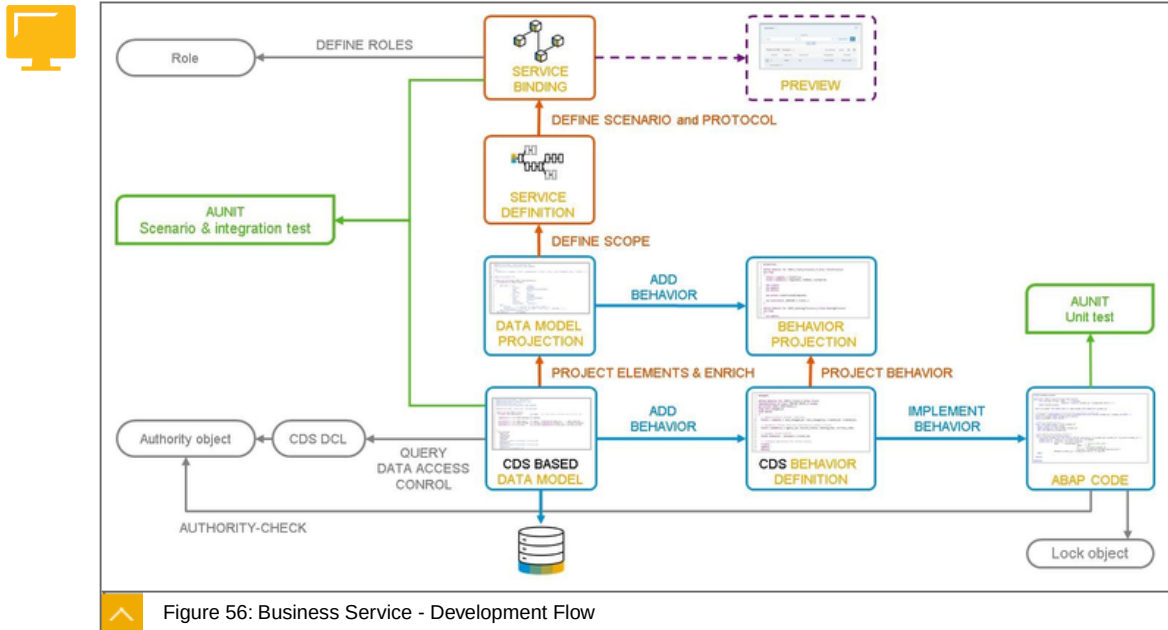
Figure 55: SAP Fiori UI and Web API services

When a service binding is created, you need to select a binding type. This specifies the service type and the specific protocol which is implemented with the service binding. Currently the protocols **OData V2** and **OData V4** are supported. OData V4 services have a wider scope than OData V2 services. It is recommended to use OData V4 wherever possible for transactional

services. The binding type also determines the way a service is offered to a consumer. There are two options of service types:

UI service– A UI service makes it possible to add an SAP Fiori elements UI or other UI clients to the service.

Web API– A service that is exposed as Web API is used for all other use cases apart from UIs. Web APIs can be consumed by an unknown consumer via OData. Web APIs can be extended.



Service definition and binding are defined on top of the business object definition and Implementation. The option to generate a preview from the service binding allows a quick and simple testing of your service during the development of the business object and its data model.



LESSON SUMMARY

You should now be able to:

Explain business services

Unit 4

Lesson 5

Defining Business Object Behavior



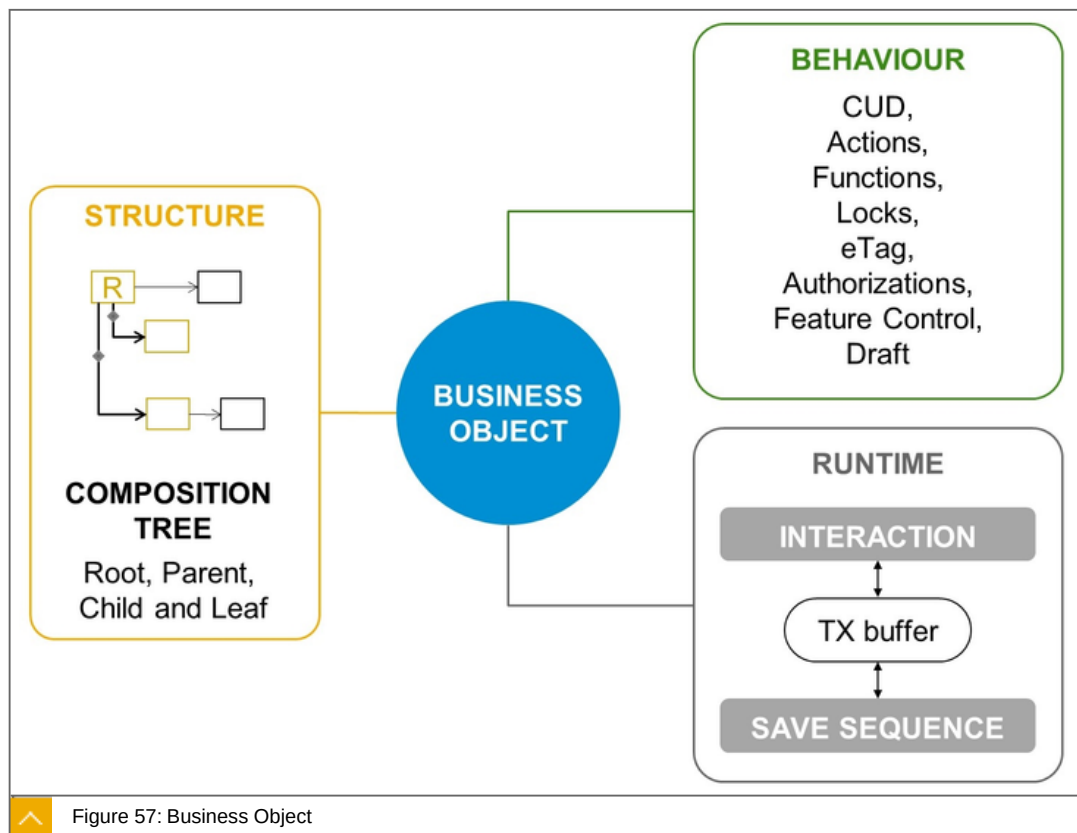
LESSON OBJECTIVES

After completing this lesson, you will be able to:

Define business object behavior and its implementation to enable transactional processing for business objects

Business Object Behavior

In transactional scenarios, the business object behavior defines which operations and what characteristics belong to a business object. For read-only scenarios, the behavior of the data model is defined by the query capabilities, for example, if the data is filterable.



To specify the business object logic, the behavior definition as the corresponding development object is used. A business object behavior definition is an ABAP Repository object that describes the behavior of a business object in the context of the ABAP RESTful application programming model. A behavior definition is defined using the Behavior Definition Language (BDL).



Figure 58: Behavior in the Data Model

A behavior definition always refers to a CDS data model. As shown in the figure, Business Object Behavior, a behavior definition relies directly on the CDS root entity. One behavior definition refers exactly to one root entity and one CDS root entity has at most one behavior definition (a 0..1 cardinality), which also handles all included child entities that are included in the composition tree. The implementation of a behavior definition can be done in a single ABAP class (behavior pool) or can be split between an arbitrary set of ABAP classes (behavior pools). The application developer can assign any number of behavior pools to a behavior definition (1..N cardinality).

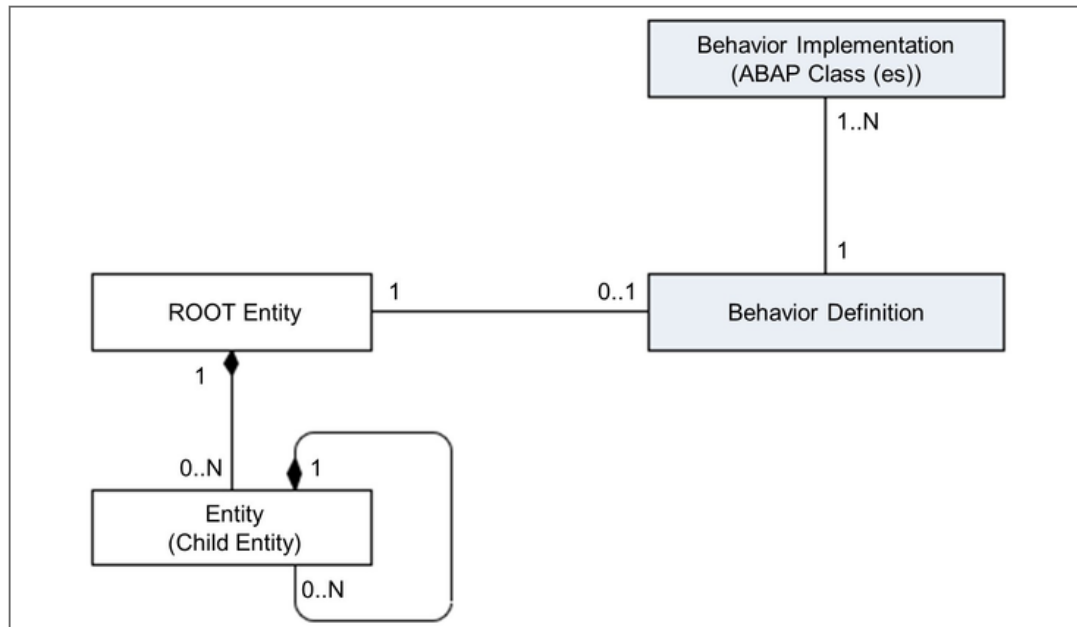


Figure 59: Business Object Behavior

A behavior specifies the operations and field properties of an individual business object in the ABAP RESTful programming model. It includes a behavior characteristic and a set of operations for each entity of the business object's composition tree.

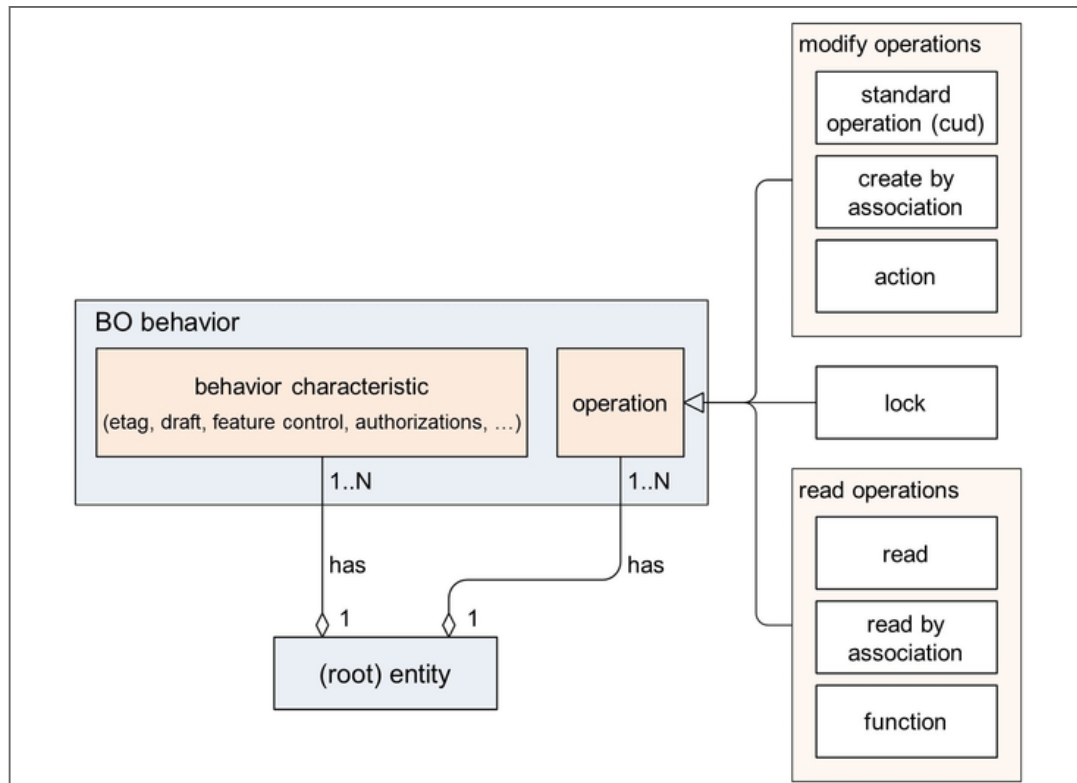


Figure 60: Behavior Characteristics and Operations

Behavior Characteristics

Behavior characteristics are the part of the business object's behavior that specifies general properties of an entity such as:

- ETag

- Draft handling

- Feature control

- Numbering

- Authorizations

Operations

Each entity of a business object can offer a set of operations. They can cause business data changes that are performed within a transactional life cycle of the business object. These modify operations include standard operations like create(), update() and delete() as well as lock implementations and application-specific operations with a dedicated input and output structure which are called actions. Another kind of operations are the read operations: they do not change any business data in the context of a business object behavior. Read operations include read, read by association, and functions (that are similar to actions, however, without causing any side effects).

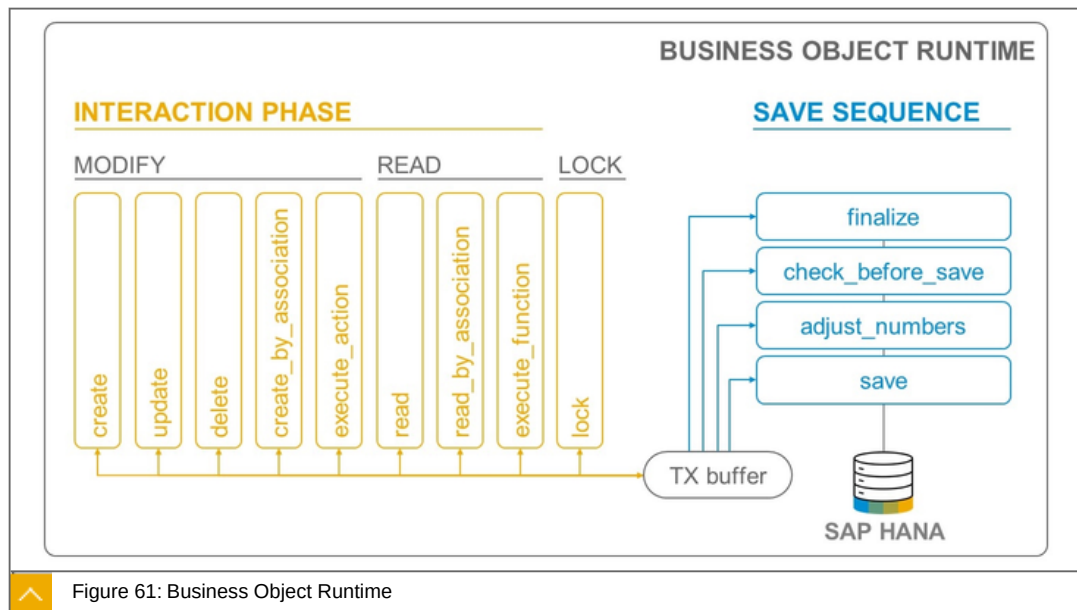
Apart from draft capabilities, these characteristics can be defined for each entity separately.

Business Object Runtime

The business object runtime mainly consists of two parts:

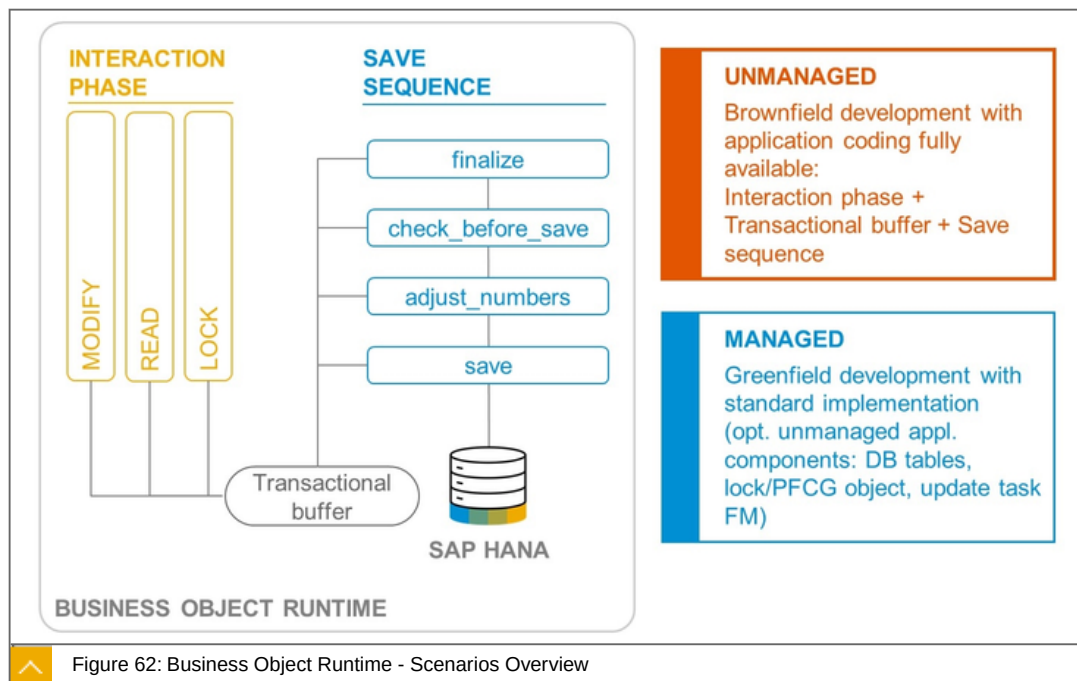
The first part is the interaction phase, in which a consumer calls the business object operations to change data and read instances with or without the transactional changes. The business object runtime keeps the changes in its internal transactional buffer which represents the state of the instance data. This transactional buffer is always required for a business object. After all changes were performed, the data can be persisted. This is realized with the save sequence.

The Business Object Runtime at runtime is implemented with a **Business Object Implementation**. This implementation can be realized in a single ABAP class (behavior pool) or can be split between an arbitrary set of ABAP classes (behavior pools).



The business object runtime mainly consists of two parts, the **Interaction Phase** and the **Save Sequence**.

During the interaction phase, a consumer calls the business object operations to change data and read instances with or without the transactional changes. The business object runtime keeps the changes in its internal transactional buffer which represents the state of the instance data. This transactional buffer is always required for a business object. After all changes were performed, the data can be persisted to the database. This is realized with the save sequence.

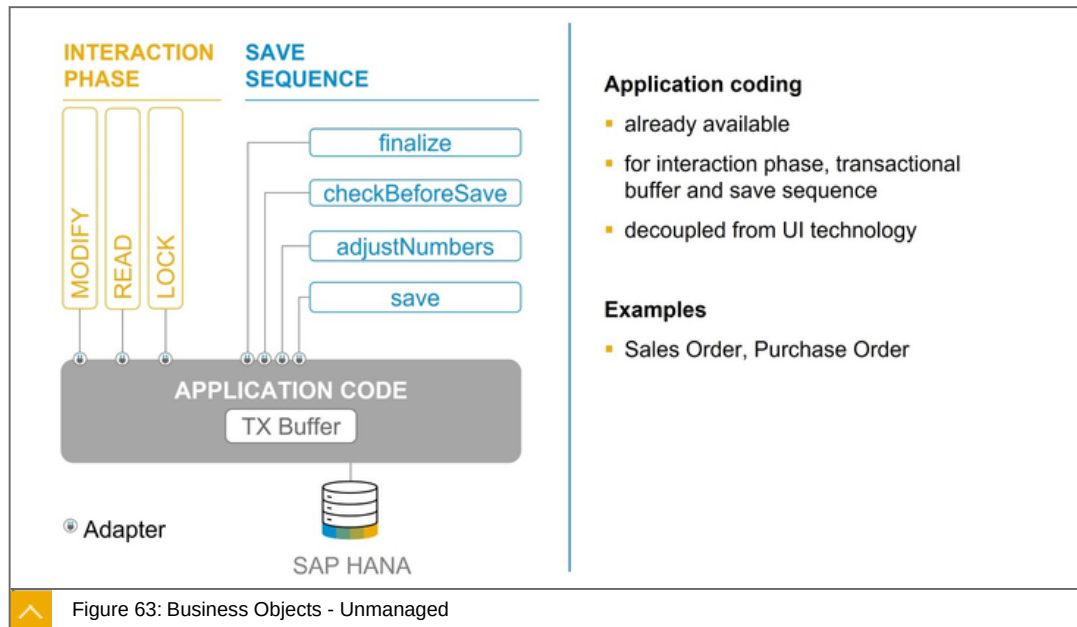


The above scenarios differ in the following way:

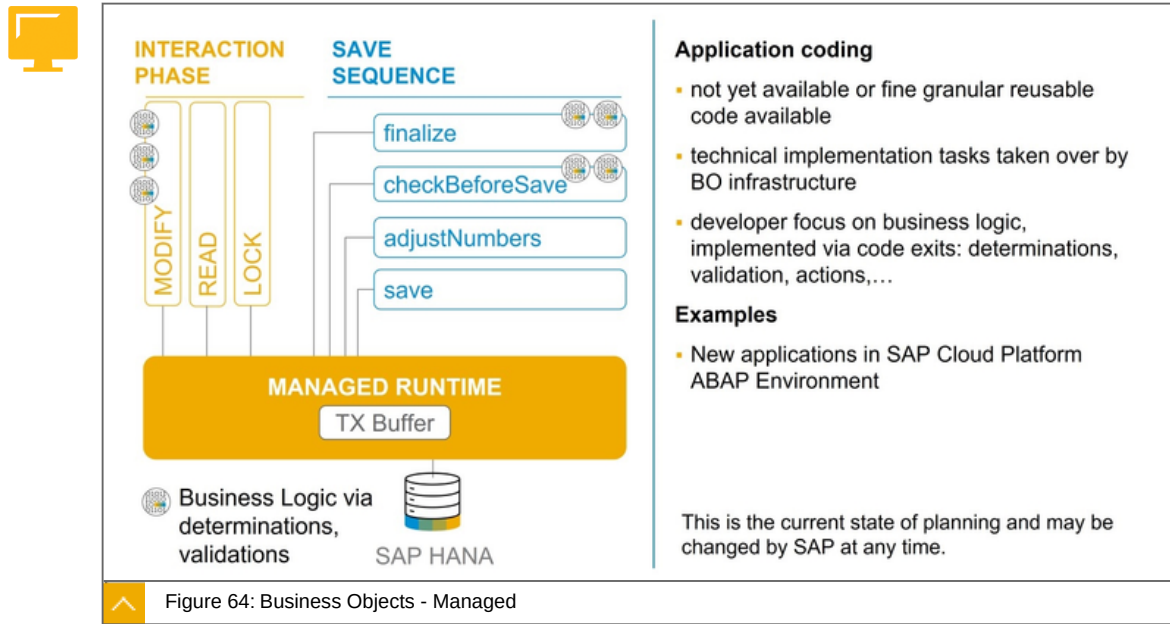
Unmanaged (aka Brown Field Development) For the unmanaged implementation type, the application developer must implement essential components of the REST contract itself. In this case, all required operations (create, update, delete, or any application-specific actions) must be specified in the corresponding behavior definition before they are manually implemented in ABAP. This implementation type is typically chosen if there is already application coding available (existing legacy business logic) and to be reused (for example, BAPIs).

Managed (aka Green Field Development). The managed implementation type is aimed for brand new applications where all essential parts of an application must be developed from scratch. However, these new applications can highly benefit from out-of-the-box support for transactional processing. Whereas for the unmanaged implementation type, the application developer must implement essential components of the REST contract manually, for the managed scenario, all required standard operations (create, update, delete) must only be specified in the behavior definition to obtain a ready-to-run business object. The technical implementation aspects are taken over by the business object runtime infrastructure. In this case, the business object framework implements the interaction phase and the save sequence generically. The application developer can then focus on business logic that is implemented using actions, validations and determinations and user interaction. The corresponding BO runtime manages the entire life cycle of your business objects and covers all aspects of your business application development.

Save self-implemented This is a speciality of the *Managed* scenario. As much as possible is taken from the framework, but apart from the coding exits, the developer has to implement the Save which they can do with Update Function Modules (CALL FUNCTION ... IN UPDATE TASK).



Recap: If there is already application coding available (Brown Field Development) this coding can be transferred into the methods of the interaction phase and save sequence, including necessary adoptions.



Recap: This is only applicable if the scenario is newly created (Green Field Development). In this scenario, the methods will be implemented by the framework. Still, there are options for the developer to influence the behaviour via coding exits.



LESSON SUMMARY

You should now be able to:

Define business object behavior and its implementation to enable transactional processing for business objects

Unit 4

Lesson 6

Enabling Draft



LESSON OBJECTIVES

After completing this lesson, you will be able to:

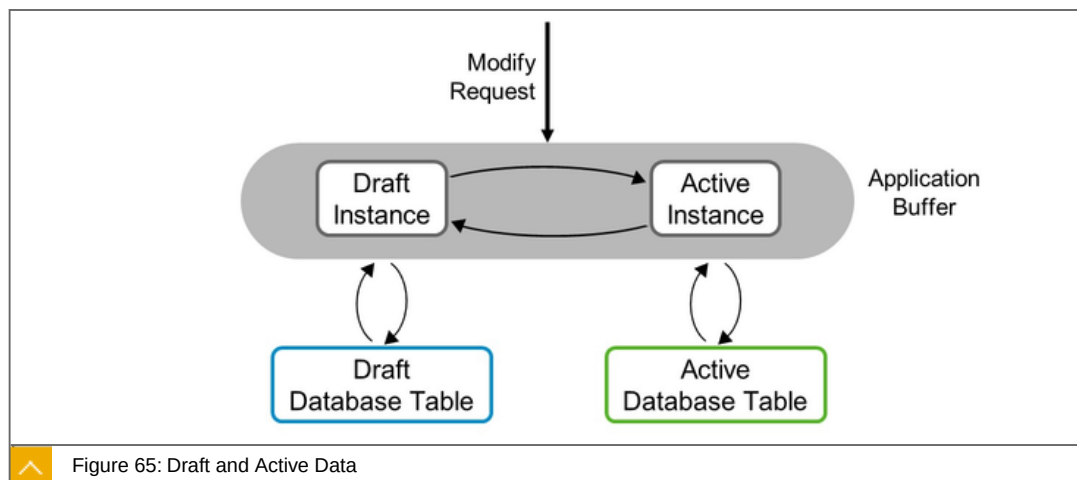
Draft-enable a business object to automatically persist transactional data in the backend

Draft

Draft handling belongs to the behavior characteristics of a business object. Draft-enabled business objects automatically persist transactional data in the backend. This approach supports stateless communication for an application.

Modern cloud-ready apps require a stateless communication pattern, for example, to leverage cloud capabilities like elasticity and scalability. Thus, there is no fixed backend session resource along a business transaction for each user and the incoming requests can be dispatched to different backend resources, which supports load balancing. On the other hand, apps are stateful from the end-user perspective. Business data that is entered by the end user needs to be locked, validated, and enriched via ABAP business logic on the backend side.

The draft concept fills the gap between a stateless communication pattern and a stateful application by applying REST principles. The draft represents the state and stores the transactional changes on the database in shadow tables. It is an addressable resource, the exact copy of the active data that is currently being edited. Between two backend roundtrips, there is no running ABAP session waiting for the next roundtrip.



Draft-enabled applications allow the end user to store changed data in the backend and continue at a later point in time or from a different device, even if the application terminates unexpectedly. This kind of scenario needs to support a stateless communication and requires a replacement for the temporary in-memory version of the business entity that is created or

edited. This temporary version is kept on a separate database table and is known as draft data. Drafts are isolated in their own persistence and do not influence existing business logic until activated.



Advantages of Draft for an application:

- ▣ **Save & Continue**
 A draft allows you to stop processing and saving of business data at any time and to continue processing later on, no matter if the data is in a consistent state or not
- ▣ **Staging**
 The draft works like a staging area so that business data is isolated during processing and can finally be activated, that is, saved to the active database
- ▣ **Device Switch**
 As the draft persists the state of processing, it can be resumed with any device. For instance, you can edit data on a laptop and continue from a mobile device



Figure 66: Advantages of Draft

State Handling

Applications usually have a transactional state that buffers all changes of a current transaction until it is committed by the application user. If the buffer state is consistent, the commit is accepted, if not, the whole transaction is rejected. This all-or-nothing-approach prevents inconsistent data on the database table.

For smaller applications, sending the whole transactional state to the backend is unproblematic. The state can be managed completely on the frontend side. We speak of frontend state handling in that case. However, in case of complex backend-located business logic (especially legacy code), configuration, and data volume, transferring everything to the frontend layer does not work for performance, maintenance, and security reasons. Thus, backend state handling (draft) is required. Basic principles of draft are:

The main business logic is implemented on the active entity, for example, with actions or feature control. This behavior can be applied to draft entities in the same manner.

There can be only up to one draft instance for each active instance at the same time.

The primary key value of a draft and the corresponding active instance is the same.

The current version of the ABAP RESTful Application Programming Model only supports the exclusive draft. A draft exclusively belongs to the user that created the draft. Only this user is able to see and process the draft instance.

Draft is an option that you can use for application development with both the managed and unmanaged implementation type and also with mixed scenarios, for example, managed with unmanaged save. In all scenarios, the draft is managed. That means that it is handled by the RAP runtime framework. You, as an application developer, do not need to care about how draft data is written to the draft database table. This is done for you. Of course, adding draft capabilities to your business object might imply changes in your business logic also for the processing of active data that you are responsible for. In addition, RAP also offers implementation exits for cases in which you need business service-specific draft capabilities that impact the draft handling.

Draft Design Time

Draft business objects need two separate database tables for each entity: One for the active persistence and one for storing draft instances.

The draft data is stored on a separate draft table to match the different access patterns of draft and active data. While the active database table stores many instances, but rarely has access on these instances, it is vice versa for the draft table. With two separate tables, the performance can be adapted to the different approaches of the two tables more easily.

With using a separate database table for the draft information, it is guaranteed that the active persistence database table remains untouched and thus consistent for existing database functionality. The draft database table contains exactly the same fields as the active database table plus some technical information the RAP runtime needs to handle draft. The technical information is added with the draft admin include (include **syh_bdl_draft_admin_inp**

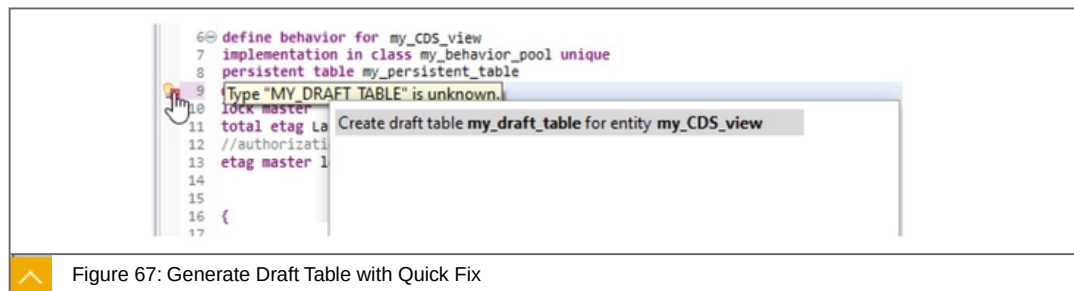


Figure 67: Generate Draft Table with Quick Fix

The draft table can be generated automatically via a quick fix in the behavior definition, which is offered as soon as the business object is draft-enabled with the syntax **with draft**. Whenever you change the active data model in the active database table, you have to regenerate the draft table to align them. The behavior definition offers a quick fix for regeneration, but it is the responsibility of the application developer to keep the two database tables synchronized. When regenerating the draft database table, it completely overwrites the existing one.

Total ETag

The total ETag is a designated field in a draft business object to enable optimistic concurrency checks during the transition from draft to active data.

Draft business objects require a total ETag. This designated field on the database table is updated by the RAP runtime framework as soon as an active instance is changed. The total ETag always refers to the complete BO. As soon as an instance of any BO entity is changed, the total ETag is updated. Its value is compared when resuming a draft instance to ensure that the active data has not been changed after the exclusive lock has expired. Thus, the total ETag is important for optimistic concurrency control during the transition from draft to active data. The total ETag is defined in the behavior definition.

The total ETag field is defined on the lock master entity (currently identical to root entity) and optimistically controls the whole business object regarding concurrency on the active data. The RAP managed framework is able to update the total ETag field automatically on saving a draft instance if the following conditions are fulfilled:

The field is annotated in CDS with the annotation **@Semantics.systemDateTime.lastChangedAt: true** with the precondition that the field has a date-compatible type

The total ETag field is included in the field-mapping prescription in the behavior definition

**Note:**

Total ETag and ETag master/dependent are two sides of the same medal. For a smoothly running application, you need both ETags.

The total ETag is used for edit-drafts, a draft that has a corresponding active instance. As soon as the exclusive lock expires and an edit-draft is resumed, the total ETag value of the draft instance is compared to the total ETag value on the active instance. Only if the values coincide can the draft be resumed. The total ETag is compared for all entities of a BO. Draft business objects requires a total ETag to ensure optimistic concurrency comparison.

The ETag master/dependent concept ensures that the end user of an OData service only changes instances with the state that is displayed on the UI. Hence, the ETag master/dependent prevents changes of the BO that are not noticed by the OData consumers. With ETag master, each BO entity can be checked independently. ETag master fields are required to provide optimistic concurrency locking for OData consumers.

Since the total ETag and the ETag master serve different purposes, you must use separate fields to enable both functionalities.

**LESSON SUMMARY**

You should now be able to:

Draft-enable a business object to automatically persist transactional data in the backend

Learning Assessment

1. Which of the following entity types implement the ABAP RESTful programming model?

Choose the correct answers.

- ☐ **A** Business Objects Projection
- ☐ **B** Service Consumption
- ☐ **C** Service Binding
- ☐ **D** Service definition

2. Which SQL language is mandatory when working with ABAP managed database procedures?

Choose the correct answer.

- ☐ **A** Open SQL
- ☐ **B** SQL Script
- ☐ **C** SQL Extended

3. All entities of a structure (except the root entity) that represent a node of the business object structure serve as a:

Choose the correct answers.

- ☐ **A** Leaf entity
- ☐ **B** Parent entity
- ☐ **C** Branch Entity
- ☐ **D** Child entity

4. A service definition is used to define which functions are to be exposed as a business service using one or more business service bindings.

Determine whether this statement is true or false.

- ☐ True
- ☐ False

5. A behavior definition is defined by using the ABAP Restful Definition Toolset.

Determine whether this statement is true or false.

☐ True

☐ False

6. The draft concept fills the gap between a stateless communication pattern and a stateful application by applying REST principles.

Determine whether this statement is true or false.

☐ True

☐ False

Learning Assessment - Answers

1. Which of the following entity types implement the ABAP RESTful programming model?

Choose the correct answers.

- ☒ **A** Business Objects Projection
- ☒ **B** Service Consumption
- ☒ **C** Service Binding
- ☒ **D** Service definition

This is correct. All answer options are correct.

2. Which SQL language is mandatory when working with ABAP managed database procedures?

Choose the correct answer.

- ☐ **A** Open SQL
- ☒ **B** SQL Script
- ☐ **C** SQL Extended

This is correct. SQL Script is mandatory, when working with ABAP managed database procedures.

3. All entities of a structure (except the root entity) that represent a node of the business object structure serve as a:

Choose the correct answers.

- ☒ **A** Leaf entity
- ☒ **B** Parent entity
- ☐ **C** Branch Entity
- ☒ **D** Child entity

This is correct. A branch entity does not exist. All other answer options are correct.

4. A service definition is used to define which functions are to be exposed as a business service using one or more business service bindings.

Determine whether this statement is true or false.

☐ True

☒ False

This is correct. A service definition is used to define which data is to be exposed as a business service using one or more business service bindings.

5. A behavior definition is defined by using the ABAP Restful Definition Toolset.

Determine whether this statement is true or false.

☐ True

☒ False

This is correct. A behavior definition is defined by using the Behavior Definition Language (BDL).

6. The draft concept fills the gap between a stateless communication pattern and a stateful application by applying REST principles.

Determine whether this statement is true or false.

☒ True

☐ False

This is correct.

UNIT 5

Side-by-Side Extensions between SAP BTP, ABAP and Remote APIs

Lesson 1

Explaining Connectivity Services	84
----------------------------------	----

Lesson 2

Accessing Internet-Facing API by URL	87
--------------------------------------	----

Lesson 3

Consuming an SAP API from the Business Hub API	89
--	----

Lesson 4

Consuming an On-Premise OData Service	94
---------------------------------------	----

Lesson 5

Consuming an On-Premise RFC	96
-----------------------------	----

Lesson 6

Using a Remote Client Proxy	98
-----------------------------	----

UNIT OBJECTIVES

- Learn how to use remote HTTP and RFC based APIs in Cloud Platform ABAP
- Create ABAP applications to consume HTTP APIs from the Internet
- Create ABAP applications to consume on-premise HTTP and RFC APIs
- Configure HTTP and RFC destinations
- Create service consumption model for foreign OData service
- Create and use a custom remote client proxy in ABAP

Unit 5

Lesson 1

Explaining Connectivity Services



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Learn how to use remote HTTP and RFC based APIs in Cloud Platform ABAP

Overview of Connectivity Services

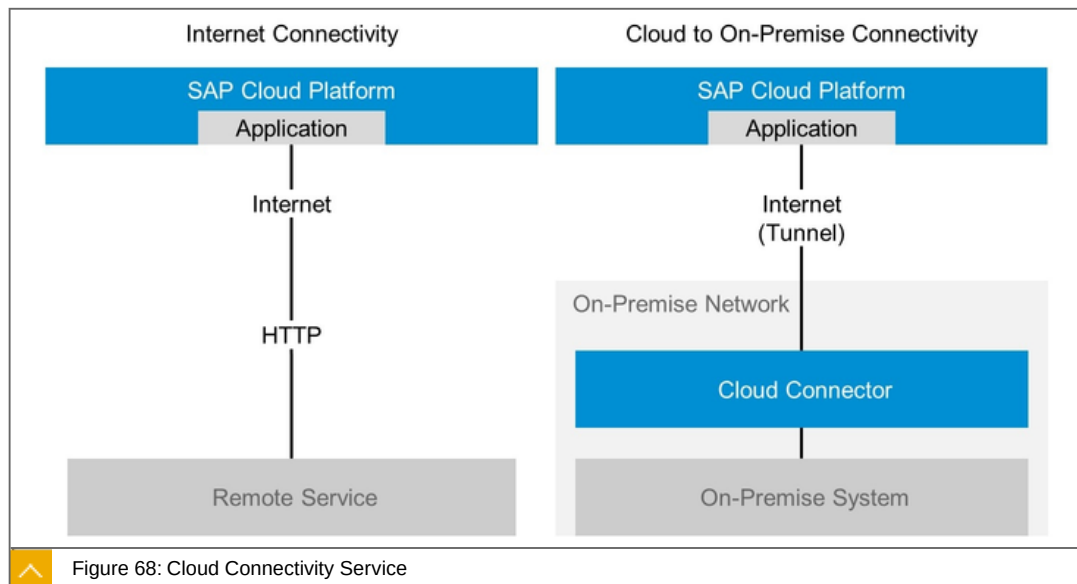
To extend your ABAP application logic with data or services provided using APIs in remote back-end systems, you must use the connection services in the SAP Cloud Foundry environment. The Connectivity service allows SAP BTP applications to securely access remote services that run on the Internet or on premise. This service provides the following functions:

Allows subaccount specific configuration of application connections via destinations.

Provides an API that application developers can use to consume remote services.

Allows you to make connections to on-premises systems, using the Cloud Connector.

Lets you establish a secure tunnel from your on-premises network to applications on SAP BTP, while you keep full control and auditability of what is exposed to the cloud.



The following scenario is a typical one for connectivity from your on-premises network to SAP BTP:

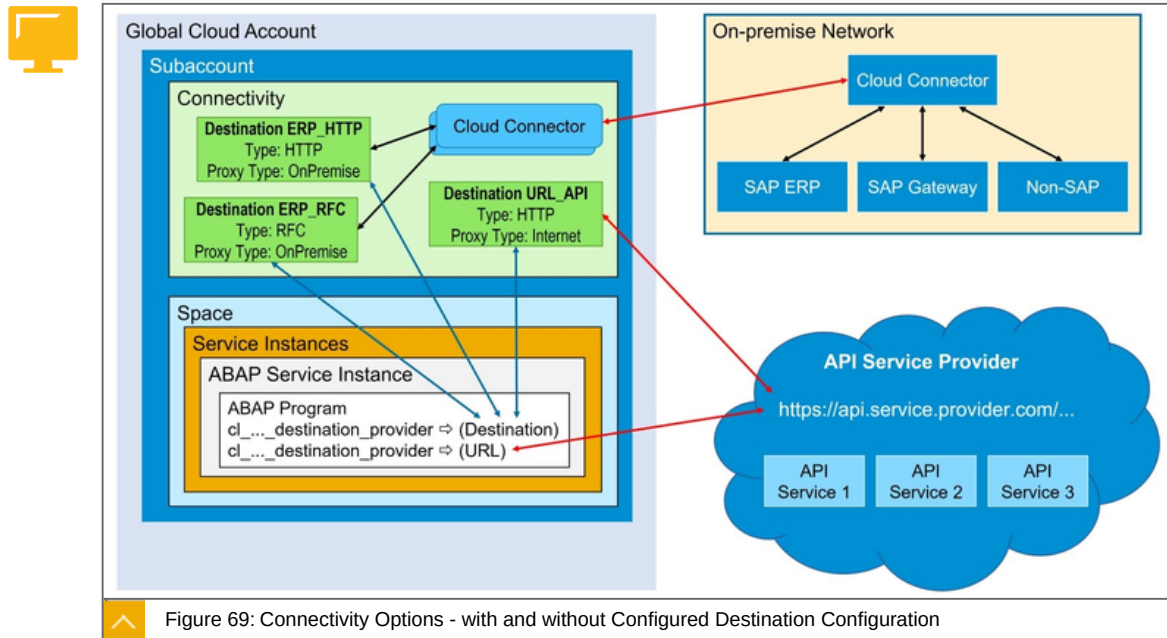
Your company owns a global account on SAP BTP and one or more subaccounts that are assigned to this global account.

Using SAP TP, you subscribe to or deploy your own applications.

To connect to these applications from your on-premises network, the Cloud Connector administrator sets up a secure tunnel to your company's subaccount on SAP BTP.

The platform ensures that the tunnel can only be used by applications that are assigned to your subaccount.

Applications assigned to other (sub)accounts cannot access the tunnel. It is encrypted via transport layer security (TLS), which guarantees connection privacy.



In principle, there are 2 options to establish a connection in your application to a foreign system. The connectivity service can connect directly and indirectly:

It can connect directly to an Internet facing API using the HTTP client without destination definition by directly providing the URL in the application program. Only the HTTP protocol is supported and all service specific connection details, like with user authentication for example, must be provided in the application coding.

It can connect indirectly through the destination service via reusable destinations. Both HTTP and RFC protocol are supported. Service-specific connection details, like with user authentication for example, can be securely maintained in the destination definition. This is SAP's recommended option.

Destinations are part of SAP Cloud configurations, generally used for the outbound communication of an SAP Cloud application to a foreign system – SAP or non-SAP. They contain the connection details for the remote communication of an application. Use the Destinations editor in SAP BTP cockpit to configure HTTP or RFC destinations to connect your Web application to the Internet or consume an on-premise's backend system API via HTTP(S) or RFC.

Table 3: Supported Connectivity Service Protocols for Destination Definition

Protocol	Details
HTTP(S)	Exchange data between your cloud application and Internet services or on-premise systems. Create and configure HTTP destination to make Web connections. Connect to on-premise with HTTP using the Cloud Connector.
RFC	Invoke on-premise ABAP function modules using RFC. Create and configure RFC destinations. Make connections to back-end systems with RFC using the Cloud Connector.

**LESSON SUMMARY**

You should now be able to:

Learn how to use remote HTTP and RFC based APIs in Cloud Platform ABAP

Unit 5

Lesson 2

Accessing Internet-Facing API by URL



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Create ABAP applications to consume HTTP APIs from the Internet

Access an Internet-Facing API by URL

One option to access or consume an Internet-facing API is to use the connectivity service and to provide the service URL and other service-related connection details directly in the application coding. While this works perfectly well, it is not recommended. Similar to how in the on-premise world we define destinations in transaction **SM59**, we can separate application-related implementation logic and connection and authentication details from one another. This provides greater flexibility in terms of re-usability and removing connection-specific details like user authentication and other security sensitive information out of the ABAP code. We will cover destination configuration and destination usage in the subsequent lessons.

In order to establish a connection to an internet-facing API, the connectivity service provides 3 methods in class `CL_HTTP_DESTINATION_PROVIDER`:



▼ **CL_HTTP_DESTINATION_PROVIDER** *Provider Class for HTTP Destinations*
 • **CREATE_BY_URL** *URL-based HTTP Destination*
 • **CREATE_BY_DESTINATION** *RFC-Destination-based HTTP Destination*
 • **CREATE_BY_CLOUD_DESTINATION** *SAP CP CF Destination Service-based HTTP Destination*

▼ **CL_WEB_HTTP_CLIENT_MANAGER** *HTTP Client Manager for Web Development*
 • **CREATE_BY_HTTP_DESTINATION** *Creates an HTTP Client for a given HTTP Destination Object*

Figure 70: Connectivity Services – HTTP Destination Provider and HTTP Client Manager

The `CREATE_BY_DESTINATION` method is for destinations which are defined in transaction **SM59** and therefore limited to be used in ABAP applications running on an on-premise system.

The `CREATE_BY_URL` and `CREATE_BY_CLOUD_DESTINATION` methods are the ones that we are focusing on in this unit. In particular, we focus on `CREATE_BY_URL`. This method (or one of the other two) has to be used to obtain the destination object. The destination object is required to create the client object using the `CL_WEB_HTTP_CLIENT_MANAGER` class.

With the client object created, the request and the response objects of the external API can be accessed.



```

50 METHOD if_oo_adt_classrun-main.
51 TRY.
52     "establish connection to HTTP destination (== Internet-facing API) by URL
53     DATA(lo_http_destination) = cl_http_destination_provider=>create_by_url( 'https://api.somewhere.com/some_service' ).
54
55     "create HTTP client by HTTP destination
56     DATA(lo_web_http_client) = cl_web_http_client_manager=>create_by_http_destination( lo_http_destination ).
57
58     DATA(lo_web_http_request) = lo_web_http_client->get_http_request( ).
59
60     "optional: adding headers with API Key etc.
61     lo_web_http_request->set_header_fields( VALUE #(
62         ( name = 'Content-Type' value = 'application/json' )
63         ( name = 'Accept' value = 'application/json' )
64         ( name = 'APIKey' value = '....' )
65     ) ).
66
67     "optional: User Authentication
68     lo_web_http_request->set_authorization_basic( i_username = '<...>' i_password = '<...>' ).
69
70     "set request method and execute request
71     DATA(lo_web_http_response) = lo_web_http_client->execute( if_web_http_client=>get ).
72
73     "pick up response
74     DATA(lv_response) = lo_web_http_response->get_text( ).
75
76 CATCH cx_http_dest_provider_error cx_web_http_client_error cx_web_message_error.
77     "error handling
78 ENDTRY.
79
80 out->write( |response: { lv_response }| ).
81 ENDMETHOD.

```

Figure 71: Sample Code for External API Consumption



LESSON SUMMARY

You should now be able to:

- Create ABAP applications to consume HTTP APIs from the Internet

Unit 5

Lesson 3

Consuming an SAP API from the Business Hub API



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Create ABAP applications to consume on-premise HTTP and RFC APIs

Consumption of an SAP API Business Hub API in an ABAP Application

The new RESTful ABAP programming model does not provide any more direct access to SAP provided classes, functions, tables or views and so on, as the SAP business data exists in another separated system. As a consequence, data from the SAP business system can only be accessed through released APIs.

The SAP API Business Hub allows you to search for and browse through released APIs. Through a provided sandbox system, you can test an API and find out how it is structured and whether it is suitable for your business needs.

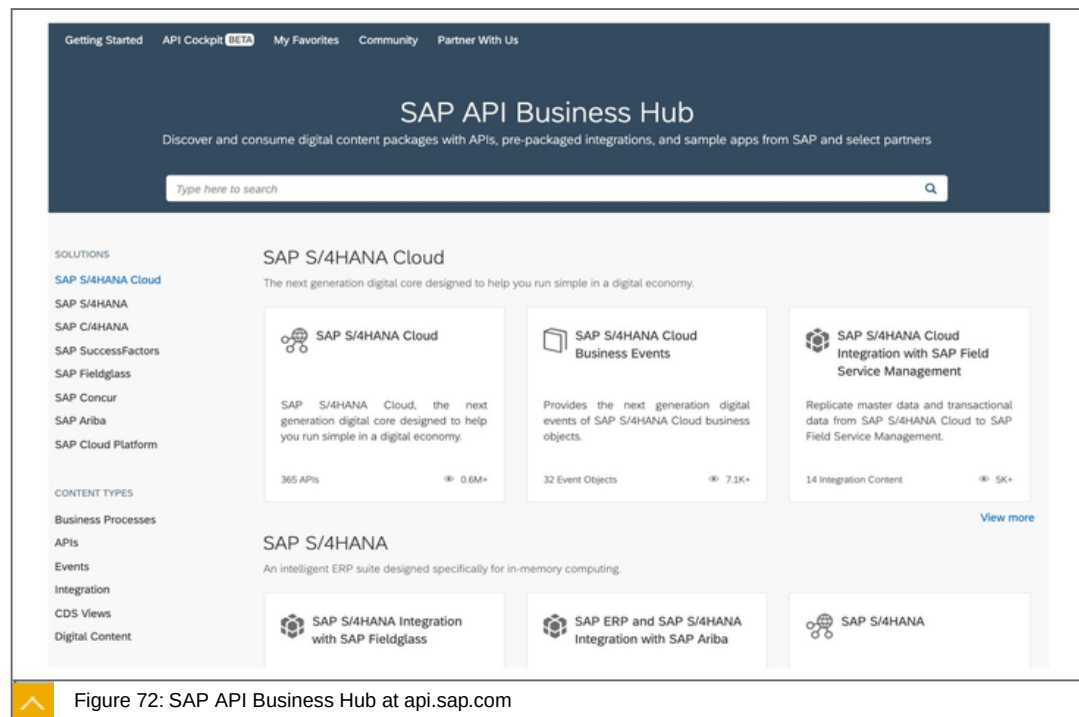
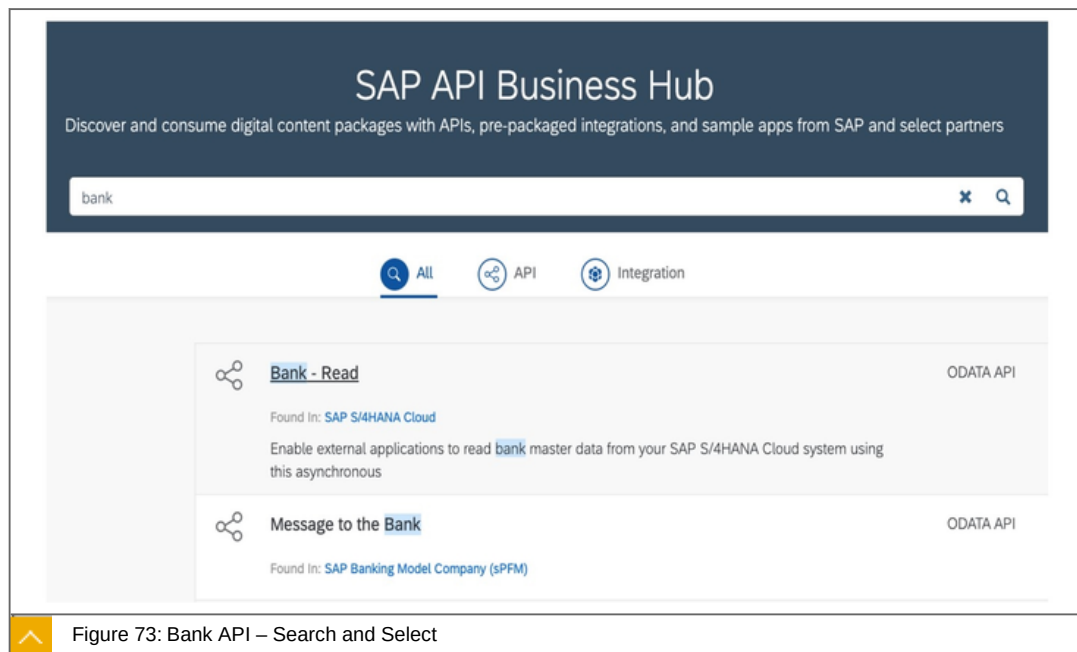


Figure 72: SAP API Business Hub at api.sap.com

The focus of this lesson is the consumption of an API of the OData type to access the API provided data and use it in the application logic of a RESTful ABAP application running in SAP Cloud Foundry. This may work bidirectionally, accessing data from the provisioning tenant system for reading and executing data changing operations in the same provisioning tenant

system. Many APIs support Create, Read, Update, and Delete (CRUD) operations and perhaps additional functions.

As an example here, we select the Bank Read API.



After selecting an API, detail information is displayed, and a number of tools are available:

Configuration of the environment allowing the switch from the SAP provided sandbox system to your own SAP S/4HANA Cloud system.

Code Snippet to get sample code for ABAP and other supported languages to implement the OData request call into your own application coding.

API Key required for testing using the sandbox system.

As required testing with a Try Out button.

Detailed information about the service and its entities.



Bank - Read

Enable external applications to read bank master data from your SAP S/4HANA Cloud system using this asynchronous inbound service

Show API Key Download SDK Add to API Cockpit

API References Details

API Environment: Sandbox

GET **IA_BankDetail** Try out Code Snippet

Get entities from A_BankDetail

Parameters Try out

Name	Description
Top integer (query)	Show only the first n items, see Paging - Top
Skip integer (query)	Skip the first n items, see Paging - Skip

Figure 74: API Details and Tools

To implement the API consumption logic in your ABAP application, an HTTP client object needs to be created. To create this client object, the destination information of the API endpoint needs to be obtained. This can be done using framework-provided utility methods in the following ways:

Create the destination information object using a URL (as already covered in the previous lesson).

Create the destination information object using a configured destination from the Destination Service instance on Cloud subaccount level.

Both options are conveniently provided as sample code in the API hub in the Code Snippet tool.



Bank - Read

Enable external applications to read bank master data from your SAP S/4HANA Cloud system using this asynchronous inbound service

Show API Key Download SDK Add to API Cockpit

API References Details

API Environment: Sandbox

GET **IA_BankDetail** Try out Code Snippet

Get entities from A_BankDetail

Parameters Try out

Name	Description
Top integer (query)	Show only the first n items, see Paging - Top
Skip integer (query)	Skip the first n items, see Paging - Skip

Code Snippet

JavaScript

TRY.

"create http destination by url; API endpoint for API sandbox

DATA(lo_http_destination) =

cl_http_destination_provider=>create_by_url('https://sandbox.api.sap.com/s4hanacloud/sap/opu/odata/sap/API_BANKDETAIL_SRV/A_BankDetail').

"alternatively create HTTP destination via destination service

"cl_http_destination_provider=>create_by_cloud_destination(i_name = '<...>'

i_service_instance_name = '<...>')

"SAP Help: https://help.sap.com/viewer/65de2977205c403bbc107264b8eccf4b/Cloud/en-US/f871712b816943b0ab5e84b68799e518.html

"Available API Endpoints

"https://{host}:{port}/sap/opu/odata/sap/API_BANKDETAIL_SRV

Copy and Close Cancel

Figure 75: Code Snippet Tool for Sample Implementation Code

The first option of providing destination information through an URL would – depending on the API requirements – in many cases require the coding of authentication data directly into the ABAP application. This should be avoided by all means.



```

[H01] ZCL_BANK_READ_BY_URL  [H01] ZCL_BANK_READ_BY_DEST
ZCL_BANK_READ_BY_URL
1 CLASS zcl_bank_read_by_url DEFINITION PUBLIC FINAL CREATE PUBLIC .
2 PUBLIC SECTION.
3 INTERFACES if_oo_adt_classrun.
4 ENDCLASS.
5
6 CLASS zcl_bank_read_by_url IMPLEMENTATION.
7 METHOD if_oo_adt_classrun-main.
8 TRY.
9     DATA(lo_http_destination) = cl_http_destination_provider=>create_by_url(
10         'https://sandbox.api.sap.com/s4hanacloud/sap/opu/odata/sap/API_BANKDETAIL_SRV/A_BankDetail'
11     ).
12
13     "create HTTP client by destination
14     DATA(lo_web_http_client) = cl_web_http_client_manager=>create_by_http_destination( lo_http_destination ).
15
16     DATA(lo_web_http_request) = lo_web_http_client->get_http_request( ).
17
18     "adding headers with API Key for API Sandbox
19     lo_web_http_request->set_header_fields( VALUE #(
20         ( name = 'Content-Type' value = 'application/json' )
21         ( name = 'Accept' value = 'application/json' )
22         ( name = 'APIKey' value = 'lfPtPvAeUX4oU0FyBD02wveJue0urTmT' )
23     ) ).
24
25     "Available Security Schemes for productive API Endpoints
26     "Bearer and Basic Authentication
27     lo_web_http_request->set_authorization_bearer( i_bearer = '<...>' ).
28     lo_web_http_request->set_authorization_basic( i_username = '<...>' i_password = '<...>' ).
29
30     "set request method and execute request
31     DATA(lo_web_http_response) = lo_web_http_client->execute( if_web_http_client=>get ).
32     DATA(lv_response) = lo_web_http_response->get_text( ).
33
34     CATCH cx_http_dest_provider_error cx_web_http_client_error cx_web_message_error.
35     "error handling
36     ENDTRY.
37
38     out->write( |response: { lv_response } | ).
39     ENDMETHOD.
40 ENDCLASS.

```

Figure 76: Create Destination Object by URL

Using destinations from the Destination Service should be the preferred method in most cases. Not only are destinations reusable, they also encapsulate user authentication information and other destination-specific properties.



```

[H01] ZCL_BANK_READ_BY_URL  [H01] ZCL_BANK_READ_BY_DEST
ZCL_BANK_READ_BY_DEST
1 CLASS zcl_bank_read_by_dest DEFINITION PUBLIC FINAL CREATE PUBLIC .
2 PUBLIC SECTION.
3 INTERFACES if_oo_adt_classrun.
4 ENDCLASS.
5
6 CLASS zcl_bank_read_by_dest IMPLEMENTATION.
7 METHOD if_oo_adt_classrun-main.
8 TRY.
9     DATA(lo_http_destination) = cl_http_destination_provider=>create_by_cloud_destination(
10         i_name = 'SAP_BUSINESS_HUB_SANDBOX'
11         i_service_instance_name = 'DESTINATIONS_CLD400'
12         i_authn_mode = if_a4c_cp_service=>service_specific ).
13
14     "create HTTP client by destination
15     DATA(lo_web_http_client) = cl_web_http_client_manager=>create_by_http_destination( lo_http_destination ).
16
17     DATA(lo_web_http_request) = lo_web_http_client->get_http_request( ).
18
19     lo_web_http_request->set_uri_path( |/s4hanacloud/sap/opu/odata/sap/API_BANKDETAIL_SRV/A_BankDetail?format=json| ).
20
21     "adding headers with API Key for API Sandbox
22     lo_web_http_request->set_header_fields( VALUE #(
23         ( name = 'Content-Type' value = 'application/json' )
24         ( name = 'Accept' value = 'application/json' )
25         ( name = 'APIKey' value = 'lfPtPvAeUX4oU0FyBD02wveJue0urTmT' )
26     ) ).
27
28     "set request method and execute request
29     DATA(lo_web_http_response) = lo_web_http_client->execute( if_web_http_client=>get ).
30     DATA(lv_response) = lo_web_http_response->get_text( ).
31
32     CATCH cx_http_dest_provider_error cx_web_http_client_error cx_web_message_error.
33     "error handling
34     ENDTRY.
35
36     out->write( |response: { lv_response } | ).
37     ENDMETHOD.
38 ENDCLASS.

```

Figure 77: Create Destination Object by Cloud Destination Configuration



LESSON SUMMARY

You should now be able to:

Create ABAP applications to consume on-premise HTTP and RFC APIs

Unit 5

Lesson 4

Consuming an On-Premise OData Service



LESSON OBJECTIVES

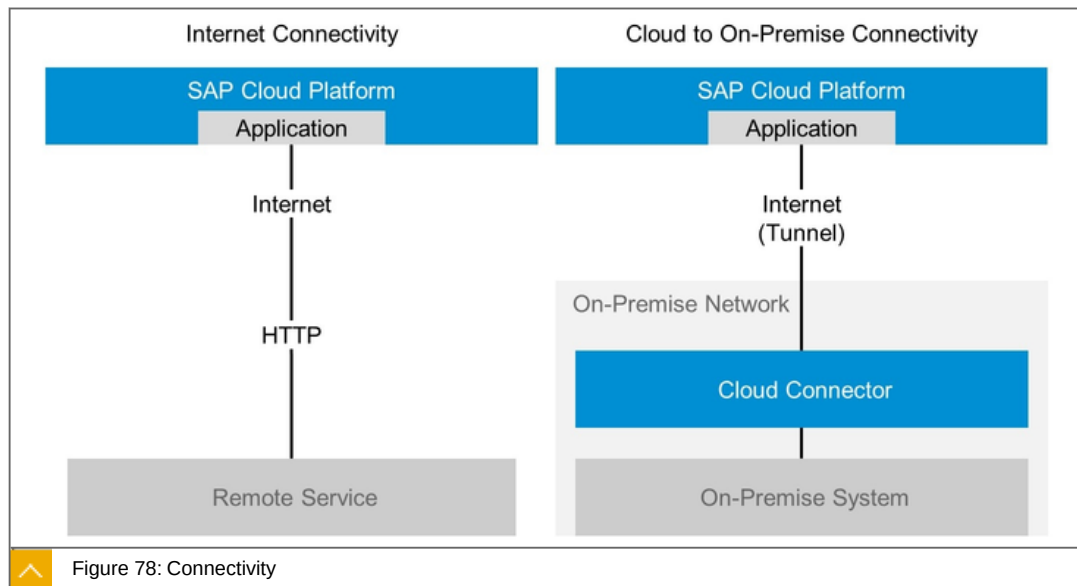
After completing this lesson, you will be able to:

Configure HTTP and RFC destinations

Consumption of On-Premise OData Service

On-premise systems can provide services directly accessible through the internet. An OData service needs to be consumed by an internet facing SAP Fiori application and must therefore be exposed to the internet. Such internet facing OData service consumption is not the focus of this lesson, as it could be simply handled as discussed in the previous lesson by defining a destination of type Internet and providing the HTTP(S) address of your internet facing on-premise server.

In the following example, we want to consume OData services which are not exposed to the internet but have to be consumed using on-premise connectivity.



The Cloud Connector provides the connectivity between the SAP BTP and on-premise systems.

The Cloud Connector has the following features:

Serves as a link between SAP BTP applications and on-premise systems.

- Combines an easy setup with a clear configuration of the systems that are exposed to the SAP BTP.
- Lets you use existing on-premise assets without exposing the entire internal landscape.

Runs as on-premise agent in a secured network.

- Acts as a reverse invoke proxy between the on-premise network and the SAP BTP.

Provides fine-grained control over the following:

- On-premise systems and resources that can be accessed by cloud applications.
- Cloud applications using the Cloud Connector.

Lets you use the features that are required for business-critical enterprise scenarios:

- Recovers broken connections automatically.
- Provides audit logging of inbound traffic and configuration changes.
- Can be run in a high-availability setup.

For more information, see https://help.sap.com/viewer/cca91383641e40ffbe03bdc78f00f681/Cloud/en-US/e6c7616abb5710148cfc3e75d96d596.html#loioe6c7616abb5710148cfc3e75d96d596__context.

The Cloud Connector offers the following benefits when compared to the approach of opening ports in the firewall and using reverse proxies in the DMZ to establish access to on-premise systems:

You do not need to configure the on-premise firewall to allow external access from SAP BTP to internal systems. For allowed outbound connections, no modifications are required.

The Cloud Connector supports HTTP as well as additional protocols. For example, the RFC protocol supports native access to ABAP systems by invoking function modules.

You can use the Cloud Connector to connect on-premise databases or Business Intelligence (BI) tools to SAP HANA databases in the cloud.

The Cloud Connector lets you propagate the identity of cloud users to on-premise systems in a secure way.

Easy installation and configuration, which means that the Cloud Connector comes with a low Total Cost of Ownership (TCO) and is tailored to fit your cloud scenarios.

SAP provides standard support for the Cloud Connector.



LESSON SUMMARY

You should now be able to:

Configure HTTP and RFC destinations

Unit 5

Lesson 5

Consuming an On-Premise RFC



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Create service consumption model for foreign OData service

Consumption of On-Premise RFC

The difference between OData service consumption and RFC execution is that while OData is transported over the HTTP protocol, RFC is not. RFC is based on Common Programming Interface - Communication (CPIC) and is not internet enabled. Therefore, you can only execute an on-premise provided RFC using a Cloud Connector connection.

The OData APIs discussed in the previous lessons were consumed either via a public HTTP connection (which required in the destination definition the Internet proxy type) or the API was not public and required a Cloud Connector connection (establishing a secure tunnel over the Internet) and a corresponding destination definition with the On-Premise proxy type.

Enabling an RFC execution in ABAP applications in Cloud Foundry requires a Cloud Connector configuration that includes exposed back-end RFCs.



The screenshot displays the 'Cloud To On-Premise' configuration page in SAP Cloud Connector. It features four tabs: 'ACCESS CONTROL', 'COOKIE DOMAINS', 'APPLICATIONS', and 'PRINCIPAL PROPAGATION'. The 'ACCESS CONTROL' tab is active, showing a table for 'Mapping Virtual To Internal System'. This table lists two entries, both with a status of 'Reachable'. The first entry maps 's4d.virtual:8000' to 's4dhost:8011' using HTTP. The second entry maps 's4d.virtual:sapgw00' to 's4dhost:sapgw11' using RFC. Below this, the 'Resources Of s4d.virtual:sapgw00' section shows a table with one entry, 'RFC_SYSTEM_INFO', with the naming policy 'Exact Name'. The interface includes various action icons for each entry, such as edit, delete, and refresh.

Status	Virtual Host	Internal Host	Check Result	Protocol	Back-end Type	Actions
Reachable	s4d.virtual:8000	s4dhost:8011	Reachable	HTTP	ABAP System	[Icons]
Reachable	s4d.virtual:sapgw00	s4dhost:sapgw11	Reachable	RFC	ABAP System	[Icons]

Status	Function Name	Naming Policy	Actions
Reachable	RFC_SYSTEM_INFO	Exact Name	[Icons]

Figure 79: Cloud Connector Configuration for Exposure of Functions

The corresponding destination definition in Cloud Foundry requires a destination of type RFC. An RFC destination service uses a Java Connector internally and requires additional properties which have to be explicitly added to the destination definition.

jco.client.ashost = < On premise SAP host as defined in the Cloud Connector >

jco.client.client = < SAP Client >

jco.client.sysnr = < SAP System Instance >

Destination configuration for RFC calls:



Destination Configuration

Name: **S4D_RFC**

Type: **RFC**

Description:

Proxy Type: **OnPremise**

User: **TRAIN-00**

Password: *********

Alias User:

Repository User:

Repository Password:

Location ID: **CLD400**

Additional Properties

New Property

jco.client.ashost	s4d.virtual	
jco.client.client	100	
jco.client.sysnr	00	

Save **Cancel**

Figure 80: RFC destination definition in Cloud Foundry

The `CL_RFC_DESTINATION_PROVIDER` utility class includes the `CREATE_BY_CLOUD_DESTINATION()` public method that is used to instantiate a destination object. This destination object contains the destination name which is required to implement the RFC call in the classic ABAP fashion. As always with RFC executions, communication errors with message texts can be obtained and should be handled in your code.

The following is an coding sample RFC destination:

```
CLASS zcl_system_info_by_rfc_c14 DEFINITION.
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
ENDCLASS.

CLASS zcl_system_info_by_rfc_c14 IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.
    DATA msg TYPE c LENGTH 255.
    DATA lv_result TYPE c LENGTH 200.

    TRY.
      DATA(lo_rfc_dest) =
        cl_rfc_destination_provider=>create_by_cloud_destination( |
S4D_RFC| ).

      DATA(lv_rfc_dest) = lo_rfc_dest->get_destination_name( ).

      CALL FUNCTION 'RFC_SYSTEM_INFO' DESTINATION lv_rfc_dest
        IMPORTING
          rfcsi_export = lv_result
      EXCEPTIONS
        system_failure = 1 MESSAGE msg
        communication_failure = 2 MESSAGE msg
        OTHERS = 3.

      .....

    ENDMETHOD.
  ENDCLASS.
```



LESSON SUMMARY

You should now be able to:

Create service consumption model for foreign OData service

Unit 5

Lesson 6

Using a Remote Client Proxy



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Create and use a custom remote client proxy in ABAP

Remote Client Proxy

In the previous lessons, you have learned how to configure the connectivity between SAP Cloud Platform and APIs on backend systems. For OData based API consumption, the ABAP implementation contained partly cumbersome details about building the HTTP request and handling of the response. With the help of a remote client proxy, the handling of both request and response can be much simplified and encapsulated into a reusable custom-developed API.

A remote client proxy is used when an OData service is consumed remotely using an HTTP request. The client proxy is a powerful means to pass on OData requests to the provisioning tenant. It provides a reusable API for client applications and can be used to call an OData service with a specific OData version (V2 or V4), to consume its business data. The ABAP remote client proxy provides the following:

Request details (such as the entity set to be read with which filter) using the client request object

Response data using the client response object

The following figure illustrates how the query works in the service consumption scenario.

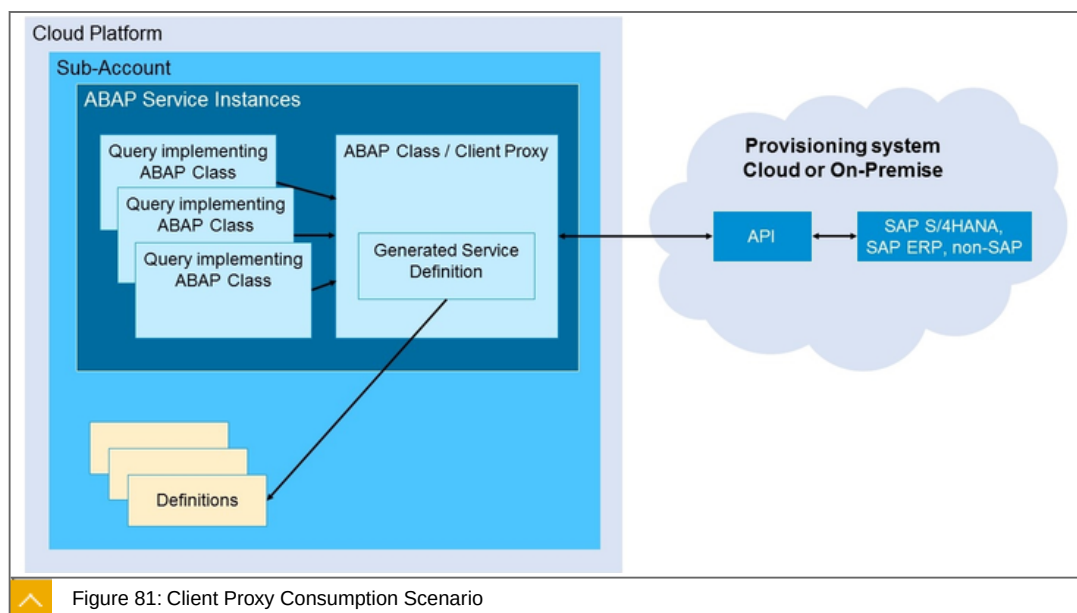


Figure 81: Client Proxy Consumption Scenario

To be able to get data from a provisioning remote service, you build an OData client proxy in your implementation to forward and transform the requests for that remote service. The client proxy can only consume the remote service if a connection to the provisioning tenant is established. The configuration of such a destination including the required communication arrangements is a precondition for developing this scenario.

With the help of the service consumption model, you can import the OData metadata of the remote service into the consuming system and create proxy artifacts of the remote service. This gives you a local representation of the remote data model in your tenant, which helps you to define a data model for the new service.

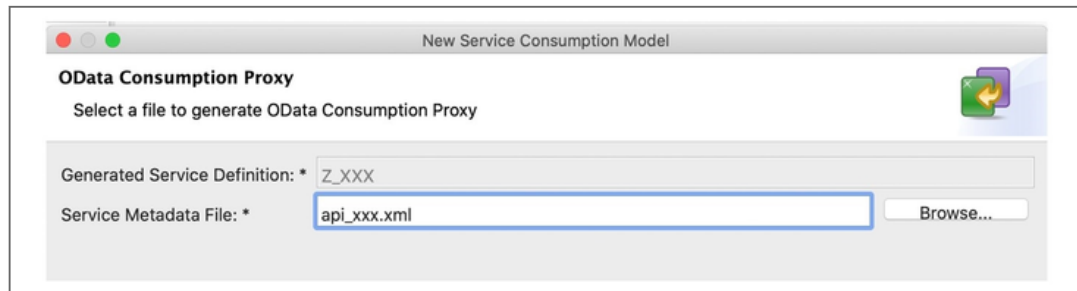


Figure 82: Creation of Service Consumption Model from Metadata File

The client proxy must be instantiated in the query implementation class. All the possible query options – like filtering, selecting, sorting, and paging – of the OData request in the consuming system must then be added to the request for the client proxy. This needs to be done for every query option separately.



```

1 @CLASS zcl_client_proxy_definition DEFINITION PUBLIC FINAL CREATE PUBLIC .
2   PUBLIC SECTION.
3   ... CLASS-METHODS:
4   ... get_client_proxy RETURNING VALUE(ro_client_proxy) TYPE REF TO /iwbp/if_cp_client_proxy
5   ... RAISING zcx_client_proxy.
6   ENDCLASS.
7
8 @CLASS zcl_client_proxy_definition IMPLEMENTATION.
9   METHOD get_client_proxy.
10    TRY.
11      " Get the destination of foreign system & create http client
12      "
13      DATA(lo_http_client) = cl_web_http_client_manager=>create_by_http_destination(
14        cl_http_destination_provider=>create_by_cloud_destination(
15          i_name = |xxx| " <-- your destination
16          i_service_instance_name = |xxx| " <-- SIN of Communication Arrangement
17          i_authn_mode = if_a4c_cp_service=>service_specific
18        ) ).
19      CATCH cx_web_http_dest_provider_error INTO DATA(lx_http_dest_provider_error).
20      CATCH cx_web_http_client_error INTO DATA(lx_web_http_client_error).
21    ENDTRY.
22
23    TRY.
24      " Instantiation of client proxy
25      "
26      ro_client_proxy = cl_web_odata_client_factory=>create_v2_remote_proxy(
27        EXPORTING
28          iv_service_definition_name = |xxx| " <-- the service definition generated with Service Consumption Model
29          io_http_client = lo_http_client
30          iv_relative_service_root = /sap/opu/odata/sap/xxxx/ ).
31    ENDTRY.
32
33    CATCH cx_web_http_client_error INTO DATA(lx_http_client_error).
34    CATCH /iwbp/cx_cp_remote INTO DATA(lx_cp_remote).
35    CATCH /iwbp/cx_gateway INTO DATA(lx_gateway).
36  ENDTRY.
37  ENDMETHOD.
38  ENDCLASS.

```

Figure 83: Client Proxy Definition



```

1= CLASS zcl_client_proxy_usage DEFINITION PUBLIC FINAL CREATE PUBLIC .
2   PUBLIC SECTION.
3     INTERFACES if_oo_adt_classrun.
4   ENDCLASS.
5
6= CLASS zcl_client_proxy_usage IMPLEMENTATION.
7=   METHOD if_oo_adt_classrun-main.
8=     TRY.
9       =====
10      * Instantiate Client Proxy
11      =====
12      DATA(lo_client_proxy) = zcl_client_proxy_definition=>get_client_proxy( ).
13      =====
14      * Create Read Request for entity set *** IMPORTANT: Entity Set in CAPITALS !
15      =====
16      DATA(lo_read_request) = lo_client_proxy->create_resource_for_entity_set( |ENTITY_SET| )->create_request_for_read( ).
17      =====
18      * optional: set Filter, set Select Properties, set Top, set Skip, set Order By
19      =====
20      * lo_read_request->set_filter( .... )
21      * lo_read_request->set_select_properties( .... )
22      * lo_read_request->set_skip( .... )
23      * lo_read_request->set_top( .... )
24      * lo_read_request->set_orderby( .... )
25      =====
26      * execute request
27      =====
28      DATA(lo_web_http_response) = lo_read_request->execute( ).
29      =====
30      * read response object
31      =====
32      DATA: lt_entity_set TYPE TABLE OF za_XXX. " <-- Data Definition from Service Consumption Model
33      =====
34      lo_web_http_response->get_business_data( IMPORTING et_business_data = lt_entity_set ).
35      =====
36      CATCH cx_root.
37      ENDTRY.
38      ENDMETHOD.
39    ENDCLASS.

```

 Figure 84: Client Proxy Usage

Development Steps to Create a New Service to Consume a Remote Service

To create a service that consumes a remote service, complete the following steps:

1. Save the XML file of the provisioning remote service locally (that is, the metadata file).
2. Have proxy artifacts generated with the service consumption model wizard.
3. Create an ABAP class for the Client Proxy Implementation.
4. Get the destination to the provisioning system and create an HTTP client in that Client Proxy class.
5. Create a query implementation ABAP class in which you instantiate the Client Proxy, create the read request, and receive the response.

Exception and Message Handling

In the service consumption scenario, several types of errors might occur. The execution of query or transactional requests is quite complex, as one request must pass not only the consuming service but also the provisioning service. In different parts of your coding, errors can happen. For example, while trying to connect to the provisioning system, while instantiating the client proxy, while executing the unmanaged query, or during the execution of the application logic. Every kind of error must be handled with a suitable exception and message. Therefore, it is convenient to unite all exceptions and their related message in one exception class and one message class.

Before starting with the implementation of the consumption of the remote service, create an exception and a message class. The exception class must inherit from the superclass CX_RAP_QUERY_PROVIDER.



LESSON SUMMARY

You should now be able to:

Create and use a custom remote client proxy in ABAP

Learning Assessment

1. The Connectivity service provides the following services:

Choose the correct answers.

- ☐ **A** Allows you to make connections to on-premises systems, using the Cloud Connector.
- ☐ **B** Allows subaccount specific configuration of application connections via destinations.
- ☐ **C** Provides an API that application developers can use to consume remote services.
- ☐ **D** Lets you establish a secure tunnel from your on-premises network to applications on SAP BTP, while you keep full control and auditability of what is exposed to the cloud.

2. Which methods are offered in the class: CL_HTTP_DESTINATION_PROVIDE to establish a connection to an internet-facing API?

Choose the correct answers.

- ☐ **A** CREATE_BY_DESTINATION
- ☐ **B** CREATE_BY_CLIENT_PROVIDER
- ☐ **C** CREATE_BY_CLOUD_DESTINATION
- ☐ **D** CREATE_BY_URL

3. What are CRUD operations?

Choose the correct answer.

- ☐ **A** Change, Read, Update and Delete
- ☐ **B** Create, Read, Update, and Distribute
- ☐ **C** Create, Read, Update, and Delete

4. The Cloud Connector provides the connectivity between the SAP BTP and on-premise systems.

Determine whether this statement is true or false.

☐ True

☐ False

5. What is the difference between OData service consumption and RFC execution?

Choose the correct answer.

☐ **A** The difference between OData service consumption and RFC execution is, that OData always requires an HTTPS connection.

☐ **B** The difference between OData service consumption and RFC execution is, that RFC is transported over an HTTP protocol.

☐ **C** The difference between OData service consumption and RFC execution is that while OData is transported over the HTTP protocol, RFC is not.

6. What is used to simplify and encapsule both, the request and handling of a reusable custome-developed API?

Choose the correct answer.

☐ **A** A remote client proxy.

☐ **B** The cloud connector.

☐ **C** The remote service.

Learning Assessment - Answers

1. The Connectivity service provides the following services:

Choose the correct answers.

- ☒ **A** Allows you to make connections to on-premises systems, using the Cloud Connector.
- ☒ **B** Allows subaccount specific configuration of application connections via destinations.
- ☒ **C** Provides an API that application developers can use to consume remote services.
- ☒ **D** Lets you establish a secure tunnel from your on-premises network to applications on SAP BTP, while you keep full control and auditability of what is exposed to the cloud.

This is correct. All answer options are correct.

2. Which methods are offered in the class: CL_HTTP_DESTINATION_PROVIDE to establish a connection to an internet-facing API?

Choose the correct answers.

- ☒ **A** CREATE_BY_DESTINATION
- ☐ **B** CREATE_BY_CLIENT_PROVIDER
- ☒ **C** CREATE_BY_CLOUD_DESTINATION
- ☒ **D** CREATE_BY_URL

This is correct. Correct are: "CREATE_BY_DESTINATION", "CREATE_BY_CLOUD_DESTINATION", and "CREATE_BY_URL".

3. What are CRUD operations?

Choose the correct answer.

- ☐ **A** Change, Read, Update and Delete
- ☐ **B** Create, Read, Update, and Distribute
- ☒ **C** Create, Read, Update, and Delete

This is correct. Correct is: Create, Read, Update, and Delete

4. The Cloud Connector provides the connectivity between the SAP BTP and on-premise systems.

Determine whether this statement is true or false.

- ☒ True
- ☐ False

This is correct. The statement is correct.

5. What is the difference between OData service consumption and RFC execution?

Choose the correct answer.

- ☐ **A** The difference between OData service consumption and RFC execution is, that OData always requires an HTTPS connection.
- ☐ **B** The difference between OData service consumption and RFC execution is, that RFC is transported over an HTTP protocol.
- ☒ **C** The difference between OData service consumption and RFC execution is that while OData is transported over the HTTP protocol, RFC is not.

This is correct. The difference between OData service consumption and RFC execution is that while OData is transported over the HTTP protocol, RFC is not.

6. What is used to simplify and encapsule both, the request and handling of a reusable custome-developed API?

Choose the correct answer.

- ☒ **A** A remote client proxy.
- ☐ **B** The cloud connector.
- ☐ **C** The remote service.

This is correct.

UNIT 6

Custom Code Transformation

Lesson 1

Explaining Custom Code on-Premise	106
-----------------------------------	-----

Lesson 2

Performing Custom Code Transformation to SAP BTP, ABAP Environment	109
--	-----

Lesson 3

Using abapGit	115
---------------	-----

UNIT OBJECTIVES

Explain custom code on-premise

Perform custom code transformation to SAP BTP, ABAP environment

Use abapGit

Unit 6

Lesson 1

Explaining Custom Code on-Premise



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explain custom code on-premise

Custom Code On-Premise



You can develop custom code in a very flexible mode:

- **Appends** to the SAP data models
- Implementation of SAP **extension points**
- Implementation of custom objects and applications in **own space**
- **Reuse** of SAP objects is possible (Basis, Appl.)
- **Modification** of SAP code is possible



Tight coupling with SAP standard leads to issues in custom code after upgrades of SAP software

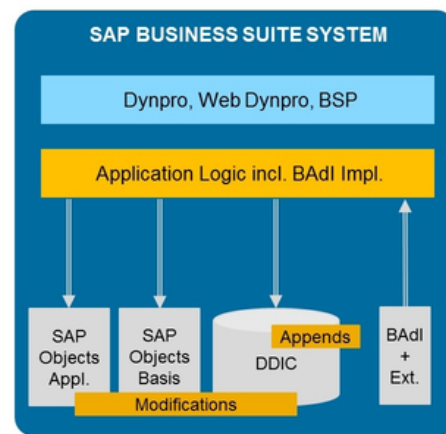


Figure 85: Custom Code Development in SAP Business Suite

With SAP Business Suite, custom code could be developed in a very flexible way. Reuse of nearly all SAP objects was possible. The core was not decoupled from the customer; instead the customer could access the core directly. Even modifications of SAP objects were feasible.

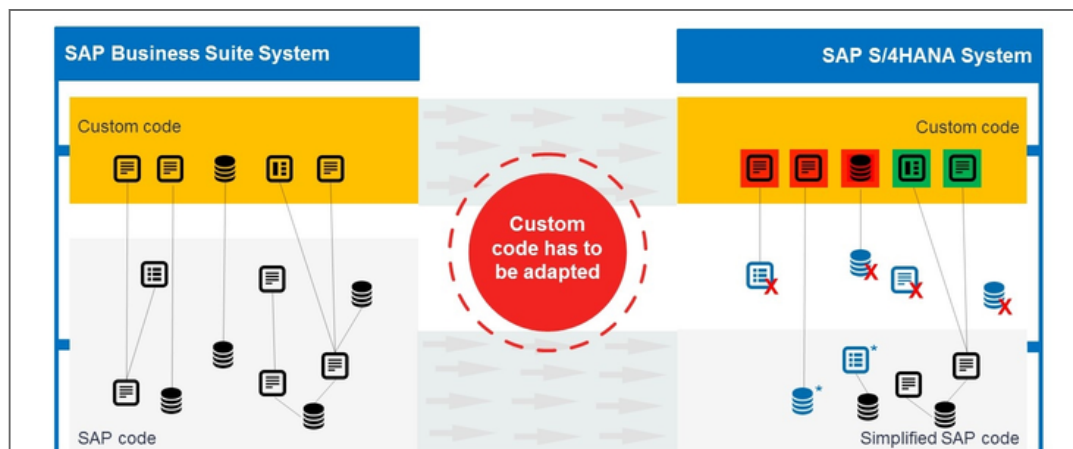


Figure 86: Challenge for Custom Code After SAP S/4HANA System Conversion

With the transition to SAP S/4HANA (on-premise), the question arises of how to keep the custom code of SAP Business Suite for this new generation of ERP system. The main challenges include that some SAP objects are not there any longer, that other SAP objects still exist but have been refactored (for example, other length of material number), and that some SAP data objects shall no longer be accessed directly as there is a new data model that divides between the physical data model and virtual data model where the developer is expected to only use the virtual data model.



Simplification Database contains a list of simplification items

Each simplification item refers to

- An SAP Note that describes the impact and how related custom code can be adapted
- Changed or removed SAP objects

SAP provides tools based on the Simplification Database to detect custom code that needs to be adapted to SAP S/4HANA

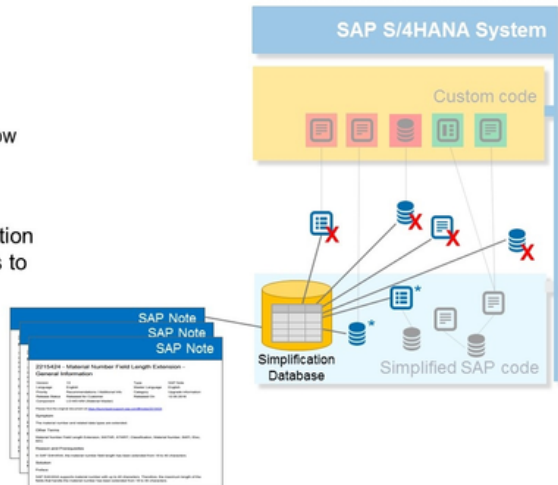


Figure 87: Adapt your Custom Code Using Simplification Database

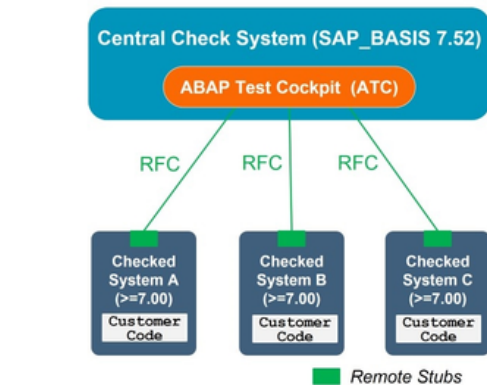
SAP assists its customers in many ways with the transition to SAP S/4HANA.

The simplification database lists simplification items and what to do to keep the custom code as much as possible.



Central ATC system to analyze custom code in whole system landscape

- **Remote stubs** return a model from custom code
- **Check logic** is executed on central system
- **Result** is analyzed in central system
- **Integrated** in development system
- **All kinds of custom code** are supported (incl. modifications and enhancements)
- **Suppress false-positive findings** with pseudo-comments, e.g. for MATNR findings



Find more about [Remote code analysis with ATC](#) in the SAP community

Figure 88: Custom Code Analysis with ABAP Test Cockpit (ATC)

The ABAP Test Cockpit can perform SAP S/4HANA readiness checks on all systems of a system landscape that are involved. The check is triggered from a central system but the result reports items of all included systems.



1. **Setup** remote ATC in central system and implement the SAP Note 2436688
2. **Download** the newest version of the Simplification Database from SAP Service Marketplace (SAP Note 2241080)
3. **Install** the Simplification Database on the central check system (transaction SYCM)
4. **Run** ATC with check variant S4HANA_READINESS_REMOTE
5. **Analyze** the ATC result list

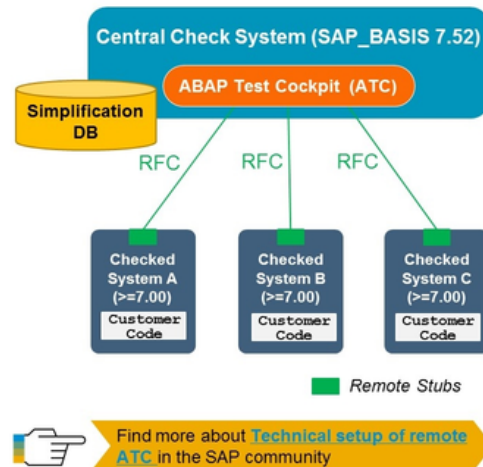


Figure 89: Setup Remote ATC Checks for SAP S/4HANA

The result list contains recommendations for simplification items from the simplification database. There is support for implementing these recommendations.



ABAP Test Cockpit in ADT

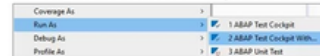
- Tightly integrated in development ("Run As..." menu)
- Browse all ATC check runs of the system (filter by users) using ATC Result Browser
- Display Simplification Item information (e.g. SAP Notes, Referenced Objects)
- Integrate checks in your on-going development activities, e.g. during transport release

Automatic code adaptations

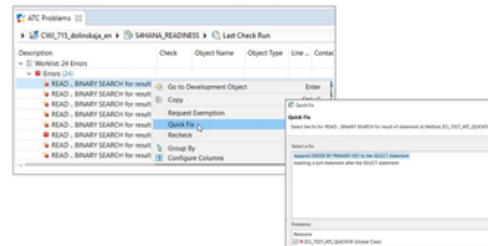
- **Quick Fixes** to minimize adaptation efforts
- Examples: No Order-Check

** Functional adaptation in ABAP Workbench (SE80) still possible with limitations: no automatic code adaptation, CDS and AMDP not supported*

Run ATC in ABAP Development Tools in Eclipse



Quick Fixes in the ATC Problems View



Quick Fixes in the Source Code Editor

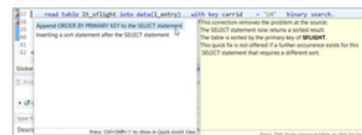


Figure 90: Functional Adaptation in ABAP Development Tools (ADT) in Eclipse

ABAP Test Cockpit is part of the ABAP perspective of ABAP Development Tools.



LESSON SUMMARY

You should now be able to:

Explain custom code on-premise

Unit 6

Lesson 2

Performing Custom Code Transformation to SAP BTP, ABAP Environment



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Perform custom code transformation to SAP BTP, ABAP environment

Custom Code Transformation to SAP BTP, ABAP Environment

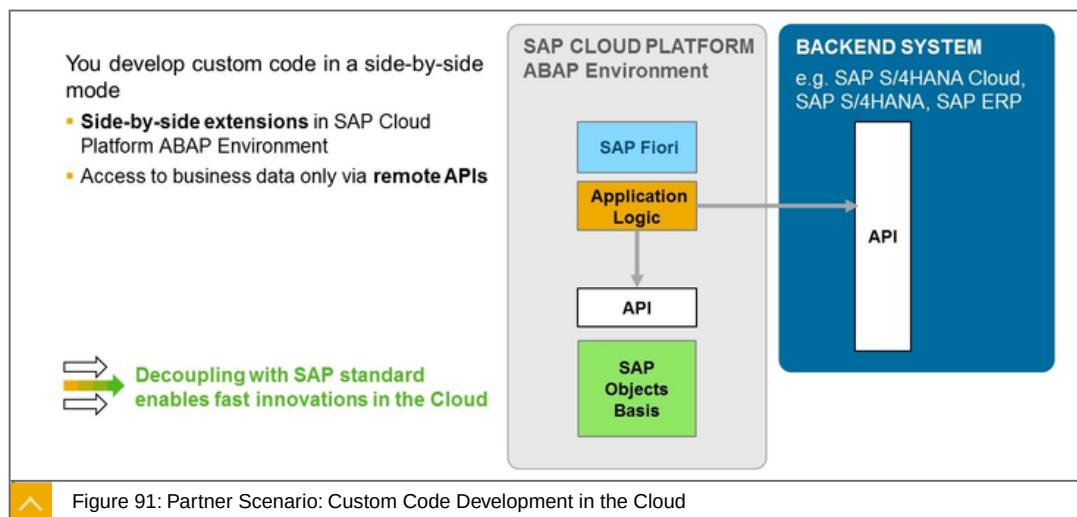


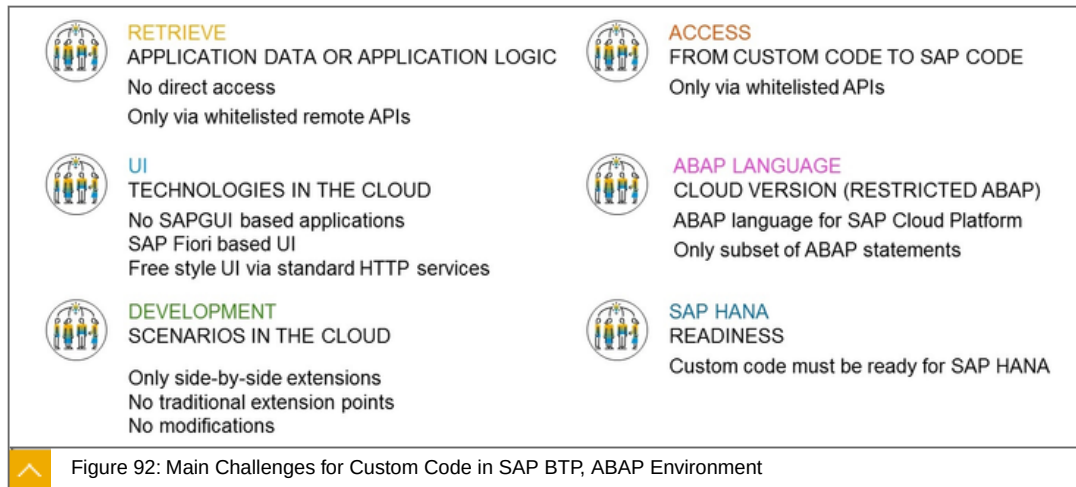
Figure 91: Partner Scenario: Custom Code Development in the Cloud

The ABAP Environment is intended for Side-by-Side extensions. This is especially needed for cloud-based systems, but is also feasible for on-premise solutions. With SAP S/4HANA Cloud there is a real decoupling of the digital core from the customer access as the customer only can call exposed services.

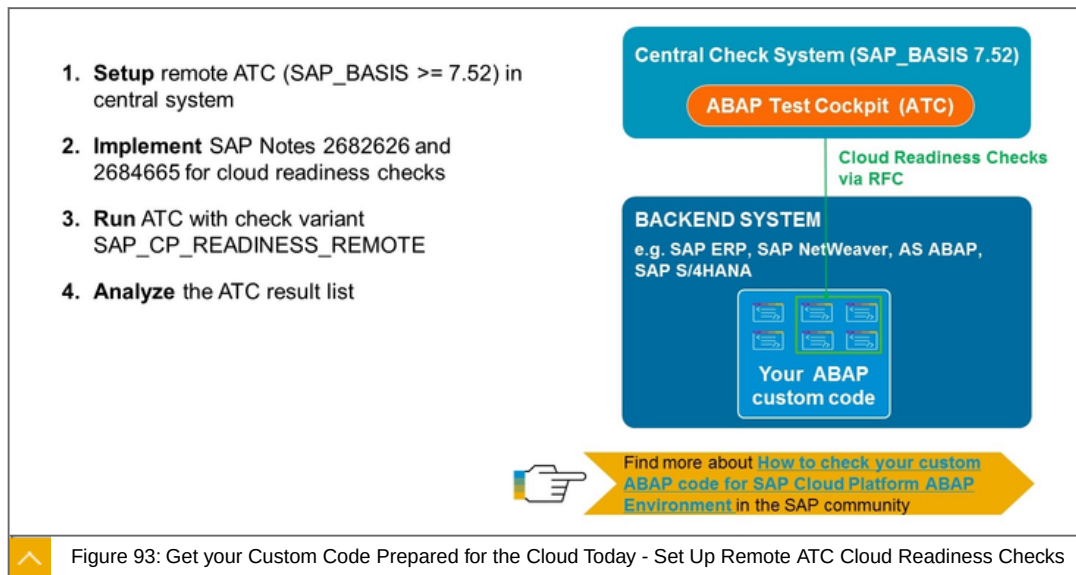
With an SAP S/4HANA on-premise solution, it might be easier to initially use other ways of enhancing (the options of the SAP Business Suite more or less still exist) but on the other hand a potential change from on-premise to cloud would be with less effort if extensions were already implemented in a side-by-side fashion.

Hitherto, the term "extension" was associated with adding a feature to something existing. Now, with a side-by-side extension, it is rather that one uses an SAP function as a part of some own product, and this product offers additional features that go beyond the SAP function.

Besides this, there is still the offer of guided in-app extensibility that is available also for SAP S/4HANA Cloud as for S/4HANA on-premise.

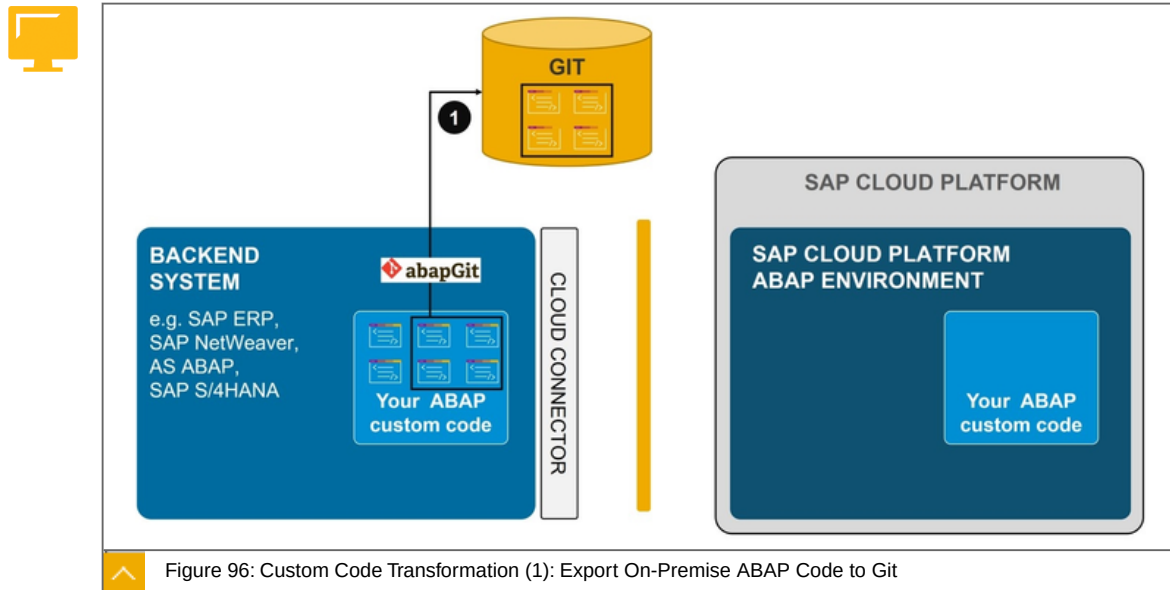


Developing with a side-by-side approach has some challenges compared with development on the back-end system. Among them are restricted access to application data of the back-end system, an ABAP language version that does not allow all features of the standard ABAP language version, and so on.

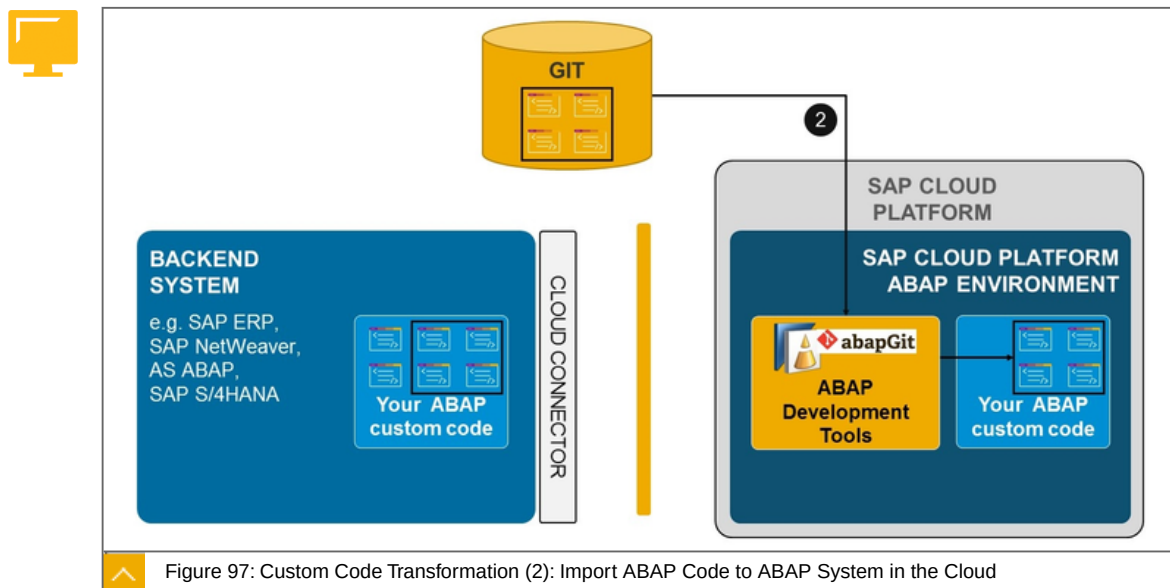


The ABAP Test Cockpit can perform SAP BTP readiness checks. Such a check can be executed from a central check system but the result reports items of all included systems.





There is an ABAP program available that can be installed on an on-premise system. When executed, this program creates a GitHub repository into which metadata of ABAP development objects are stored. (Here, metadata means all information that is part of the definition of a development object.)



On ABAP Development Tools (ADT) the abapGit plug-in can be installed. This allows you to pull information from a GitHub repository in such a way that the ABAP Development Objects that are described there will be created as development objects in ADT.

In a very rough way, this can be compared with the way that the ABAP transportation system works, though it is not the same. One big difference is that the objects that are created with abapGit are originals, not copies like it would be with the ABAP transportation system.

**POSSIBLE ADAPTATIONS:**

- Adjust to ABAP language version 'Cloud'
- Adapt whitelisted APIs
- Correct SAP HANA functional issues (ORDER BY)
- Develop SAP Fiori Uis
- Use ABAP programming model for SAP Fiori apps
- Optimize towards SAP HANA (e.g. code pushdown)

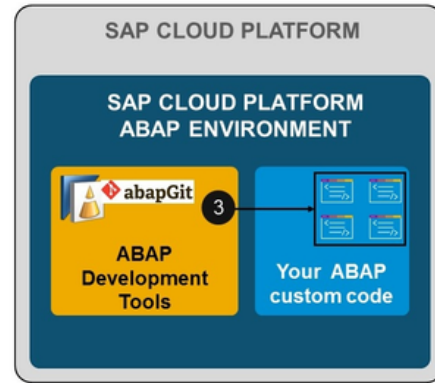


Figure 98: Custom Code Transformation (3): Adapt ABAP Code for Cloud

In many cases, it will be necessary to adopt the development objects that have been created in the ABAP Environment with abapGit, as these objects are still in a state that is appropriate for an on-premise system but not necessarily for the ABAP Environment.

The adaptation can take place after the import of the GitHub repository. The SAP BTP Readiness Check that can already be done on the on-premise system might be very helpful here as the developer can get an overview of what they need to adapt.

Outlook

**Custom Code Migration App in SAP Cloud Platform**

- Checks are executed remotely via Cloud Connector on on-premise system
- Detailed analysis capabilities of required custom code adaptation for the transformation to SAP Cloud Platform ABAP Environment

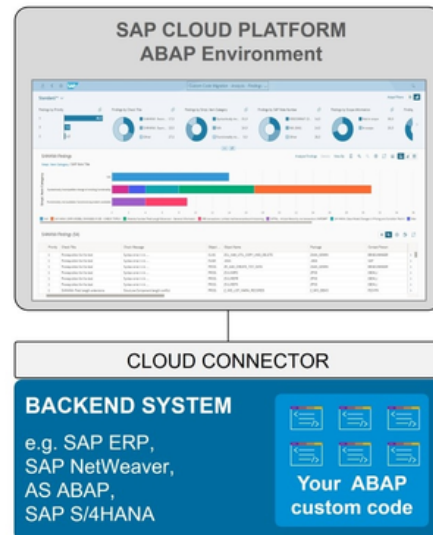


Figure 99: Custom Code Analysis

SAP BTP offers the Custom Code Migration app that can check against an on-premise system what checks need to be done so that the custom coded is suited for the ABAP Environment. This app might be used instead of ABAP Test Cockpit's Cloud Readiness Check.



QuickFixes in ADT in Eclipse to minimize adaptation efforts

- Adapt to ABAP Language Version 5 (e.g. no SUBTRACT/MULTIPLY)
- Adapt to usage of whitelisted objects

Quick Fixes in the ATC Problems View of ABAP Development Tools

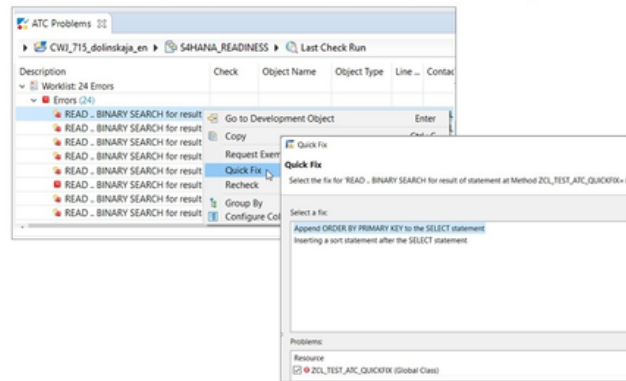


Figure 100: Automatic Code Adaptation

The ATC editor of ADT offers quickfixes for adaptation that help to minimize the effort even more.



LESSON SUMMARY

You should now be able to:

- Perform custom code transformation to SAP BTP, ABAP environment

Unit 6

Lesson 3

Using abapGit



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Use abapGit

abapGit

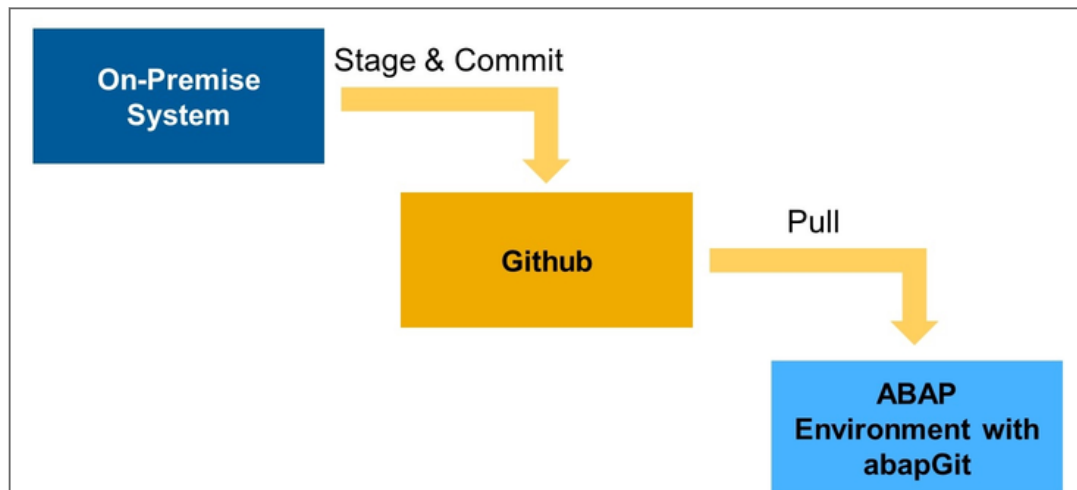


Figure 101: Overview: Utilizing abapGit

Essentially, the following steps need to be considered when working with abapGit:

- 1. Prepare GitHub repository** This needs to be done only once. The repository is the pivot for the transporting actions.
- 2. Push from on-premise to GitHub** The copying of development object meta data to a Git repository on the net.
- 3. Pull from GitHub into ABAP Environment** The copying of development object meta data into the ABAP Environment. Development objects will be created there when new, or will be updated (Git supports version handling as well).
- 4. Adapt within ABAP Environment** Adapt the development objects in such a way that they are fitting for the restrictions of the ABAP Environment.



Create new repository in your github.com account

Our new Terms of Service and Privacy Statement are in effect.

Browse activity Discover repositories

You've been added to the SAPDocuments organization!
Here are some quick tips for a first-time organization member.

- After you switch contexts you'll see an organization-focused dashboard that lists out organization repositories and activities.

Recent activity

SAPDocuments/Tutorials · You commented 8 days ago

Discover interesting projects and people to populate your personal news feed.
Your news feed helps you keep up with recent activity on repositories you watch and people you follow.

Explore GitHub

Figure 102: Preparing GitHub: Create New Repository

At github.com you can create a user account and then within this account, you can create as many public repositories as you wish. Each repository should correspond with one source ABAP package.



Enter a name and description and check *Initialize this repository with a README*

Create a new repository
A repository contains all the files for your project, including the revision history.

Owner: github_user / Repository name: abapgit_workflow

Great repository names are short and memorable. Need inspiration? How about animated-waddle.

Description (optional): Repository for abapGit Workflow Tutorial

Public: Anyone can see this repository. You choose who can commit.

Private: You choose who can see and commit to this repository.

☒ Initialize this repository with a README
This will let you immediately clone the repository to your computer. Skip this step if you're importing an existing repository.

Add .gitignore: None Add a license: None

Create repository

Figure 103: Preparing GitHub: Repository Details

The name of the repository can be chosen arbitrarily.

The repository is only completely created when it contains at least one file. The easiest way to achieve this is to use function *Initialize this repository with a README* – this creates a dummy *readme* file within the repository.



1. Copy the content of the latest build from the program ZABAPGIT, which you will find in the abapGit repository <https://github.com/larshp/abapGit> ⇒ [Readme](#) ⇒ latest build.
2. Open your on-premise system of your choice and create a new program (use another ID, e.g. ZABAPGIT_FULL) and paste the content from step 1 into it.
3. Activate and execute the program.
4. You can find all installation information under <https://github.com/larshp/abapGit> → Documentation/Guides

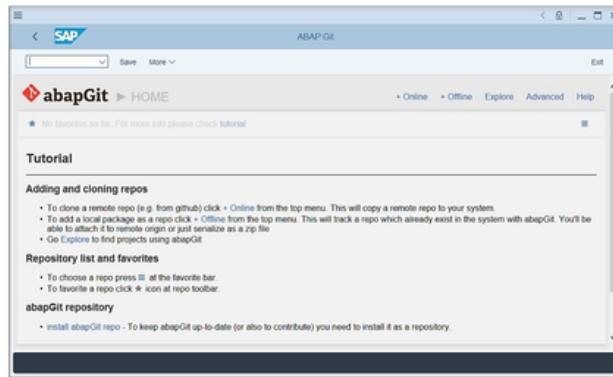


Figure 104: Installing Program ZABAPGIT on the On-Premise System

You can find the source code of program ZABAPGIT on github.com/larshp/abapGit.

Next, create a program on the on-premise system in the customer namespace. It is recommended that you do not name the program ZABAPBIT – name it differently, for example, ZABAPGIT_FULL. This is because it is possible to establish an update routine for newer versions of ZABAPGIT and this update should not happen without a check.



Set Proxy settings under menu *Advanced* ⇒ *Settings*

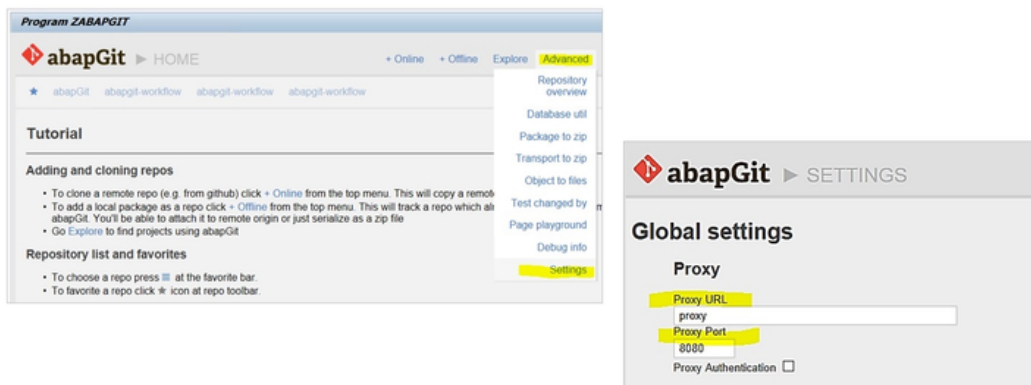


Figure 105: Program ZABAPGIT Settings: Proxy Settings

After installation, the program can be executed. Some settings might be necessary.

Via menu *Advanced* - *Settings*, you can set Proxy attributes.



Find everything under:
Program ZABAPGIT Starting Screen;
Menu **Help** ⇒ **Documentation**

In Documentation:
Choose **Setup** ⇒ **SSL Setup**

There you will find how to create and install certificates.

Add system profile parameters:

```
■ ssl/client_ciphersuites =
  150:PFS:HIGH::EC_P256:EC_HIGH

■ ssl/ciphersuites =
  135:PFS:HIGH::EC_P256:EC_HIGH
```



Figure 106: Program ZABAPGIT Settings: SSL Setup

Typically, the SSL (Secure Socket Layer) setup will not be done by a developer but by a system administrator. Via **Help - Documentation**, you will find the necessary settings.

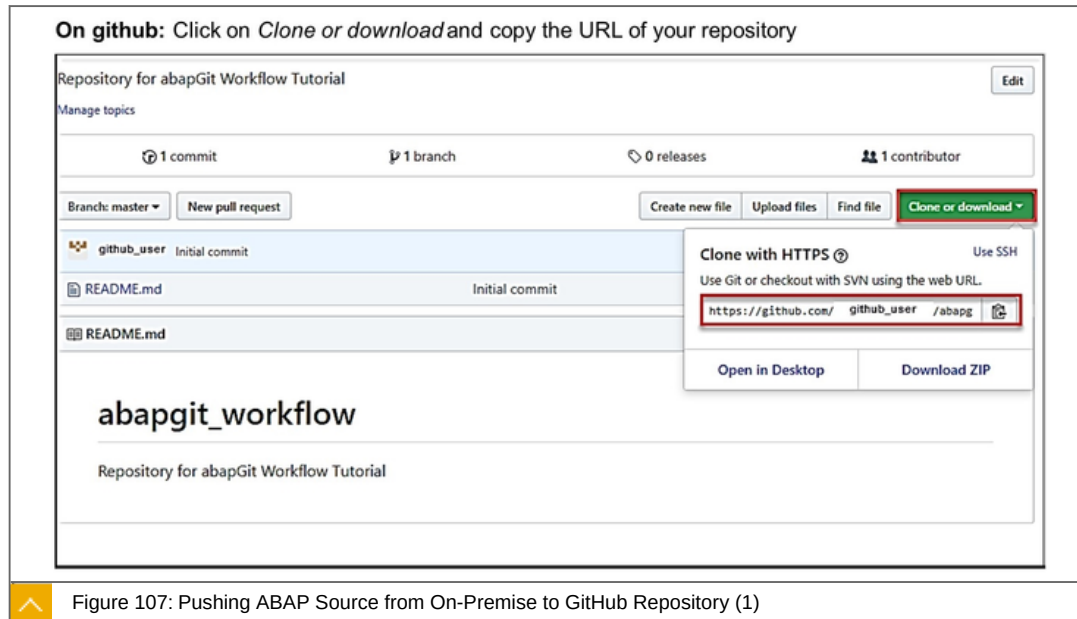


Figure 107: Pushing ABAP Source from On-Premise to GitHub Repository (1)

Prior to executing the ZABAPGIT program, copy the URL of your GitHub repository into your clipboard.



In Program ZABAPGIT: Press the “+ Online” button

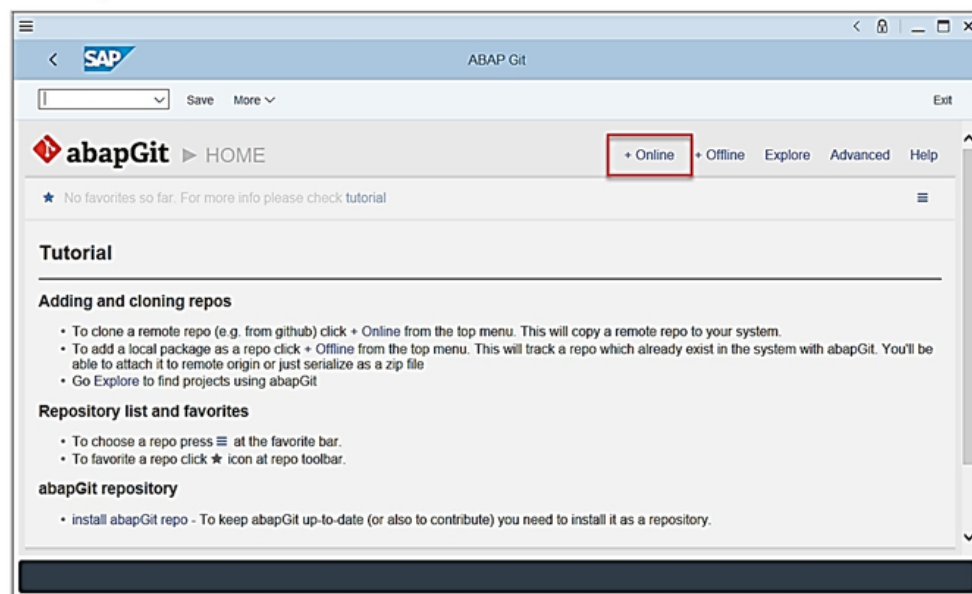


Figure 108: Pushing ABAP Source from On-Premise to GitHub Repository (2)

With the function **+ Online**, the push process starts. (**+ Online** is for creating an online repository. There are other options like **+ Offline** that will not be discussed here.)



Paste the repository URL and click on *Create Package* or enter existing package

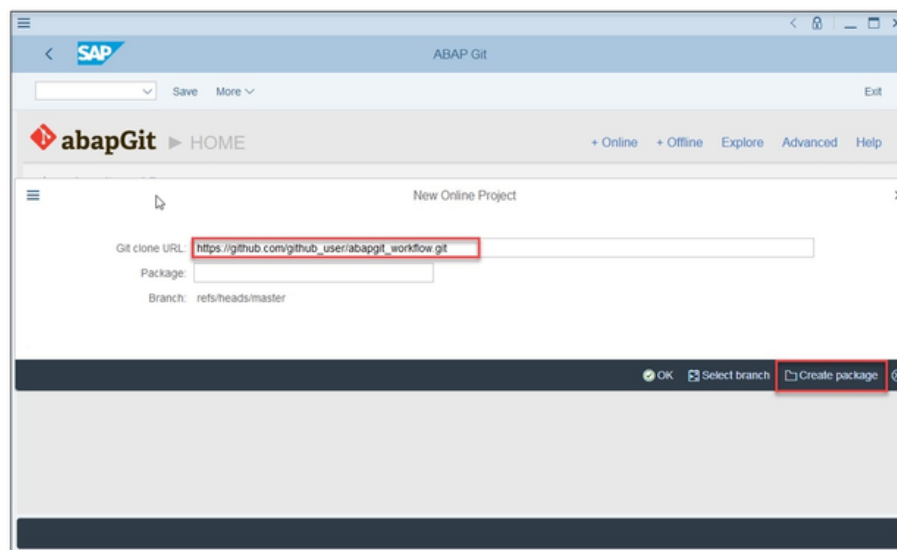


Figure 109: Pushing ABAP Source from On-Premise to GitHub Repository (3)

Enter the GitHub repository's URL and the name of an ABAP package whose development objects are to be transported. Alternatively, create a new empty package.



Set Attributes of package

Assign Development Objects to Package (by Creating, Copying, or Reassignment)

Figure 110: Pushing ABAP Source from On-Premise to GitHub Repository (4)

This figure describes the scenario where you have a new empty package and you add new development objects into it. It is also possible to have an existing package with development objects and the Online repository refers to this package.

Press *Refresh*, then *Stage* developed objects

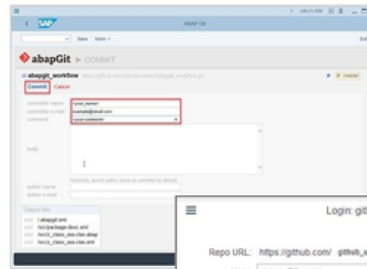
Select single objects to add or *Add all and commit*

Figure 111: Pushing ABAP Source from On-Premise to GitHub Repository (5)

In program ZABAPGIT, the next step is to refresh the list of development objects. You can decide per object whether to add it to the GitHub repository or not. Then, go on with *Commit* or *Add all and commit*.



Enter committer name, committer e-mail and a comment and press **Commit**



Enter your GitHub repository server credentials and click **Execute**



After everything went well, you can see the pushed ABAP objects in your GitHub repository

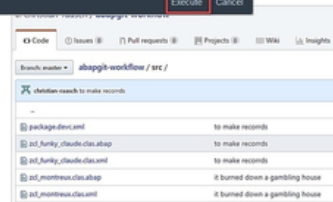


Figure 112: Pushing ABAP Source from On-Premise to GitHub Repository (6)

Enter committer name, committer e-mail address and a comment.

Log on to GitHub with your GitHub user account.

GitHub displays the contents of your repository.

There are other options that are not described here in detail, like updating an existing GitHub repository from ZABAPGIT, as GitHub supports version handling, branch handling, and so on.



- Open your Eclipse with installed ADT.
- Make sure that the abapGit Eclipse plugin is installed, too. It can be installed from <http://eclipse.abapgit.org/updatesite/>. Select *abapGit for ABAP Development Tools (ADT)*.
- Select your cloud project system in the Project Explorer and open the *abapGit repositories* view by opening *Window → Show View → Other ...*
- Expand the category *ABAP* and select *abapGit Repositories* and click *Open*.

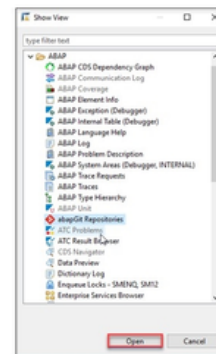
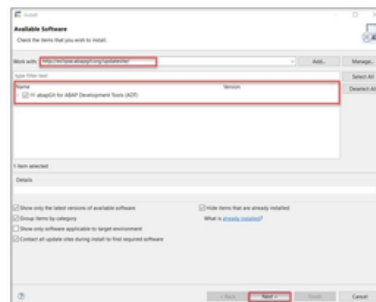


Figure 113: Pulling GitHub Repository into ABAP Environment (1)

Pulling a GitHub repository into the ABAP Environment immediately creates ABAP development objects. To make this possible, the first thing to do in ABAP Development Tools (ADT) is to install the *abapGit Repositories* plug-in. This can be downloaded from eclipse.abapgit.org/updatesite.



Click the *clone button* (green + button) in the abapGit repositories view.

Enter your Git repository URL and press *Next*.

Select Branch and a Package, where your Git repository should be cloned. (If you have no packages, you need to create a new one before) and click *Next*.

Select a Transport Request and click *Finish*.

Your imported sources are now available under your package

In ADT, open the view *abapGit Repositories* . Create a new Git repository (this actually means that you create an entry for the Git repository in GitHub). Now you can import the GitHub repository's content. The ABAP Development Objects are created in the ABAP Environment.

It is not the classical Correction and Transport System (CTS) that is used here. This is a completely independent solution. Technically, the ABAP Development Objects are created as new original objects in the ABAP Environment.



LESSON SUMMARY

You should now be able to:

Use abapGit

Learning Assessment

1. Which of the following statements are true for Custom Code Development in SAP Business Suite?

Choose the correct answers.

- ☐ **A** You can replace an SAP database table with a customer database table.
- ☐ **B** You can reuse SAP development objects.
- ☐ **C** Modification of SAP development objects is possible.
- ☐ **D** You can only access whitelisted APIs so that the digital core is decoupled from customer access.

2. Which of the following is correct for Custom Code Development in the cloud?

Choose the correct answer.

- ☐ **A** You can reuse every SAP development object.
- ☐ **B** Modification is possible.
- ☐ **C** Cloud custom code is developed directly in the cloud back-end system.
- ☐ **D** You develop custom code in a side-by-side mode.

3. Which adaptations are necessary when transforming ABAP Custom Code to SAP BTP, ABAP environment?

Choose the correct answers.

- ☐ **A** Usage of OpenSQL is prohibited.
- ☐ **B** Adjustment to ABAP language version 'Cloud' is necessary.
- ☐ **C** Custom code must be SAP HANA ready.
- ☐ **D** For the user interface, only usage of classical dynpro SAP GUI screens is allowed.

Learning Assessment - Answers

1. Which of the following statements are true for Custom Code Development in SAP Business Suite?

Choose the correct answers.

- ☐ A You can replace an SAP database table with a customer database table.
- ☒ B You can reuse SAP development objects.
- ☒ C Modification of SAP development objects is possible.
- ☐ D You can only access whitelisted APIs so that the digital core is decoupled from customer access.

You are correct! In SAP Business Suite, you can reuse SAP development objects. Modification of SAP development objects is possible.

2. Which of the following is correct for Custom Code Development in the cloud?

Choose the correct answer.

- ☐ A You can reuse every SAP development object.
- ☐ B Modification is possible.
- ☐ C Cloud custom code is developed directly in the cloud back-end system.
- ☒ D You develop custom code in a side-by-side mode.

You are correct! You develop custom code in a side-by-side mode.

3. Which adaptations are necessary when transforming ABAP Custom Code to SAP BTP, ABAP environment?

Choose the correct answers.

- ☐ A Usage of OpenSQL is prohibited.
- ☒ B Adjustment to ABAP language version 'Cloud' is necessary.
- ☒ C Custom code must be SAP HANA ready.
- ☐ D For the user interface, only usage of classical dynpro SAP GUI screens is allowed.

You are correct! It is necessary to adjust to ABAP language version 'Cloud' and the custom code must be SAP HANA ready.

UNIT 7

Transport of Software Components

Lesson 1

Transporting Software Components

126

UNIT OBJECTIVES

Transport software components

Unit 7

Lesson 1

Transporting Software Components



LESSON OBJECTIVES

After completing this lesson, you will be able to:

Transport software components

Software Component Transportation

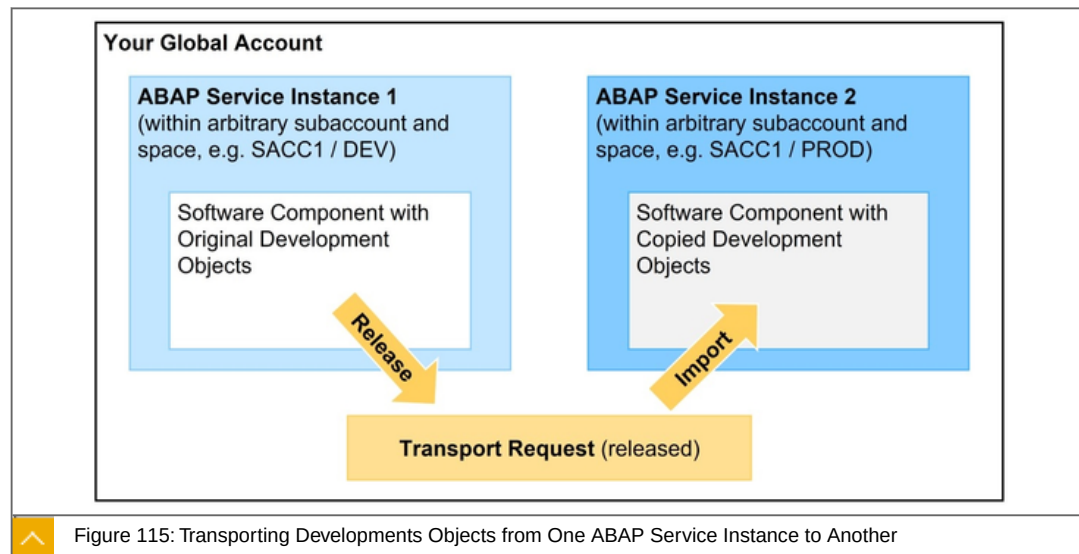


Figure 115: Transporting Developments Objects from One ABAP Service Instance to Another

Transport of ABAP Repository Objects with the Correction and Transport System (CTS) is possible within the same Global Account (meaning for the same customer). ABAP Repository Objects can be transported from one ABAP Service instance to another ABAP Service instance.



Actions on Exporting Side (1):

- To use the transport mechanism, it is necessary to work with Software Components.
- First you have to create a Software Component with an SAP Fiori app from the ABAP Service Dashboard. This implies the generation of a package with the same name as the Software Component.
- Import the Software Component into ABAP Service.

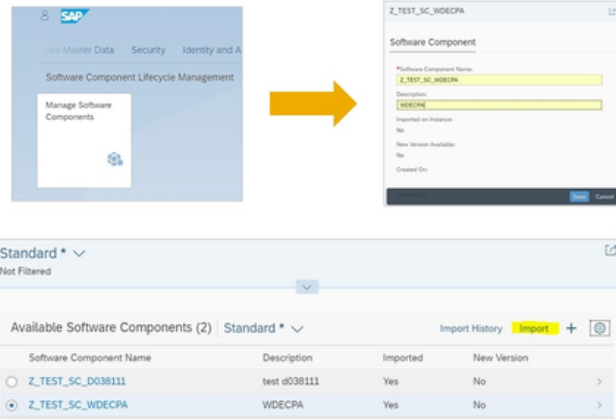


Figure 116: Actions on Exporting Side: Software Component

To make this transport mechanism available between two specific ABAP Service instances, some settings must be configured.

First, you need to create a Software Component on the exporting side. This is done via the ABAP Service's Dashboard with the application *Manage Software Components*.

Create a new software component (name and description) and trigger the function *Import*. With *Import*, the Software Component become available for the transporting mechanism.



- Actions on Exporting Side (2):
- Add Development to Package (Subpackages may be created)
- Release Task
- Release Transport Request

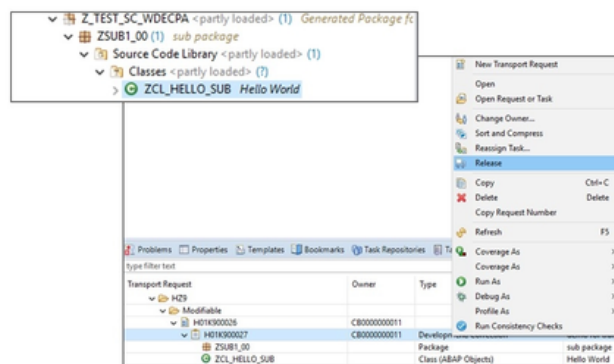


Figure 117: Actions on Exporting Side: Adding Development Objects and Releasing Transport

The Import action implies the creation of an ABAP main package of the same name. To this package, you can now add your own sub packages that contain ABAP repository objects. When maintaining these Repository Objects, the changes will be saved in transport requests. When the project is finished, the transport request must be released.

The typical order of work is as follows: first create the software component (this implicitly creates the main package), then create a development package within this main package, then start creating development objects.



The Importing Side must be another ABAP Service instance within the same Global Account of SAP Cloud Platform.

Actions on Importing Side:

- Go to the Dashboard of the importing ABAP Service. You can find the Software Component that was defined previously on the Exporting Side in the SAP Fiori app "Manage Software Components".
- Import Software Component into this ABAP Service.

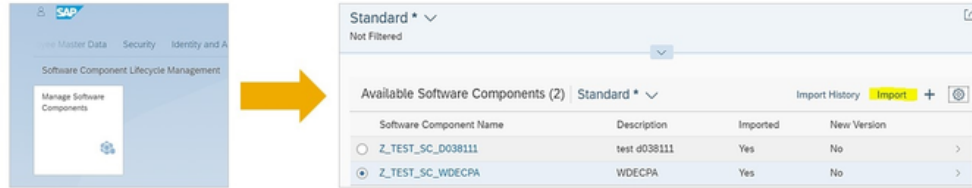


Figure 118: Actions on Importing Side

On the receiving side (the receiving or importing ABAP Service Instance) of the transport request, you have to import the same Software Component with the application *Manage Software Component* of this ABAP Service Instance (this only needs to be done once). The Software Component will already be visible in the list as it is visible throughout the whole global account. After *Import*, the released transport requests and their content will be available within the corresponding ABAP Environment.



LESSON SUMMARY

You should now be able to:

Transport software components

Learning Assessment

1. Which of the following statements about using abapGit is correct?

Choose the correct answer.

- ☐ **A** The program ZABAPGIT must be installed on SAP BTP, ABAP environment.
- ☐ **B** Custom source code can be transported in both directions – from on-premise to cloud and vice versa.
- ☐ **C** The abapGit Eclipse plug-in must be installed in ABAP Development Tools.

2. Which of the following statements regarding transporting software components in SAP CloBAP environment is correct?

Choose the correct answer.

- ☐ **A** Transport of software components is only possible within the same global account.
- ☐ **B** To transport any software components, no actions are necessary in ABAP Development Tools. All actions are performed from the ABAP Service's dashboard.
- ☐ **C** There must exist a transport layer – that is – there is a dedicated development system and a dedicated production system.

Learning Assessment - Answers

1. Which of the following statements about using abapGit is correct?

Choose the correct answer.

- ☐ **A** The program ZABAPGIT must be installed on SAP BTP, ABAP environment.
- ☐ **B** Custom source code can be transported in both directions – from on-premise to cloud and vice versa.
- ☒ **C** The abapGit Eclipse plug-in must be installed in ABAP Development Tools.

You are correct! The abapGit Eclipse plug-in must be installed in ABAP Development Tools.

2. Which of the following statements regarding transporting software components in SAP CloBAP environment is correct?

Choose the correct answer.

- ☒ **A** Transport of software components is only possible within the same global account.
- ☐ **B** To transport any software components, no actions are necessary in ABAP Development Tools. All actions are performed from the ABAP Service's dashboard.
- ☐ **C** There must exist a transport layer – that is – there is a dedicated development system and a dedicated production system.

You are correct! Transport of software components is only possible within the same global account.

UNIT 8

Roadmap of SAP BTP, ABAP Environment

Lesson 1

Exploring the Roadmap of SAP BTP, ABAP Environment

132

UNIT OBJECTIVES

Explore the roadmap of SAP BTP, ABAP environment

Unit 8

Lesson 1

Exploring the Roadmap of SAP BTP, ABAP Environment

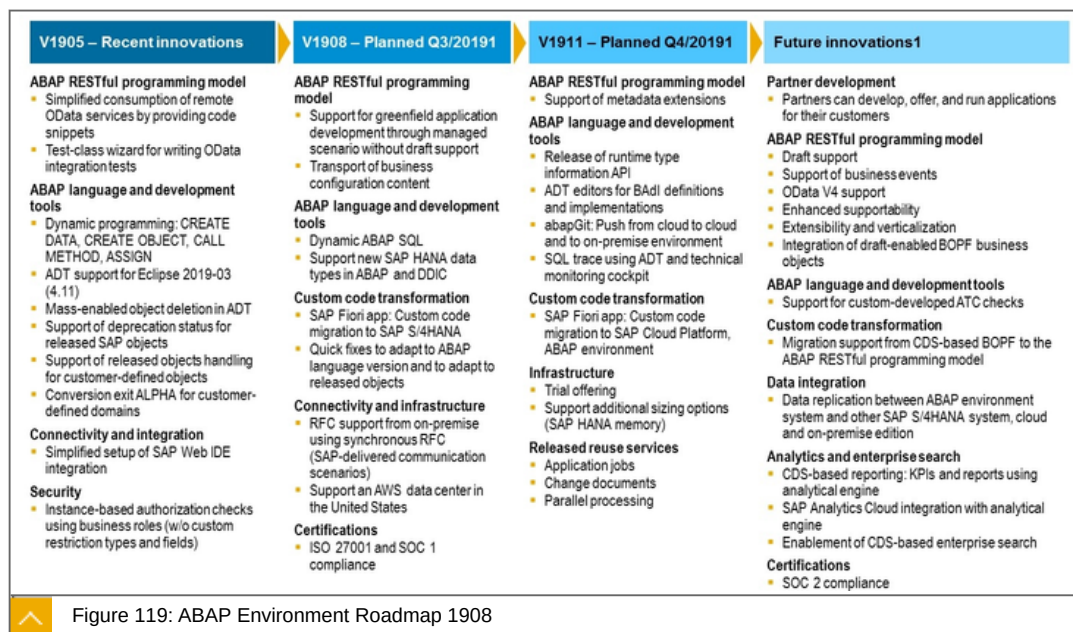


LESSON OBJECTIVES

After completing this lesson, you will be able to:

Explore the roadmap of SAP BTP, ABAP environment

Roadmap of SAP BTP, ABAP Development



Regularly, there will be updates and innovations to the ABAP Environment. The planning for the future includes more options for dynamic programming and improvements for the ABAP RESTful Programming Model.

Please refer to <https://www.sap.com/products/roadmaps.html> to see the newest state of planning. Search for "SAP Cloud Platform ABAP environment".



LESSON SUMMARY

You should now be able to:

Explore the roadmap of SAP BTP, ABAP environment

Learning Assessment

1. The main development directions of the SAP BTP, ABAP environment are more options for dynamic programming and improvements for the ABAP RESTful Programming Model.

Determine whether this statement is true or false.

- ☐ True
- ☐ False

Learning Assessment - Answers

1. The main development directions of the SAP BTP, ABAP environment are more options for dynamic programming and improvements for the ABAP RESTful Programming Model.

Determine whether this statement is true or false.

☒ True

☐ False

This is correct. The statement is correct.